

Table of Contents

Part I. User Guide	9
1. Software Overview	9
2. Software Requirements and Setup	10
2.1 Python environment	10
2.2 Required Python packages	10
2.3 ORCA quantum-chemistry package	11
2.4 Microsoft MPI for parallel execution on Windows (optional)	12
3. Running the SPINKS Workflow	13
3.1 Running SPINKS on a Local PC	14
3.1.1 User input	14
3.1.2 Review of the shared ORCA settings	17
3.1.3 Interactive editing during the workflow	21
3.1.4 Energy range and rate calculations	26
3.1.5 Output files, directory structure, and interactive energy analysis	28
3.2 Running SPINKS on an HPC Cluster	31
3.2.1 Cluster job monitoring, existing outputs, and resubmission	34
Part II. Detailed Computational Implementation of SPINKS	36
4. Step-by-Step Implementation of the SPINKS Workflow	36
4.1 Step 1: Minima Optimization, Interpolation Scan, and MECP Optimization	36
4.1.1 Workflow initialization	36
4.1.2 Collection of molecular and computational input	37
4.1.3 Project-directory organization	38
4.1.4 Construction and review of ORCA inputs	40
4.1.5 Optimization of the two spin-state minima	41
4.1.6 Automatic selection of the reactant reference minimum	43
4.1.7 Construction of the Cartesian interpolation	44
4.1.8 Single-point energies on both spin surfaces	45
4.1.9 Selection of the MECP starting geometry	46
4.1.10 Interpolation-energy visualization	46
4.1.11 Construction of the MECP input	47
4.1.12 Protected MECP optimization	47

4.1.13 Step 1 output variables	48
4.1.14 Files generated by step 1	49
4.1.15 Summary of the step 1 algorithm	50
4.2 Step 2: Orbital Preparation and SOC Calculation at the MECP	51
4.2.1 Initialization	51
4.2.2 High-spin ROHF orbital preparation	52
4.2.3 Automatic SOMO detection	52
4.2.4 UHF/UNO orbital preparation	53
4.2.5 Localized SOMO analysis	54
4.2.6 Selection of the orbital source	54
4.2.7 Automatic construction of the SOC input	55
4.2.8 Extraction of the SOC matrix	56
4.2.9 Construction of the <i>MS</i> -resolved SOC matrix	56
4.2.10 Effective, intermediate, and <i>MS</i> -specific SOC	56
4.2.11 User override of the effective SOC	57
4.2.12 Variables exported by step 2	57
4.2.13 Summary of the step 2 algorithm	58
4.3 Step 3: MECP Energy Gradients	60
4.3.1 Import of quantities from the preceding steps	60
4.3.2 Preparation of the gradient-calculation method	61
4.3.3 Directory preparation	62
4.3.4 Construction of the ORCA Engrad inputs	63
4.3.5 Generation of the two spin-state jobs	63
4.3.6 Execution and protected input repair	64
4.3.7 Location and parsing of the ORCA .engrad files	65
4.3.8 Storage of the spin-state gradients	66
4.3.9 Gradient norms	66
4.3.10 Cartesian gradient-difference vector	67
4.3.11 Crossing-seam normal direction	68
4.3.12 Preliminary assignment of the gradient-gap magnitude	69
4.3.13 Files generated by step 3	69
4.3.14 Variables exported by step 3	70
4.3.15 Summary of the step 3 algorithm	71

4.4 Step 4: Gradient Analysis and Crossing Classification	73
4.4.1 Import of the two MECP gradients	73
4.4.2 Removal of stale projected-gradient variables	74
4.4.3 Surface-gradient-difference convention	74
4.4.4 Reverse surface-gradient convention	75
4.4.5 Gradient-difference magnitude	75
4.4.6 Individual gradient norms	76
4.4.7 Cartesian geometric-mean gradient diagnostic	77
4.4.8 Crossing classification	77
4.4.9 Effective-Hessian weighting parameter	78
4.4.10 Normalized gradient-difference direction	79
4.4.11 Reverse-gradient Cartesian direction	79
4.4.12 MECP-only workflow enforcement	80
4.4.13 Separation of step 4 and the final projected gradient mean	81
4.4.14 Files generated by step 4	81
4.4.15 Variables exported by step 4	82
4.4.16 Summary of the step 4 algorithm	83
4.5 Step 5: MECP Hessians and Rotational Constants	85
4.5.1 Import of quantities from the preceding steps	86
4.5.2 Energy-unit conversion	86
4.5.3 Reuse of reference-state data from step 1	87
4.5.4 Extraction of reference rotational constants and symmetry number	87
4.5.5 Extraction and selection of the reference frequency block	88
4.5.6 Independent calculation of the reference ZPE from frequencies	89
4.5.7 Preparation of the MECP frequency directory	90
4.5.8 Construction of the SurfCrossNumFreq inputs	90
4.5.9 Generation of both spin-surface orderings	91
4.5.10 Execution and protected repair of the MECP frequency jobs	92
4.5.11 Collection of the two Hessian files	93
4.5.12 Extraction of MECP rotational constants and symmetry number	94
4.5.13 Extraction of diagnostic PES2 frequencies	94
4.5.14 Electronic MECP barrier bookkeeping	95
4.5.15 Preparation of downstream variables	96

4.5.16 Distinction between reference frequencies and MECP diagnostic frequencies	97
4.5.17 Files generated by step 5	98
4.5.18 Variables exported by step 5	99
4.5.19 Summary of the step 5 algorithm	99
4.6 Step 6: Effective Hessian and Reduced Mass	102
4.6.1 Import of gradient and Hessian data	103
4.6.2 MECP geometry parsing and atomic-mass assignment.....	103
4.6.3 Validation of the gradient dimensions.....	104
4.6.4 Gradient-difference and gradient-sum vectors.....	105
4.6.5 Gradient-difference magnitude and normalized crossing direction.....	105
4.6.6 Projected gradient components and the final projected gradient mean	106
4.6.7 Direct Cartesian gradient-direction reduced mass	107
4.6.8 Crossing classification	108
4.6.9 ORCA Hessian parsing	108
4.6.10 Mass weighting of the Hessians.....	109
4.6.11 Construction of the peaked-crossing effective Hessian.....	110
4.6.12 Construction of the sloped-crossing effective Hessian.....	110
4.6.13 Selection of the active effective Hessian.....	111
4.6.14 Construction of the translational and rotational subspace	111
4.6.15 Generation of candidate reaction-coordinate seeds.....	113
4.6.16 Removal of rigid-body contamination from each seed	114
4.6.17 Projected zero-mode reaction-coordinate analysis	114
4.6.18 Direct reaction-coordinate reduced mass	115
4.6.19 Automatic selection of the final reaction coordinate.....	116
4.6.20 Distinction among the three reduced masses.....	117
4.6.21 Projection of the effective Hessian into the transverse subspace.....	117
4.6.22 Diagonalization of the projected effective Hessian	118
4.6.23 Conversion of effective-Hessian eigenvalues to frequencies	119
4.6.24 Expected number of effective-Hessian frequencies.....	120
4.6.25 Storage of the two original Hessians	120
4.6.26 Reaction-coordinate candidate diagnostics.....	121
4.6.27 Atomic contributions to the selected reaction coordinate	122
4.6.28 Files generated by step 6	122

4.6.29 Variables exported by step 6	123
4.6.30 Summary of the step 6 algorithm	124
4.7 Step 7: Reaction Coordinate Energy and Velocity Grid	128
4.7.1 Import of quantities from the preceding steps	129
4.7.2 Recovery of the reference zero-point energy	129
4.7.3 Validation of the effective-Hessian MECP frequencies	130
4.7.4 Effective-Hessian MECP zero-point energy	131
4.7.5 MECP–reference zero-point-energy difference	131
4.7.6 Final ZPE-corrected MECP barrier	131
4.7.7 Barrier-source labeling	132
4.7.8 User definition of the maximum excess energy	133
4.7.9 Definition of the common energy spacing	133
4.7.10 Construction of the total-energy grid	134
4.7.11 Identification of the MECP threshold bin	134
4.7.12 Reaction-coordinate excess-energy grid	135
4.7.13 Why the velocity is zero at and below the MECP	136
4.7.14 Conversion of excess energy to joules	137
4.7.15 Conversion of the reduced mass to kilograms	137
4.7.16 Reaction-coordinate velocity in meters per second.....	138
4.7.17 Velocity in bohr per second.....	138
4.7.18 Numerically safe velocity array.....	139
4.7.19 Common-grid consistency with later steps.....	139
4.7.20 Velocity-grid plot	140
4.7.21 Step 7 output directory	141
4.7.22 Energy and velocity summary file	141
4.7.23 Complete energy and velocity grid file	142
4.7.24 Variables exported by step 7	143
4.7.25 Summary of the step 7 algorithm	143
4.8 Step 8: Landau-Zener and Weak Coupling Probabilities	146
4.8.1 Import of active quantities	147
4.8.2 Reuse of the common Step 7 grid	148
4.8.3 Validation of the energy-grid dimensions	148
4.8.4 Verification of the uniform common energy grid.....	149

4.8.5 Verification of the MECP threshold bin	149
4.8.6 Active gradient quantities.....	150
4.8.7 Reduced-mass conversions.....	150
4.8.8 Independent velocity consistency check.....	151
4.8.9 Effective spin-orbit coupling.....	152
4.8.10 Conversion of the SOC to joules.....	152
4.8.11 Energy mask used for the LZ probability	153
4.8.12 Time derivative of the diabatic energy difference	153
4.8.13 Implemented Landau-Zener parameter	154
4.8.14 Implemented Landau-Zener probability	155
4.8.15 Energy dependence of the LZ probability.....	157
4.8.16 Conversion of SOC and excess energy to atomic units	157
4.8.17 Weak coupling prefactor	158
4.8.18 Weak coupling Airy scale	158
4.8.19 Evaluation of the Airy function	159
4.8.20 Subthreshold WC probabilities	160
4.8.21 Effective Landau-Zener probability	161
4.8.22 Effective weak coupling probability.....	161
4.8.23 Selection of the SOC matrix.....	162
4.8.24 Validation of the MS dimensions	162
4.8.25 Individual SOC channel definition	163
4.8.26 Individual channel probability curves	164
4.8.27 No sum over MS -specific channels.....	164
4.8.28 Dependence of individual curves on $ H_{ij} $	165
4.8.29 Grouping equal- $ H_{SO} $ channels for plotting	165
4.8.30 Interpretation of grouped channel labels.....	166
4.8.31 Grouping by $ MS, \text{low} $	167
4.8.32 Row-resolved intermediate SOC magnitude.....	167
4.8.33 Intermediate SOC for a $ MS $ group.....	168
4.8.34 Intermediate probability curves	169
4.8.35 No weighted sum of intermediate curves	170
4.8.36 Plotting ranges	170
4.8.37 Landau-Zener probability plot	171

4.8.38 Weak coupling probability plot.....	171
4.8.39 Step 8 summary file	171
4.8.40 Effective probability grid file	172
4.8.41 Intermediate probability summary	173
4.8.42 <i>MS</i> -resolved channel summary	173
4.8.43 Cluster output transfer	174
4.8.44 Effective probability variables	175
4.8.45 Intermediate and channel-specific variables	176
4.8.46 Exported probability functions	176
4.8.47 Summary of the step 8 algorithm	176
4.9 Step 9: Microcanonical Rate Constants	180
4.9.1 Import of quantities from the preceding steps	180
4.9.2 Selection of the active ZPE-corrected MECP threshold	182
4.9.3 Construction of the common total-energy grid	185
4.9.4 Evaluation of the reactant and MECP rovibrational densities of states	187
4.9.5 Independent evaluation of the rovibrational densities of states	191
4.9.6 Evaluation of the Landau-Zener transition probability.....	194
4.9.7 Calculation of the effective number of crossing states and microcanonical rate constants ..	198
4.9.8 Calculation of the effective Landau-Zener and weak coupling microcanonical rate constants	201
4.9.9 Calculation of the intermediate microcanonical rate constants	204
4.9.10 Calculation of <i>MS</i> -specific microcanonical rate constants	207
4.9.11 Construction of symmetry-collapsed <i>MS</i> -specific rate curves	210
4.9.12 Generation of final rate arrays for canonical calculations	212
4.9.13 Summary of the step 9 algorithm	214
4.10 Step 10: Canonical Rate Constants	217
4.10.1 Import of step 9 microcanonical quantities	217
4.10.2 Construction of the canonical temperature grid	220
4.10.3 Canonical Boltzmann averaging	222
4.10.4 Evaluation of the effective canonical Landau-Zener and weak coupling rate constants	224
4.10.5 Evaluation of the intermediate canonical Landau-Zener and weak coupling rate constants	227
4.10.6 Evaluation of the symmetry-collapsed <i>MS</i> -specific canonical rate constants.....	229
4.10.7 Generation of canonical rate plots	232

4.10.8 Export of canonical results and output files	235
4.10.9 Summary of the step 10 algorithm.....	236
4.11 Step 11: Conditional WC Threshold-Truncation Correction	239
4.11.1 Import of required quantities and conditional workflow setup.....	239
4.11.2 Reconstruction of the raw weak coupling probability.....	240
4.11.3 Application of the weak coupling threshold-truncation rule.....	243
4.11.4 Conditional evaluation of the threshold-corrected effective weak coupling rate	245
4.11.5 Correction of intermediate weak coupling probabilities and rates	247
4.11.6 Correction of symmetry-collapsed MS-specific weak coupling channels	249
4.11.7 Recalculation of canonical weak coupling rate constants.....	251
4.11.8 Diagnostics, output files, plots, and exported variables	253
4.11.9 Summary of the step 11 algorithm.....	256
4.12 Step 12: Results Reporting and Interactive Diagnostics	259
4.12.1 Initialization and helper functions.....	259
4.12.2 Automatic selection of the active weak coupling results.....	263
4.12.3 Reporting of canonical rate constants	265
4.12.4 Interactive microcanonical diagnostic	266
4.12.5 Construction of the final output directory.....	270
4.12.6 Collection of final physical quantities	271
4.12.7 Determination of global probability and rate extrema.....	276
4.12.8 Collection or reconstruction of intermediate spin–orbit couplings.....	278
4.12.9 Construction of the final SPINKS report	281
4.12.10 Validation of quantities required for NAST input.....	285
4.12.11 Construction of the NAST input template.....	286
4.12.12 Generation of the two NAST barrier conventions.....	289
4.12.13 Optional cluster upload and global export	290
4.12.14 Summary of the step 12 algorithm	291
References	295

Part I. User Guide

1. Software Overview

SPINKS is distributed in two implementations: a PC version intended for execution on a personal computer and an HPC cluster version intended for use with a remote high-performance computing environment. Both implementations perform the same electronic-structure and kinetic calculations, but they differ in how ORCA jobs are executed and how files are managed. The PC version launches ORCA directly on the local computer, whereas the HPC cluster version connects to a remote system by SSH/SFTP, generates batch-submission scripts, submits calculations to a queueing system, monitors their progress, and retrieves the resulting files. The current SPINKS implementation was developed and tested on a Windows operating system using Python 3.13 in the Spyder integrated development environment and ORCA 5.0.4. The source code uses platform-independent Python path and subprocess handling and is designed to support execution on Linux and macOS; however, those operating systems were not directly tested in the present work. The complete source code for both implementations is freely available through the official SPINKS GitHub repository:

https://github.com/Hamed-Barzinmehr/SPINKS

2. Software Requirements and Setup

2.1 Python environment

SPINKS was developed and tested using the Spyder integrated development environment distributed through the Anaconda Python distribution. The latest Anaconda distribution can be downloaded from:

```
https://www.anaconda.com/download
```

Because the workflow is organized as sequential Spyder code cells that preserve variables between execution steps, users are strongly encouraged to run SPINKS in Spyder. After installing Anaconda, launch the **Anaconda Prompt** and install **Spyder** if it is not already included with your installation. Spyder may also be installed from the Anaconda Prompt using:

```
conda install spyder -y
```

The workflow described in this work was developed and tested using Python 3.13 in the Spyder IDE distributed with Anaconda. A dedicated SPINKS environment may be created by pasting the following commands in the Anaconda Prompt:

```
conda create -n SPINKS python=3.13 -y  
conda activate SPINKS
```

2.2 Required Python packages

The SPINKS workflow requires the following Python packages:

- **NumPy** (numpy)
- **SciPy** (scipy)
- **Matplotlib** (matplotlib)
- **Spyder** (spyder)
- **Paramiko** (paramiko, required only for the cluster version)

Install the required packages by pasting the following commands in Anaconda Prompt:

```
conda install -n SPINKS numpy scipy matplotlib spyder -y
```

```
conda install -n SPINKS -c conda-forge paramiko -y
```

The **Paramiko** package is required only for the cluster version. Installing it for the cluster-free PC version is harmless and does not affect the operation of the workflow.

2.3 ORCA quantum-chemistry package

SPINKS interfaces directly with the **ORCA** quantum-chemistry package¹ to perform all electronic-structure calculations. Consequently, a working ORCA installation is required before executing the SPINKS workflow.

For the **PC version**, ORCA must be installed locally on the user's computer.

ORCA is freely available for academic use. It can be obtained by registering through the official ORCA website:

```
https://orcaforum.kofo.mpg.de/
```

After creating an account and agreeing to the license conditions, users may download the appropriate ORCA version for their operating system from the ORCA download portal. ORCA distributions are available for Windows, Linux, and macOS. For local execution of SPINKS, users must install the ORCA distribution appropriate for their operating system. The present SPINKS implementation was validated with ORCA on Windows; use with Linux or macOS requires a compatible ORCA installation and may require operating-system-specific configuration of executable permissions, environment variables, and the MPI runtime.

For the **HPC cluster version**, ORCA must be installed on the remote high-performance computing (HPC) cluster. Before using the cluster workflow, users are encouraged to verify whether ORCA is already available on their system. One convenient way is to search for the ORCA executable from the Linux command line using:

```
which orca
```

If the command returns the location of the ORCA executable (for example, `/usr/bin/orca`), ORCA is available in the user's environment. If no path is returned, ORCA may not be installed or may not be loaded into the current environment, and the user should consult the HPC administrator or load the appropriate software module.

2.4 Microsoft MPI for parallel execution on Windows (optional)

Microsoft MPI (MS-MPI) is an optional requirement for the **Windows PC version** of SPINKS. It allows ORCA calculations executed locally on a Windows computer to use more than one processor.

Without Microsoft MPI, the PC version of SPINKS can still be used, but ORCA calculations must be performed using a single processor. When Microsoft MPI is installed and available, users may specify multiple processors and thereby reduce the execution time of suitable ORCA calculations.

Microsoft MPI can be downloaded from the official Microsoft Download Center:

<https://www.microsoft.com/en-us/download/details.aspx?id=105289>

After installation, the mpiexec.exe executable is typically located in:

[C:\Program Files\Microsoft MPI\Bin\mpiexec.exe](#)

Microsoft MPI is not required for the HPC cluster version because parallel execution on the cluster is handled through the MPI implementation provided by the HPC system.

3. Running the SPINKS Workflow

Once the required software has been installed and configured as described in Section 2, the SPINKS workflow is ready to be executed. SPINKS is distributed as two independent Python scripts, both of which are freely available from the official GitHub repository:

https://github.com/Hamed-Barzinmehr/SPINKS

- **SPINKS_PC_version.py** – for local execution using a locally installed ORCA distribution. The implementation was developed and tested on Windows and is designed to support Linux and macOS through platform-independent path handling and automatic detection of the corresponding ORCA executable. Linux and macOS execution has not yet been directly validated by the authors.
- **SPINKS_HPC_Cluster_version.py** – for execution on a remote HPC cluster through SSH/SFTP. The local client implementation was developed and tested on Windows and is designed to support local computers running Linux or macOS; however, these local operating systems have not yet been directly validated by the authors.

After either script is launched, SPINKS guides the user through the entire workflow using a sequence of interactive prompts displayed in the Spyder console. At each stage, the program requests the information required for the next calculation and provides clear instructions regarding the available options, allowing the workflow to be completed without prior knowledge of the internal implementation. The following sections provide a detailed description of each stage of the interactive workflow, the corresponding user prompts, and the calculations performed by SPINKS.

3.1 Running SPINKS on a Local PC

The instructions and screenshots in this section describe the Windows environment used during development and validation. The local implementation also includes operating-system-aware detection of the ORCA executable and platform-independent file handling intended to support Linux and macOS. Because complete SPINKS calculations were not performed on those systems during the present validation, Linux and macOS users may need to adapt environment activation, ORCA installation paths, executable permissions, and MPI configuration to their local setup.

On Windows, open the Anaconda Prompt. On Linux or macOS, open a terminal in which Conda has been initialized. Activate the SPINKS Conda environment by typing:

```
conda activate SPINKS
```

Next, launch the Spyder integrated development environment by typing:

```
spyder
```

Once Spyder has opened, select **File** → **Open** and open the following script:

```
SPINKS_PC_version.py
```

Now the user may execute the script from the beginning. The program will then guide the user through a sequence of prompts to collect the information required to initiate the automated workflow.

3.1.1 User input

The first prompt requests the ORCA installation/extraction directory. This should be the directory containing the ORCA executable. For example, if ORCA was extracted to:

```
C:\ORCA
```

then the user should enter:

```
C:\ORCA
```

On Linux or macOS, the user should instead provide the directory containing the local orca executable. For example:

```
Linux: /home/<username>/orca  
macOS: /Users/<username>/Library/Orca
```

Next, the program asks the user to provide a **project name**. The chosen name is automatically appended to the prefix **Spin_Inversion_** to create the project directory. For example, if the user enters:

Iron_cluster

the project directory created by SPINKS will be:

Spin_Inversion_Iron_cluster

On the validated Windows configuration used in this work, the complete project directory would be:

C:\ORCA\Spin_Inversion_Iron_cluster

On Linux or macOS, the corresponding project directory is created inside the ORCA installation directory supplied by the user. This behavior is implemented using platform-independent Python path handling but has not been directly validated by the authors on those operating systems.

All input files, output files, intermediate calculations, figures, and final results generated during the workflow are organized automatically within this project directory.

The program then prompts the user to specify the two spin multiplicities to be investigated. The multiplicities should be entered as two comma-separated integers. For example:

1,3

requests calculations for the singlet and triplet spin states.

The multiplicity convention adopted by SPINKS follows the standard quantum-chemical definition:

Multiplicity	Spin state
1	Singlet
2	Doublet

3	Triplet
4	Quartet
5	Quintet
6	Sextet
7	Septet
8	Octet
...	Higher multiplicities

Next, the user is asked to specify the molecular charge.

It is important to emphasize that not every combination of charge and spin multiplicity corresponds to a physically meaningful electronic state. The user is therefore responsible for selecting chemically appropriate values. Incorrect charge or multiplicity assignments may cause subsequent ORCA calculations to terminate unsuccessfully.

The program then requests the desired **electronic-structure method (level of theory)**. Because SPINKS interfaces directly with ORCA, any electronic method supported by ORCA may be specified, including density-functional methods, Hartree–Fock, and correlated wavefunction methods. Examples include:

- B3LYP
- PBE
- HF
- MP2

The default electronic method is PBE. Therefore, pressing Enter without typing a value automatically selects PBE.

The user is subsequently asked to specify the basis set. Any basis set supported by ORCA may be used, for example:

- 6-31G
- 6-31G(d)
- def2-SVP
- def2-TZVP

➤ cc-pVTZ

The default basis set is def2-TZVP. Pressing Enter without entering a value automatically selects this basis.

The program next requests the **number of processors** and the **memory per processor** to be allocated for ORCA calculations. On Windows systems, multiple processors may be used only if **Microsoft MPI** has been installed as described in Section 2.4. When Microsoft MPI is detected, the requested number of processors is passed directly to ORCA. If Microsoft MPI is not available, SPINKS automatically forces the calculation to use a single processor, regardless of the number entered by the user. The memory requested at this stage corresponds to the ORCA %maxcore value specified for each processor.

Finally, the program requests the **initial Cartesian geometry** of the molecular system. This is the only molecular input that must be supplied manually by the user. SPINKS displays the following example:

```
atom_label x_coordinate y_coordinate z_coordinate
Example:
O  0.000000000000000  0.00000000818129  0.11690363084563
H  0.000000000000000  0.76528672468584  -0.46747581229803
H  0.000000000000000 -0.76528673286712  -0.46747581854760

Press ENTER on an empty line when the geometry is complete.
```

The user should simply paste the complete Cartesian coordinate block and then press **Enter** on an empty line to indicate that the geometry is complete. Depending on the terminal or Spyder console, the pasted coordinates may not remain perfectly aligned in columns. This formatting change is purely cosmetic and has no effect on the execution of the program. The coordinates should therefore be pasted directly without attempting to manually realign them.

The initial geometry may be generated using molecular editors such as **Avogadro**, **GaussView**, or similar software. A fully optimized structure is **not** required. An approximate chemically reasonable geometry is sufficient because SPINKS automatically performs all required geometry optimizations, interpolation calculations, minimum-energy crossing-point searches, frequency calculations, and subsequent electronic-structure calculations throughout the workflow.

3.1.2 Review of the shared ORCA settings

After the molecular geometry and the basic calculation parameters have been entered, SPINKS automatically generates the ORCA input files required for optimization and vibrational-frequency analysis of the two requested spin-state minima. Before any calculations are submitted, the

complete inputs for both multiplicities are displayed in the Spyder console, and the user is asked whether the shared ORCA settings should be edited.

An example generated input for a singlet minimum is shown below:

Complete generated minimum input | Mn_by_singlet_min.inp

```
! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq

%pal nprocs 12 end
%maxcore 8000

%scf
  MaxIter 800
  SOSCFStart 0.01
  LevelShift 0.5
end

%output
  PrintLevel 3
End

* xyz 0 1
C      -3.20587267   0.34191104 -0.13279854
C      -1.09651374   1.22323127 -0.59633772
C      -1.52218711   2.49658112 -0.17654073
C      -2.84503891   2.67749915  0.26178877
C      -3.70673693   1.57500652  0.28606120
C      -3.98685429 -0.97151299 -0.12878342
C      -3.76823299 -3.24253787 -0.61535409
C      -5.09006267 -3.47359606 -0.19085037
C      -5.87057810 -2.39848438  0.26626241
C      -5.30707130 -1.11885242  0.29861025
H      -0.09177117   1.07410194 -0.93288653
H      -0.84262804   3.32262814 -0.19206589
H      -3.18719275   3.64270672  0.57298589
H      -4.71897750   1.67395907  0.61849059
H      -3.16456312 -4.05311650 -0.96744128
H      -5.49883731 -4.46180399 -0.21651210
H      -6.87946876 -2.55603710  0.58558413
H      -5.87178134 -0.27793621  0.64254377
N      -3.27212686 -1.99876936 -0.57741987
N      -1.96036161  0.19967109 -0.57427770
Mn     -1.54166454 -1.54008291 -1.14614303
C       0.17668970 -0.95226802 -1.82635724
C      -1.04823068 -3.37946219 -1.51176563
C      -0.82807151 -1.80542306  0.63583876
Br      -2.39248738 -1.22371848 -3.27082327
O      -0.40587730 -1.96457005  1.65865916
O       1.16574646 -0.61677431 -2.21580124
O      -0.76681196 -4.43787311 -1.73578431
*
```

The corresponding input for the second multiplicity is generated using the same computational settings but the appropriate spin multiplicity. For example, the triplet input contains:

 Complete generated minimum input | Mn_by_triplet_min.inp

```
! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq

%pal nprocs 12 end
%maxcore 8000

%scf
  MaxIter 800
  SOSCFStart 0.01
  LevelShift 0.5
end

%output
  PrintLevel 3
end

* xyz 0 3
C      -3.20587267   0.34191104  -0.13279854
C      -1.09651374   1.22323127  -0.59633772
C      -1.52218711   2.49658112  -0.17654073
C      -2.84503891   2.67749915   0.26178877
C      -3.70673693   1.57500652   0.28606120
C      -3.98685429  -0.97151299  -0.12878342
C      -3.76823299  -3.24253787  -0.61535409
C      -5.09006267  -3.47359606  -0.19085037
C      -5.87057810  -2.39848438   0.26626241
C      -5.30707130  -1.11885242   0.29861025
H      -0.09177117   1.07410194  -0.93288653
H      -0.84262804   3.32262814  -0.19206589
H      -3.18719275   3.64270672   0.57298589
H      -4.71897750   1.67395907   0.61849059
H      -3.16456312  -4.05311650  -0.96744128
H      -5.49883731  -4.46180399  -0.21651210
H      -6.87946876  -2.55603710   0.58558413
H      -5.87178134  -0.27793621   0.64254377
N      -3.27212686  -1.99876936  -0.57741987
N      -1.96036161   0.19967109  -0.57427770
Mn     -1.54166454  -1.54008291  -1.14614303
C       0.17668970  -0.95226802  -1.82635724
C      -1.04823068  -3.37946219  -1.51176563
C      -0.82807151  -1.80542306   0.63583876
Br     -2.39248738  -1.22371848  -3.27082327
O      -0.40587730  -1.96457005   1.65865916
O       1.16574646  -0.61677431  -2.21580124
O      -0.76681196  -4.43787311  -1.73578431
```

*

After displaying both inputs, SPINKS presents the following prompt:

Edit the shared ORCA settings block before minimum submissions? (Y/N, default N):

If the user answers N, or simply presses Enter, SPINKS proceeds using the automatically generated inputs containing the method, basis set, number of processors, memory allocation, molecular charge, spin multiplicities, and Cartesian geometry previously supplied by the user.

If the user answers Y, SPINKS displays the shared pre-geometry portion of the ORCA input and allows it to be modified before the calculations begin. Only the section preceding the * xyz line is editable at this stage. In an ORCA input, the * xyz line marks the beginning of the molecular charge, spin multiplicity, and Cartesian-coordinate block. The molecular charge, spin multiplicities, and Cartesian geometry are protected because they were defined earlier and must remain consistent throughout the workflow.

The editing stage is intended for users who wish to customize the electronic-structure calculations while maintaining consistent computational settings. For example, the user may:

- modify or remove the default %scf block;
- change SCF convergence parameters such as MaxIter, SOSCFStart, or LevelShift;
- add implicit-solvent settings such as %cpcm or SMD;
- add dispersion corrections or other ORCA keywords to the ! line;
- introduce custom basis-set or effective-core-potential definitions through a %basis block;
- add integration-grid, method, output, or other transferable ORCA blocks.

For example, the default header:

! B3LYP def2-TZVP TightSCF SlowConv Opt Freq

may be changed to:

! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq

to include a dispersion correction. Similarly, an implicit-solvent treatment may be introduced using:

```
%pcm  
  smd true  
  SMDsolvent "CHLOROFORM"  
End
```

A custom basis set or effective-core potential may be added, for example:

```
%basis  
  NewECP Mn "SDD" end  
End
```

Any changes made to the reviewed shared settings are automatically propagated to the subsequent DFT calculations performed by SPINKS. The program adapts the job-specific keywords while retaining the selected method, basis set, SCF settings, solvent model, dispersion correction, and other transferable ORCA blocks. The reviewed settings are therefore used consistently for:

- spin-state minimum optimizations and frequency calculations;
- interpolation single-point calculations;
- minimum-energy crossing-point optimization;
- energy-gradient calculations; and
- numerical-frequency and Hessian calculations.

The SOC calculation is treated separately because it uses a CASSCF-based input with its own job-specific settings and output requirements. The transferable electronic-structure settings are incorporated where appropriate, but the complete SOC input is displayed and reviewed independently, as described in Section 3.1.3.

To preserve the integrity of the automated workflow, the Opt and Freq keywords cannot be removed from the minimum inputs because they are required for geometry optimization and vibrational-frequency analysis of the two spin-state minima. The optimized geometries are used in the subsequent interpolation and MECP searches, while the calculated frequencies provide the zero-point energies required to determine the ZPE-corrected MECP barrier height.

3.1.3 Interactive editing during the workflow

Once the shared ORCA settings have been accepted or modified, SPINKS automatically performs the optimizations of the two spin-state minima, constructs the interpolation pathway between

them, identifies the interpolation geometry with the smallest spin-state energy gap, performs the MECP optimization, and extracts the optimized MECP geometry.

If an ORCA calculation does not terminate normally during the minimum optimizations, interpolation calculations, MECP optimization, orbital-preparation calculations, SOC calculation, energy-gradient calculations, or numerical-frequency calculations, SPINKS pauses the workflow and displays information describing the failed calculation.

For a general failed ORCA calculation, the program displays a message of the following form:

```
=====
ORCA INPUT REVISION REQUIRED
=====
WARNING: ORCA did not terminate normally.
Failed calculation: <calculation label>
```

Current input is shown below:

Paste the corrected input and finish with EOF.

The current ORCA input is then printed in the console. The user may paste a corrected version and terminate the edited input with a line containing only:

EOF

For interpolation calculations, only the pre-geometry ORCA settings are editable because the interpolation geometry is generated automatically. SPINKS displays:

Only the pre-geometry ORCA settings block is editable for interpolation jobs. The geometry, charge, and multiplicity are protected because interpolation uses automatically generated geometries.

Current editable block:

! B3LYP D3 def2-TZVP TightSCF SlowConv

%pal nprocs 12 end

%maxcore 8000

%scf

MaxIter 800

SOSCFStart 0.01

```
LevelShift 0.5  
end
```

```
%output  
PrintLevel 3  
end
```

Modify convergence/header settings only.
Paste the corrected pre-geometry block and finish with EOF.

This allows the user to change convergence settings, SCF parameters, grid settings, or other ORCA options without altering the automatically generated interpolation geometry.

If a revised input violates one of the workflow requirements, SPINKS rejects it and displays:

INPUT VALIDATION FAILED:

<description of the protected setting that was changed>

Please edit again. Finish with EOF.

During MECP optimization, the method, basis set, molecular charge, multiplicities, generated MECP starting geometry, %mecp block, and required SurfCrossOpt keyword are protected. These quantities cannot be removed or altered because they define the crossing-point calculation.

After successful optimization of the MECP, SPINKS performs high-spin ROHF and UHF/UNO orbital analyses and allows the user to choose the orbital reference used to construct the SOC input. The program displays the available options and then requests:

Use which orbitals for the generated SOC input?
Enter ROHF or UHF (default ROHF):

ROHF is the recommended default for most systems, whereas UHF/UNO is provided as an advanced option for difficult open-shell systems.

SPINKS then generates the CASSCF SOC input and displays it for optional editing. The corresponding prompt has the following form:

===== GENERATED Mn_by_SOC.inp =====

! ROHF def2-TZVP TightSCF SlowConv

%pal nprocs 12 end

%maxcore 8000

%scf

MaxIter 800

SOSCFStart 0.01

```

LevelShift 0.5
end

%rel
SOType 3
SOCFlags 1,4,3,0
SOCMaxCenter 4
end

%casscf
maxiter 1
mult 1,3
nroots 1,1
nel 2
nOrb 2

rel
dosoc true
doss false
PrintLevel 3
TPrint 0.01
end
end

%output
PrintLevel 3
end

* xyz 0 3
C -3.169581589396 0.319564681057 -0.090043621650
C -1.154250839700 1.338323631375 -0.606033868267
C -1.634613314236 2.584679389638 -0.248831796464
C -2.942986431434 2.688461536098 0.208458567920
C -3.715177775868 1.543018336537 0.289409603261
C -3.901711002999 -0.949268926711 -0.059039276249
C -3.787575531764 -3.224115865901 -0.488578734228
C -5.096758283324 -3.417407196690 -0.092248957324
C -5.832156481102 -2.317110358155 0.334978063424
C -5.225712739628 -1.074554817138 0.352765447572
H -0.145890512990 1.215367003953 -0.972730826814
H -0.995883250846 3.451923818270 -0.336987810927
H -3.354551306477 3.647316895030 0.493486913722
H -4.736101148133 1.599705482476 0.635594668079
H -3.185425954029 -4.049121584614 -0.839145348221
H -5.528315464363 -4.407440120852 -0.124773426946
H -6.862138047075 -2.427980561706 0.646199471103
H -5.778507689104 -0.206397112976 0.678544845337
N -3.195538041581 -2.022969256159 -0.479446672524
N -1.895776265226 0.227623784656 -0.532362588631
Mn -1.354744300958 -1.617281089802 -1.268305531258
C 0.360949696042 -1.189370574908 -1.899874750833
C -0.874853230279 -3.316707766472 -1.830258914102
C -0.640950296555 -1.928524679190 1.123775383562
Br -2.519316775345 -0.943404656145 -3.444165874538

```

```
O -1.246722227647 -2.486046094930 1.904505754869
O 1.399120475295 -0.953105540011 -2.313725258284
O -0.512396051277 -4.330701306730 -2.222283491588
*
```

```
=====
Edit Mn_by_SOC.inp before local ORCA run? (Y/N, default N):
```

If the user answers N, the generated SOC input is submitted without modification. If the user answers Y, SPINKS displays:

```
Paste the full modified text below.
Finish with a line containing only EOF.
```

Unlike the DFT inputs, the SOC input is intentionally left largely editable. The user may modify the active space, orbital reference, %casscf settings, number of roots, convergence parameters, relativistic settings, the ! line, and additional ORCA blocks as required. However, the basis set, molecular charge, multiplicities, and optimized MECP geometry must remain consistent with the preceding calculations.

After the ORCA SOC matrix has been parsed, SPINKS reports the effective SOC and explains the operator convention used in the calculation. For example:

```
Diagnostic full-matrix norm:
```

```
H_SO_full_matrix_norm_cm = 28.923281 cm^-1
```

```
=====
SOC OPERATOR CONVENTION
=====
```

The ORCA SOC value printed above is obtained with the ORCA mean-field spin-orbit treatment requested by the %rel block, here SOCType 3 with SOCFlags 1,4,3,0 and SOCMaxCenter 4.

The exact Breit-Pauli spin-orbit operator contains one-electron and two-electron terms. ORCA does not evaluate the full Breit-Pauli operator directly in this workflow; instead, it uses an effective one-electron mean-field approximation that includes the dominant Coulomb and exchange contributions to the two-electron SOC operator.

Therefore, ORCA and GAMESS SOC values may differ if GAMESS is using a full Breit-Pauli treatment or a different two-electron SOC convention.

```
Use a user-supplied SOC value instead of the ORCA effective SOC?
(Y/N, default N):
```

The user may therefore retain the effective SOC calculated by ORCA or replace it with a non-negative externally calculated SOC value. In this effective-SOC-only mode, SPINKS calculates only the effective probability and rate curves; intermediate and M_S -specific results are not generated.

The subsequent energy-gradient and numerical-frequency calculations are generated automatically using the previously accepted shared DFT settings and are not displayed again before their initial execution. If either calculation fails, SPINKS pauses and permits the corresponding input to be edited and resubmitted. During these revisions, the required **Engrad** or **SurfCrossNumFreq** keyword, molecular charge, multiplicity, basis set, MECP geometry, and %mecp information are preserved automatically.

3.1.4 Energy range and rate calculations

Following completion of the electronic-structure calculations and construction of the effective-Hessian reaction coordinate, SPINKS proceeds to the nonadiabatic statistical analysis.

The program first asks the user to specify the maximum reaction-coordinate energy above the ZPE-corrected MECP barrier. The following message is displayed:

```
=====
COMMON TOTAL-ENERGY GRID
=====

The entered value is the maximum reaction-coordinate energy above the
ZPE-corrected MECP barrier.

Step 7 converts this value into the common 1 cm-1 total-energy grid
used by Steps 8 and 9.

Maximum reaction-coordinate energy above MECP, cm-1, e.g. 5000:
```

The entered value does not represent the total energy relative to the reference minimum. Instead, it specifies how far the energy grid should extend beyond the ZPE-corrected MECP barrier.

For example, if the ZPE-corrected MECP barrier is:

```
10663.867 cm-1
```

and the user enters:

```
5000
```

the total-energy grid extends from 0 cm⁻¹, relative to the selected reference minimum, to approximately:

15663.867 cm⁻¹

The total-energy grid is generated using a spacing of 1 cm⁻¹. SPINKS then calculates the reaction-coordinate kinetic energy and velocity at each accessible energy, followed by the Landau-Zener and weak coupling spin-crossing probabilities and the corresponding microcanonical rate constants.

After completion of the microcanonical calculations, SPINKS requests the temperature range for canonical averaging. The program displays:

```
=====
CANONICAL TEMPERATURE GRID
=====
```

Canonical averaging formula:

$$k(T) = \frac{\sum k(E) \rho_R(E) \exp[-E/(kBT)] \Delta E}{\sum \rho_R(E) \exp[-E/(kBT)] \Delta E}$$

Energies are relative to the reactant/reference minimum in cm⁻¹.

Minimum temperature for canonical rates, K, e.g. 100:

Maximum temperature for canonical rates, K, e.g. 1000:

Number of temperature points, e.g. 100:

The user specifies the minimum temperature, maximum temperature, and number of temperature points. SPINKS then generates an evenly spaced temperature grid including both endpoints.

For example, the entries

Minimum temperature for canonical rates, K, e.g. 100: 300

Maximum temperature for canonical rates, K, e.g. 1000: 350

Number of temperature points, e.g. 100: 6

produce the temperature grid:

300, 310, 320, 330, 340, and 350 K

The canonical Landau-Zener and weak coupling rate constants are subsequently evaluated at each of these temperatures using the previously calculated microcanonical rate constants and reference-state densities of states.

3.1.5 Output files, directory structure, and interactive energy analysis

Throughout execution, SPINKS automatically organizes all intermediate and final results into a structured directory hierarchy created within the project directory. Separate folders are generated for the principal stages of the workflow, including:

- Minima;
- Interpolation_scan;
- MECP;
- SOC;
- Engrad;
- Frequencies;
- MECP_gradient_definitions;
- Effective_Hessian_RC;
- RC_energy_velocity_grid;
- LZ_WC_probabilities;
- Microcanonical_rates;
- Canonical_rates;
- WC_threshold_correction;
- final_output_and_interoperability_exports.

These directories contain the ORCA input and output files, extracted geometries, Hessians, gradients, SOC matrices, intermediate numerical data, textual summaries, energy and velocity grids, probability tables, rate-constant tables, and publication-quality figures generated during the workflow.

After all probability and rate calculations have been completed, SPINKS reports the active ZPE-corrected MECP barrier and starts an interactive microcanonical diagnostic. For example:

```
===== INTERACTIVE MICROCANONICAL DIAGNOSTIC =====  
  
Active MECP barrier: 10663.867220 cm-1  
  
Enter an energy in cm-1 to print LZ/WC probabilities and rates.  
Type 'stop' to finish.  
  
Energy in cm-1, or 'stop':
```

The energy entered at this stage is the total energy relative to the reference minimum, which is assigned an energy of 0 cm⁻¹. The user may enter any energy from 0 cm⁻¹ to the maximum value included in the common total-energy grid.

For example, the entry

Energy in cm⁻¹, or 'stop': 11663.867

is mapped to the nearest 1 cm⁻¹ energy bin, and SPINKS displays:

===== MICROCANONICAL COMPARISON AT 11664.000 cm⁻¹ =====

Grid energy	= 11664.000 cm ⁻¹
Requested energy	= 11663.867 cm ⁻¹
Energy index	= 11664
Crossing-energy bin index	= 10664
MECP	= 10663.867220 cm ⁻¹
WC source	= Original clipped WC

Density of states:

Reactant state density	= 5.984835003877828e+03
Shifted crossing state density	= 1.342193005282502e+02
Internal crossing state density	= 1.342193005282502e+02

Effective probabilities:

P_LZ	= 5.335188251202716e-03
P_WC	= 6.514828516034694e-03

Effective cumulative states:

N_LZ	= 3.470165216947797e-03
N_WC	= 3.688691535043188e-03

Microcanonical rates:

k_LZ	= 3.815074311094852e+09 s ⁻¹
k_WC	= 4.055320550205378e+09 s ⁻¹
log10(k_LZ)	= 9.581503002
log10(k_WC)	= 9.608025188

In this output, the crossing-energy bin index identifies the discrete total-energy-grid bin corresponding to the MECP threshold. The reactant state density is evaluated on the common total-energy grid, the shifted crossing state density is the crossing-region density mapped onto that grid, and the internal crossing state density is expressed relative to the crossing threshold.

The diagnostic identifies the requested energy, the actual discrete energy bin used by the calculation, the MECP barrier, the reactant and crossing-region densities of states, the effective Landau-Zener and weak coupling probabilities, the corresponding effective numbers of states, and the resulting microcanonical rate constants.

Energies below the MECP may also be entered because the total-energy reference is the selected minimum rather than the MECP. In this region, the crossing may not yet be energetically accessible, and the crossing density of states, effective probabilities, effective numbers of states, and microcanonical rates may therefore be zero or negligibly small.

After displaying the requested data, SPINKS presents the energy prompt again:

Energy in cm⁻¹, or 'stop':

The user may repeat the analysis for as many energies as desired.

Entering:

stop

terminates the interactive diagnostic and causes SPINKS to proceed with generation of the final output files.

After completion of the workflow, SPINKS creates the directory:

final_output_and_interoperability_exports

This directory contains the comprehensive SPINKS final report and the automatically generated NAST-compatible interoperability input files. The report records the calculated SOC matrix and effective SOC, MECP barrier, gradient quantities, effective-Hessian reaction-coordinate properties, reduced mass, reaction-coordinate velocities, Landau-Zener and weak coupling probabilities, densities and cumulative numbers of states, microcanonical rate constants, canonical rate constants, and the sources of the quantities used in the calculations. The generated interoperability files may be used for independent NAST calculations and comparison with the SPINKS results.

3.2 Running SPINKS on an HPC Cluster

The HPC implementation of SPINKS follows the same computational workflow described in Section 3.1, including interactive editing of ORCA input files, automated execution of geometry optimizations, interpolation scans, MECP optimization, spin-orbit coupling calculations, gradient and frequency calculations, and the subsequent evaluation of spin-crossing probabilities and reaction rate constants. Consequently, only the features that differ from the local PC implementation are described below.

In addition to the remote working directories created on the HPC cluster, the cluster version creates a local directory on the computer from which SPINKS is executed. This local directory stores downloaded ORCA files, intermediate data, figures, summaries, and final results. The location is selected automatically according to the operating system:

Windows:

```
C:\SPINKS\Spin_Inversion_<project_name>_cluster_local
```

macOS:

```
/Users/<username>/SPINKS/Spin_Inversion_<project_name>_cluster_local
```

Linux:

```
/home/<username>/SPINKS/Spin_Inversion_<project_name>_cluster_local
```

On macOS and Linux, the local path is designed to be constructed automatically from the current user's home directory. This behavior follows the platform-independent `Path.home()` implementation in the source code. The Windows path was directly validated in the present work, whereas local execution of the cluster client on macOS and Linux has not yet been experimentally tested by the authors. The remote cluster directory is created separately under the remote home directory supplied by the HPC system. Thus, the local operating system determines only where downloaded and processed SPINKS files are stored; the ORCA calculations themselves are executed on the remote HPC cluster. The default remote execution workflow was developed and tested for a Linux-based OpenPBS/PBS cluster. Use with other schedulers or cluster environments requires modification of the submission template, queue-monitoring commands, MPI module, and ORCA executable path.

Unlike the PC version, the cluster implementation first requests the information required to establish a secure connection to the remote computing system. Specifically, the user is prompted to provide the cluster hostname (or login URL), username, and password. These credentials are

used to establish an SSH connection and to transfer the required input files automatically throughout the calculation.

Before the first calculations are submitted, SPINKS also displays the PBS batch script that will be used to execute the ORCA jobs on the cluster. An example is shown below:

```
#!/bin/bash
#PBS -N Mn_by_singlet_min
#PBS -l nodes=1:ppn=12
#PBS -o Mn_by_singlet_min.pbs.out
#PBS -e Mn_by_singlet_min.pbs.err

cd $PBS_O_WORKDIR

echo "=====
echo "Job name: Mn_by_singlet_min"
echo "Working directory:"
pwd
echo "Start time:"
date
echo "=====

module load openmpi/4.1.5
export PATH=/usr/local/apps/orca/5.0.4:$PATH

/usr/local/apps/orca/5.0.4/orca Mn_by_singlet_min.inp > Mn_by_singlet_min.out

echo "=====
echo "End time:"
date
echo "=====
```

Before using SPINKS on a new computing cluster, users should verify that the cluster-specific commands in the batch script are consistent with their local computing environment. In particular, the two most important settings that typically require modification are **(i)** the MPI module specified by the module load command and **(ii)** the ORCA installation directory specified by the export PATH command. These settings are cluster dependent and should be updated to match the software environment available on the target system.

The MPI module specified in the batch script:

```
module load openmpi/4.1.5
```

should be replaced with the MPI module available on the target cluster. If the appropriate module name is unknown, users can list the available OpenMPI modules by executing:

```
module avail openmpi
```

which may produce output similar to:

```
----- /usr/local/Modules/modulefiles -----  
openmpi/4.1.1    openmpi/4.1.1-aocc    openmpi/4.1.5    openmpi/5.0.2    openmpi/5.0.8  
openmpi/5.0.8-borked  
  
----- /cm/shared/modulefiles -----  
openmpi/gcc/64/4.1.8 openmpi4/gcc/4.1.8
```

Users should then select the OpenMPI module appropriate for their computing environment and update the module load command in the batch script accordingly. For example, if openmpi/5.0.2 is available on the target cluster instead of openmpi/4.1.5, replace:

```
module load openmpi/4.1.5
```

with:

```
module load openmpi/5.0.2
```

The ORCA installation directory specified by:

```
export PATH=/usr/local/apps/orca/5.0.4:$PATH
```

should also be verified before running SPINKS on a new computing cluster. Users can first determine the available ORCA version(s) on their system using:

```
module avail orca
```

which may produce output similar to:

```
----- /usr/local/Modules/modulefiles -----  
orca/6.1.1
```

After identifying the desired ORCA version, the corresponding installation directory can be obtained by displaying the module information. For example:

```
module show orca/6.1.1
```

may contain an entry such as:

```
prepend-path PATH=/opt/software/orca/6.1.1:$PATH
```

indicating that the ORCA installation directory is **/opt/software/orca/6.1.1**. The export PATH command in the batch script should then be modified accordingly by replacing:

```
export PATH=/usr/local/apps/orca/5.0.4:$PATH
```

with:

```
export PATH=/opt/software/orca/6.1.1:$PATH
```

Based on the example above, where the available MPI module is **openmpi/5.0.2** and the ORCA installation directory is **/opt/software/orca/6.1.1**, the batch script should be modified as follows:

```
#!/bin/bash
#PBS -N Mn_by_singlet_min
#PBS -l nodes=1:ppn=12
#PBS -o Mn_by_singlet_min.pbs.out
#PBS -e Mn_by_singlet_min.pbs.err

cd $PBS_O_WORKDIR

echo "=====
echo "Job name: Mn_by_singlet_min"
echo "Working directory:"
pwd
echo "Start time:"
date
echo "=====

module load openmpi/5.0.2
export PATH=/opt/software/orca/6.1.1:$PATH

/opt/software/orca/6.1.1/orca Mn_by_singlet_min.inp > Mn_by_singlet_min.out

echo "=====
echo "End time:"
date
echo "=====
```

3.2.1 Cluster job monitoring, existing outputs, and resubmission

SPINKS automatically monitors each submitted calculation by checking both the cluster queue and the corresponding ORCA output file. The cluster queue is examined using the `qstat` command, which reports whether a submitted job is waiting, running, held, suspended, or no longer active.

The principal job states are interpreted as follows:

- **Q:** the job is queued and waiting for computational resources.
- **R:** the job is currently running.
- **H or S:** the job is held or suspended, and SPINKS continues monitoring it.
- **E, or no longer listed in the cluster queue:** the job is no longer active, and SPINKS examines the ORCA output file before deciding what to do next.

SPINKS then responds according to the available output:

- If the output contains ORCA TERMINATED NORMALLY, the calculation is accepted as successfully completed.
- If a normally terminated output already exists when the workflow is restarted, the calculation is skipped and the existing result is reused.
- If the output indicates abnormal termination, SPINKS displays the final 100 lines of the output and allows the user to edit and resubmit the ORCA input.
- If the job is no longer active but no .out file was generated, SPINKS interprets this as a cluster, scheduler, or submission problem and automatically resubmits the unchanged input.
- Automatic missing-output resubmission is attempted up to 50 times. If no output appears after these attempts, the job is assigned a MISSING status.

Before resubmission, files from unsuccessful attempts are moved into timestamped archive directories. This prevents accidental overwriting and preserves failed calculations for later inspection.

Through this monitoring and recovery system, SPINKS can resume interrupted workflows, avoid repeating completed calculations, distinguish ORCA failures from cluster-level failures, and recover automatically from missing-output jobs with minimal user intervention.

Part II. Detailed Computational Implementation of SPINKS

4. Step-by-Step Implementation of the SPINKS Workflow

The NAST gradient conventions, effective-Hessian treatment, weak coupling formulation, density-of-states calculations, and statistical rate expressions implemented in the following sections are based on the theoretical and computational framework described in the NAST package.² SPINKS provides a fully automated workflow that requires only an initial molecular geometry as input and integrates these procedures with the preceding electronic-structure calculations.

4.1 Step 1: Minima Optimization, Interpolation Scan, and MECP Optimization

Step 1 performs the complete sequence required to obtain the optimized minimum-energy crossing point from a single user-supplied Cartesian geometry. The principal operations are:

1. collection and validation of molecular and computational input;
2. generation and optional editing of the ORCA inputs;
3. optimization and frequency analysis of the two requested spin-state minima;
4. automatic selection of the reactant reference minimum;
5. construction of a linear Cartesian interpolation between the optimized minima;
6. evaluation of both spin-state energies at every interpolation geometry;
7. selection of the interpolation point with the smallest absolute energy gap;
8. MECP optimization using the selected geometry; and
9. storage of the geometries, energies, settings, and metadata required by later steps.

The local-PC and HPC implementations use the same electronic-structure inputs and numerical procedures. They differ only in how ORCA calculations are executed, monitored, and transferred. The following discussion describes the common scientific logic using the local implementation as the representative code.

4.1.1 Workflow initialization

The workflow is initialized using:

```
RUN_MODE = "LOCAL"  
workflow_mode = "MECP_ONLY"  
run_mecp_optimization = True
```

The variable RUN_MODE identifies the execution environment, whereas workflow_mode specifies that the initial objective is to locate and characterize the minimum-energy crossing point. The Boolean variable run_mecp_optimization enables the MECP search.

For the local implementation, the ORCA executable is identified from the directory supplied by the user. The function:

```
find_orca_executable(orca_dir_text)
```

searches for orca.exe on Windows or orca on Linux and macOS. If a parent directory is supplied, the code also performs a recursive search for a unique ORCA executable. The resulting paths are stored as:

```
ORCA_EXE  
ORCA_DIR
```

where ORCA_EXE is the complete executable path and ORCA_DIR is the installation directory.

This platform-specific executable detection is implemented in the source code and was verified on Windows. The Linux and macOS branches are intended to support the corresponding ORCA distributions but were not directly exercised during the present validation.

On the validated Windows implementation, SPINKS additionally tests whether Microsoft MPI is available:

```
microsoft_mpi_available()
```

searches the standard Microsoft MPI directories and the system PATH. If MPI is unavailable, the requested number of processors is automatically reset to one to prevent failure of parallel ORCA calculations.

On Linux and macOS, SPINKS does not perform an equivalent automatic validation of the OpenMPI environment. Users on those systems are therefore responsible for ensuring that the MPI version required by their ORCA distribution is installed and configured correctly.

4.1.2 Collection of molecular and computational input

The user supplies the following quantities:

- jobname
- multiplicities
- charge

- method
- basis
- nprocs
- mem_gb
- Initial_geometry

The variable `jobname` defines the project name and is used in all generated filenames and directories. The two spin multiplicities are parsed from a comma-separated entry:

```
multiplicities = parse_int_list(...)
```

SPINKS requires exactly two multiplicities because Step 1 treats a crossing between two spin surfaces.

The lower and higher numerical multiplicities are subsequently stored as:

```
low_spin_mult = min(multiplicities)
high_spin_mult = max(multiplicities)
```

The user also specifies the molecular charge, electronic-structure method, basis set, processor count, and ORCA memory allocation. The default values are:

```
default_method = "PBE"
default_basis = "def2-TZVP"
```

The ORCA `%maxcore` value is stored in megabytes per processor:

```
maxcore_mb = mem_gb * 1000
```

The initial geometry is read as a multiline Cartesian block and stored in:

```
Initial_geometry
```

This initial geometry is used for optimization of both spin-state minima. It does not need to be a stationary point, but the atom order must remain identical for both optimized structures because the subsequent Cartesian interpolation assumes direct correspondence between atomic coordinates.

4.1.3 Project-directory organization

For the local implementation, Step 1 creates a project directory inside the ORCA installation directory:

```
local_base = os.path.join(
    str(ORCA_DIR),
    f"Spin_Inversion_{jobname}"
)
```

Three principal subdirectories are then generated:

```
local_minima = os.path.join(local_base, "Minima")
local_interp = os.path.join(local_base, "Interpolation_scan")
```

```
local_mecp = os.path.join(local_base, "MECP")
```

These directories contain, respectively:

- the two minimum optimization and frequency calculations;
- the Cartesian interpolation geometries and single-point calculations; and
- the MECP optimization and final MECP geometry.

Backward-compatible aliases such as **remote_minima**, **remote_interp**, and **remote_mecp** are assigned to the same local paths so that later workflow steps can use common variable names in both the local and HPC implementations.

For the HPC implementation, the remote project directory is created under the authenticated user's remote cluster home directory:

```
remote_base = posixpath.join(
    remote_home,
    f"Spin_Inversion_{jobname}"
)
```

The remote Minima, Interpolation_scan, and MECP directories are created using POSIX paths because the target HPC system is assumed to use a Linux-compatible filesystem. A separate local project directory is also created on the computer running the SPINKS Python script. The local root is selected automatically according to the operating system:

```
if os.name == "nt":
    local_output_root = Path("C:/SPINKS")
else:
    local_output_root = Path.home() / "SPINKS"
```

The corresponding local project directory is then defined as:

```
local_base = local_output_root / (
    f"Spin_Inversion_{jobname}_cluster_local"
)
```

Therefore, the default local project locations are:

```
Windows:
C:\SPINKS\Spin_Inversion_<jobname>_cluster_local

macOS:
/Users/<username>/SPINKS/Spin_Inversion_<jobname>_cluster_local

Linux:
```

```
/home/<username>/SPINKS/Spin_Inversion_<jobname>_cluster_local
```

The Windows directory behavior was directly validated during development. The macOS and Linux paths follow from the Path.home() implementation and represent the expected directory structure on those systems. Because the complete workflow was not tested on macOS or Linux, these paths should be regarded as the intended implementation rather than experimentally validated platform behavior. The remote directories contain the files required for cluster execution, whereas the local directories contain transferred outputs, processed data, figures, summaries, and final results. pathlib.Path is used for local path construction, while posixpath is used for remote cluster paths.

4.1.4 Construction and review of ORCA inputs

SPINKS defines separate templates for minimum, interpolation single-point, and MECP calculations. The minimum template is represented by:

```
! {method} {basis} TightSCF SlowConv Opt Freq
```

and includes the processor, memory, SCF, output, charge, multiplicity, and geometry placeholders.

The minimum calculations require both Opt and Freq. These keywords are verified by:

```
enforce_minimum_input_requirements(  
    inp_text,  
    job_label  
)
```

The validation function searches the ORCA ! line and rejects an edited input if either keyword is absent. This requirement is enforced because the optimized geometries are needed for interpolation, whereas the vibrational analyses provide the zero-point energies required for reference selection and MECP-barrier correction.

The complete minimum inputs are initially generated using:

```
ORCA_INPUT_TEMPLATES["minimum"].format(  
    method=method,  
    basis=basis,  
    charge=charge,  
    mult=mult,  
    geom=Initial_geometry,  
    nprocs=nprocs,  
    maxcore_mb=maxcore_mb  
)
```

SPINKS displays the two generated inputs and allows the user to edit the shared pre-geometry ORCA settings. The geometry, charge, and multiplicity are protected. The editable portion is separated from the Cartesian block by:

```
split_orca_xyz_block(inp_text)
```

The reviewed settings are stored in:

```
DFT_REVIEWED_PRE_XYZ
```

All transferable ORCA percent blocks are extracted using:

```
extract_orca_percent_blocks_from_pre(...)
```

and stored as:

```
ORCA_TRANSFER_BLOCKS_ALL  
ORCA_TRANSFER_BLOCKS_NO_OUTPUT
```

These captured settings are propagated automatically to later DFT calculations. Job-specific keywords are removed and replaced by the keywords appropriate for each calculation through:

```
adapt_bang_line_for_job(  
    bang_line,  
    job_type  
)
```

For example:

```
job_type == "minimum"
```

adds Opt Freq;

```
job_type == "single_point"
```

removes geometry-optimization and frequency keywords; and

```
job_type == "mecp"
```

adds SurfCrossOpt.

This approach allows the method, basis set, solvent model, dispersion correction, SCF settings, numerical integration settings, and other transferable blocks to remain consistent throughout Step 1 while adapting the input to each specific ORCA job.

4.1.5 Optimization of the two spin-state minima

A separate optimization and frequency calculation is generated for each requested multiplicity. The corresponding job names are created as:

```
job_i = f"{jobname}_{spin_name}_min"
```

where spin_name is assigned using:

```
multiplicity_name(mult)
```

For example, multiplicities 1 and 3 are mapped to singlet and triplet.

Each calculation is executed through:

```
submit_and_monitor_orca_job(...)
```

In the local version, this function calls ORCA through a blocking Python subprocess. The job is considered successful only if the output contains:

```
ORCA TERMINATED NORMALLY
```

If a calculation fails, SPINKS:

- displays the last 100 lines of the output;
- asks whether the user wishes to revise the input;
- validates the revised input;
- archives the failed files; and
- reruns the calculation in the same job directory.

For minimum calculations, the repair policy is:

```
repair_policy = "free_minimum"
```

which permits extensive editing but retains the requirement that both Opt and Freq remain present.

After successful completion, Step 1 extracts the electronic energy and zero-point energy from the ORCA output:

```
ele, zpe = extract_ele_zpe_from_text(  
    out_text,  
    out_path  
)
```

The optimized Cartesian geometry is extracted from the ORCA-generated XYZ file:

```
geom = extract_geometry_from_xyz_text(  
    xyz_text,  
    xyz_path  
)
```

The results for each multiplicity are stored in the dictionary:

```
minima_data[mult]
```

with the structure:

```
minima_data[mult] = {  
    "spin_name": spin_name,  
    "jobname": job_i,  
    "Electronic_Eh": ele,
```

```

    "ZPE_Eh": zpe,
    "E_plus_ZPE_Eh": ele + zpe,
    "geometry": geom
}

```

Thus, minima_data becomes the main Step 1 data structure containing the optimized geometry and energetic information for both spin states.

4.1.6 Automatic selection of the reactant reference minimum

The reactant reference is selected according to the electronic-plus-zero-point energy rather than the electronic energy alone.

The multiplicities are ordered using:

```

sorted_mults = sorted(
    multiplicities,
    key=lambda m: minima_data[m]["E_plus_ZPE_Eh"]
)

```

The lower-energy state becomes:

```
reference_multiplicity = sorted_mults[0]
```

and the second state becomes:

```
other_multiplicity = sorted_mults[1]
```

The optimized geometry of the reference state is stored as:

```
Reference_geometry
```

and its multiplicity is stored as:

```
Reference_multiplicity
```

The reference-selection description is stored in:

```
Reference_source
```

The energy comparison is based on:

$$E_i^{\text{ref}} = E_{\text{elec},i} + E_{\text{ZPE},i}$$

The relative energies are calculated using the lowest electronic-plus-ZPE energy, E_0 , as the zero:

```
E0 = minima_data[reference_multiplicity]["E_plus_ZPE_Eh"]
```

For each multiplicity:

```
dEh = minima_data[mult]["E_plus_ZPE_Eh"] - E0
```

is converted into reciprocal centimetres and kilocalories per mole using:

```

hartree_to_cm = 219474.6
hartree_to_kcal = 627.509474

```

This reference minimum defines the zero of energy for the later MECP barrier, density-of-states, microcanonical-rate, and canonical-rate calculations.

4.1.7 Construction of the Cartesian interpolation

Step 1 constructs a linear Cartesian interpolation between the two optimized minimum geometries.

The default number of interpolation points is:

```
n_interp = 10
```

For interpolation point i , the interpolation parameter is:

```
lam = i / (n_interp - 1)
```

which spans the interval from 0 to 1, including both optimized minima.

The interpolated coordinates are evaluated according to

$$\mathbf{R}(\lambda) = (1 - \lambda)\mathbf{R}_A + \lambda\mathbf{R}_B,$$

implemented by:

```
interpolate_geometry(  
    geom_a,  
    geom_b,  
    lam  
)
```

Before interpolation, the code verifies that both geometries have the same atom ordering and coordinate dimensions:

```
if sym_a != sym_b:  
    raise RuntimeError(  
        "Atom order differs between optimized minima."  
    )
```

Each interpolation geometry is saved as an XYZ file containing the corresponding λ value.

The geometry and metadata for each interpolation point are stored in:

```
interp_jobs_by_index[i + 1]
```

including:

```
{  
    "index": i + 1,  
    "lambda": lam,  
    "geometry": geom_i,  
    "geometry_file": local_xyz,  
    "jobs": []  
}
```

4.1.8 Single-point energies on both spin surfaces

For every interpolation geometry, SPINKS performs one single-point calculation for each multiplicity. Therefore, a 10-point interpolation requires 20 ORCA single-point calculations.

The job-specific ORCA input is created from the reviewed DFT settings using:

```
apply_dft_settings_to_input(  
    sp_inp,  
    "single_point"  
)
```

This removes the Opt, Freq, and other job-specific keywords while preserving the method, basis set, SCF settings, solvent model, dispersion correction, memory, processor allocation, and transferable percent blocks.

The interpolation geometry, charge, and multiplicity are protected during failed-job repair. For interpolation jobs, the repair policy is:

```
repair_policy = "interp_header_only"
```

which allows modification only of the pre-geometry ORCA settings.

The final electronic energy is extracted from the last occurrence of:

```
FINAL SINGLE POINT ENERGY
```

using:

```
extract_final_sp_energy_from_text(...)
```

For interpolation point i , the signed energy difference is:

```
gap = energies[mult_a] - energies[mult_b]
```

and the absolute gap is stored as:

```
gap_abs_Eh = abs(gap)
```

The corresponding values in reciprocal centimetres are:

```
gap_cm1 = gap * hartree_to_cm  
gap_abs_cm1 = abs(gap * hartree_to_cm)
```

The complete results for every interpolation geometry are stored in the list:

```
interp_records
```

Each record contains:

```
{  
    "index": idx,  
    "lambda": lam,  
    "geometry": geom_i,  
    "energies_Eh": energies,  
    "gap_Eh": gap,  
    "gap_abs_Eh": abs(gap),  
}
```

```

"gap_cm1": gap * hartree_to_cm,
"gap_abs_cm1": abs(gap * hartree_to_cm),
"gap_kcal": gap * hartree_to_kcal
}

```

4.1.9 Selection of the MECP starting geometry

The interpolation point with the smallest absolute energy difference between the two spin surfaces is selected using:

```

best_record = min(
    interp_records,
    key=lambda r: r["gap_abs_Eh"]
)

```

The selected Cartesian structure is stored as:

```
MECP_start_geometry = best_record["geometry"]
```

The principal associated quantities are:

```

best_record["index"]
best_record["lambda"]
best_record["gap_abs_Eh"]
best_record["gap_abs_cm1"]

```

This structure is not assumed to be the crossing point itself. It is only the interpolation geometry that most closely approaches degeneracy and therefore provides the most suitable starting structure available along the linear interpolation pathway.

The selected geometry is written to:

```
<jobname>_best_MECP_crossing_guess.xyz
```

and the complete interpolation data are saved in:

```
<jobname>_interpolation_scan_summary.txt
```

4.1.10 Interpolation-energy visualization

SPINKS also generates an interpolation-energy plot. The energies of both spin surfaces are shifted relative to the lowest electronic energy appearing anywhere in the interpolation:

```

interpolation_reference_Eh = min(
    np.min(low_spin_energies_Eh),
    np.min(high_spin_energies_Eh)
)

```

The relative energies are calculated as:

```

low_spin_relative_kcal = (
    low_spin_energies_Eh -
    interpolation_reference_Eh
) * hartree_to_kcal

```

with an analogous expression for the high-spin surface.

The resulting plot is saved in both PNG and PDF formats:

```
interpolation_plot_png  
interpolation_plot_pdf
```

The PNG file is written at 600 dpi, whereas the PDF file retains vector graphics suitable for publication.

4.1.11 Construction of the MECP input

The main multiplicity used in the ORCA MECP input is the reference-state multiplicity:

```
mult_main = reference_multiplicity  
mult_other = other_multiplicity
```

The MECP input is generated using the selected interpolation geometry and the ORCA keyword:

```
SurfCrossOpt
```

The second multiplicity is defined through the %mecp block:

```
%mecp Mult <mult_other>  
end
```

Programmatically, the input is constructed using:

```
mecp_inp = apply_dft_settings_to_input(  
    mec_inp,  
    "mecp",  
    job_specific_blocks=  
        f"%mecp Mult {mult_other}\\nend"  
)
```

This procedure preserves the reviewed DFT settings while introducing the MECP-specific keyword and multiplicity block.

4.1.12 Protected MECP optimization

The MECP calculation is executed using:

```
submit_and_monitor_orca_job(  
    local_mecp,  
    mec_jobname,  
    mec_inp,  
    "MECP SurfCrossOpt",  
    repair_policy="mecp_restricted",  
    locked={...}  
)
```

The restricted repair policy protects the quantities defining the MECP search:

```
locked = {  
    "method": method,  
    "basis": basis,  
    "charge": charge,  
    "mult": mult_main,
```

```
"mecp_mult": mult_other,  
"geom": MECP_start_geometry  
}
```

Any revised MECP input must therefore retain:

- the selected electronic-structure method;
- the basis set;
- the SurfCrossOpt keyword;
- the molecular charge;
- the main multiplicity;
- the second multiplicity in the %mecp block; and
- the automatically selected starting geometry.

The final optimized MECP geometry is extracted from the ORCA-generated XYZ file and stored as:

[MECP_geometry](#)

This geometry becomes the common nuclear configuration used in the subsequent SOC, gradient, Hessian, reduced-mass, and kinetic calculations.

4.1.13 Step 1 output variables

The most important variables passed from Step 1 to later stages are:

[Initial_geometry](#)
[Reference_geometry](#)
[MECP_start_geometry](#)
[MECP_geometry](#)

representing, respectively:

- the user-supplied initial structure;
- the optimized electronic-plus-ZPE reference minimum;
- the best interpolation geometry used to initialize the MECP search; and
- the optimized minimum-energy crossing point.

The energetic and spin-state information is retained in:

[minima_data](#)
[Reference_multiplicity](#)
[reference_multiplicity](#)
[Reference_source](#)
[mult_main](#)
[mult_other](#)

The interpolation information is retained in:

```
interp_records  
best_record
```

The reviewed ORCA settings are retained in:

```
DFT_REVIEWED_PRE_XYZ  
ORCA_TRANSFER_BLOCKS_ALL  
ORCA_TRANSFER_BLOCKS_NO_OUTPUT
```

These variables ensure that the same electronic-structure settings can be propagated consistently into the later gradient and numerical-frequency calculations.

4.1.14 Files generated by step 1

Step 1 saves the following principal output files:

```
<jobname>_Reference_geometry.txt  
<jobname>_Reference_info.txt  
<jobname>_MECP_geometry.txt  
<jobname>_MECP_start_interpolated_geometry.txt  
<jobname>_final_MECP_input.inp  
<jobname>_interpolation_scan_summary.txt  
<jobname>_best_MECP_crossing_guess.xyz  
<jobname>_interpolation_energy_scan.png  
<jobname>_interpolation_energy_scan.pdf  
<jobname>_step1_orca_input_templates.txt
```

The reference-information file records:

- the selected reference multiplicity;
- electronic and zero-point energies;
- electronic-plus-ZPE energies;
- relative energies;
- interpolation-point selection;
- ORCA executable and directory;
- method and basis set; and
- the source of the reviewed ORCA settings.

The template file records the reviewed DFT pre-geometry block and the transferable ORCA blocks used to construct subsequent inputs.

4.1.15 Summary of the step 1 algorithm

The complete scientific sequence implemented in Step 1 can be summarized as:

Initial geometry $\rightarrow \begin{cases} \text{Minimum optimization and frequency, state A} \\ \text{Minimum optimization and frequency, state B} \end{cases}$
→ lowest ($E_{\text{elec}} + E_{\text{ZPE}}$) reference
→ Cartesian interpolation
→ $E_A(\lambda), E_B(\lambda)$
→ $\lambda_{\min} = \arg \min_{\lambda} |E_A(\lambda) - E_B(\lambda)|$
→ SurfCrossOpt MECP optimization
→ optimized MECP geometry.

This automated procedure provides all geometries, energies, reference-state assignments, and electronic-structure settings required for the subsequent SPINKS calculations.

4.2 Step 2: Orbital Preparation and SOC Calculation at the MECP

Step 2 performs the complete multireference spin–orbit coupling workflow at the optimized MECP obtained in Step 1. Starting from the MECP geometry, SPINKS automatically generates the electronic wavefunctions required for the SOC calculation, determines the active space using either ROHF singly occupied molecular orbitals (SOMOs) or unrestricted natural orbitals (UNOs), constructs an editable CASSCF SOC input, performs the SOC calculation, extracts the complete M_S -resolved spin–orbit coupling matrix from the ORCA output, and evaluates the effective, intermediate, and M_S -specific spin–orbit couplings used throughout the remaining workflow.

The principal operations performed in Step 2 are:

1. preparation of ROHF and UHF/UNO reference wavefunctions,
2. automatic determination of active orbitals,
3. localized-orbital analysis of the ROHF SOMOs,
4. generation of the SOC input,
5. execution of the multireference SOC calculation,
6. extraction of the complex SOC matrix,
7. construction of the M_S -resolved SOC matrix, and
8. evaluation of the effective and intermediate SOC quantities.

Unlike Step 1, which performs only DFT calculations, Step 2 combines Hartree–Fock, orbital localization, CASSCF, and relativistic SOC calculations into a single automated workflow.

4.2.1 Initialization

Step 2 begins by importing the principal variables generated in Step 1:

```
required_vars_step2 = [  
    "jobname",  
    "MECP_geometry",  
    "charge",  
    "mult_main",  
    "mult_other",  
    ...  
]
```

These variables define the optimized MECP geometry, the charge, the two spin multiplicities participating in the crossing, the computational settings, and the directory structure established during Step 1. The calculation therefore proceeds directly from the optimized crossing geometry without requiring additional user input.

4.2.2 High-spin ROHF orbital preparation

The first electronic-structure calculation performed in Step 2 is a restricted open-shell Hartree–Fock calculation on the higher-spin state.

The multiplicity is selected automatically as:

```
soc_orbital_mult = max(multiplicities)
```

and the corresponding ORCA input is generated using:

```
! ROHF {basis} TightSCF SlowConv
```

The MECP geometry obtained in Step 1 is inserted directly into the input, and the user-reviewed SCF settings from Step 1 are propagated automatically to ensure consistency between all DFT and Hartree–Fock calculations.

Before submitting the calculation, SPINKS checks whether a completed ROHF output and its corresponding GBW file already exist. If both files are available, the calculation is skipped and the existing wavefunction is reused. Otherwise, the ROHF calculation is executed automatically.

4.2.3 Automatic SOMO detection

After the ROHF calculation has completed, SPINKS parses the **ORBITAL ENERGIES** section of the ORCA output to identify singly occupied molecular orbitals.

The implementation searches for orbitals satisfying

$$\text{Occupation} \approx 1.0000,$$

using the function:

```
parse_singly_occupied_orbitals_from_rohf_out(...)
```

The detected SOMO indices are stored in:

```
rohf_somo_indices
```

For example:

```
Detected ROHF SOMOs:
```

```
[97,98]
```

would automatically produce:

```
first_somo = 97  
last_somo = 98
```

The corresponding active space is then defined as:

```
nel_soc = len(rohf_somo_indices)  
norb_soc = len(rohf_somo_indices)
```

yielding

$$\text{CAS}(n_{\text{SOMO}}, n_{\text{SOMO}}).$$

Thus, a system containing two SOMOs automatically generates a CAS(2,2) active space, whereas four SOMOs produce CAS(4,4), without requiring any user intervention.

4.2.4 UHF/UNO orbital preparation

Although ROHF is used by default, SPINKS also performs an unrestricted Hartree–Fock calculation with unrestricted natural orbitals.

The corresponding input contains:

```
! UHF {basis} TightSCF SlowConv UNO
```

together with:

```
Print[P_UNO_OccNum] 1  
Print[P_UNO_AtPopMO_M] 1
```

which instruct ORCA to print the UNO occupation numbers required for automatic active-space determination.

The UNO occupation numbers are extracted using:

```
parse_uno_occupations_from_uhf_out(...)
```

The active orbitals are then selected according to the occupation criterion

$$0.02 < n_i < 1.98,$$

implemented by:

```
select_uno_active_orbitals(  
    low=0.02,  
    high=1.98  
)
```

The resulting active space is

$$\text{CAS}(N_{\text{active}}, M_{\text{active}}),$$

where N_{active} is obtained by summing the UNO occupations within the selected window and rounding to the nearest integer.

This strategy follows the commonly used UNO active-space criterion while remaining fully automatic.

4.2.5 Localized SOMO analysis

SPINKS next evaluates whether the ROHF SOMOs are predominantly metal centered.

The previously generated GBW file is supplied to **orca_loc**, which localizes the SOMOs identified in the preceding section.

The localized orbital compositions are parsed using:

```
parse_lmo_compositions_from_orca_loc_output(...)
```

For every SOMO, the metal contribution is evaluated.

For example:

```
MO 97
Fe population = 0.961
```

indicates that approximately 96% of the localized orbital resides on the metal atom.

The diagnostic criterion:

```
metal_population_threshold = 0.93
```

is then applied.

If every localized SOMO exceeds this threshold, the orbitals are classified as predominantly metal centered.

This analysis is intended only as a diagnostic. It does not alter the active space or the SOC calculation, but provides valuable information regarding the nature of the magnetic orbitals involved in spin crossing.

4.2.6 Selection of the orbital source

Following completion of both ROHF and UHF/UNO calculations, SPINKS summarizes the two automatically generated active spaces.

The user is then asked whether the SOC calculation should employ

- ROHF orbitals (recommended), or
- UHF/UNO orbitals.

ROHF is selected by default because it generally provides a balanced reference for transition-metal systems, whereas UHF/UNO is primarily intended for difficult open-shell systems in which ROHF convergence or orbital quality may become problematic.

Once the orbital source has been selected, the corresponding active orbitals and active-space dimensions are stored as:

```
soc_active_indices  
soc_nel  
soc_norb
```

These quantities are used throughout the remaining SOC calculation.

4.2.7 Automatic construction of the SOC input

Using the selected active space, SPINKS automatically generates the complete CASSCF SOC input.

The central portion of the generated input is:

```
%casscf  
  maxiter 1  
  mult ...  
  nroots ...  
  nel ...  
  nOrb ...  
  
  rel  
    dosoc true  
    dosscc false  
  end  
end
```

The active-space dimensions are inserted automatically from:

```
soc_nel  
soc_norb
```

while the multiplicities are taken directly from the two crossing states determined in Step 1.

Unlike the DFT calculations, the SOC input is intentionally left fully editable so that advanced users may modify any aspect of the multireference calculation before execution.

4.2.8 Extraction of the SOC matrix

After the CASSCF calculation has completed, SPINKS parses the section:

```
NONZERO SOC MATRIX ELEMENTS (cm**-1)
```

from the ORCA output.

Each printed matrix element is converted into a complex number,

$$H_{ij} = H_{ij}^{\text{real}} + iH_{ij}^{\text{imag}},$$

and stored together with its corresponding spin quantum numbers:

```
bra_S  
bra_Ms  
ket_S  
ket_Ms
```

using:

```
parse_orca_soc_matrix_elements(...)
```

The complete complex matrix therefore retains all M_S -resolved information printed by ORCA.

4.2.9 Construction of the M_S -resolved SOC matrix

The extracted SOC elements are reorganized into the matrix:

```
SOC_Ms_matrix_cm1
```

whose rows correspond to the lower-spin M_S values and whose columns correspond to the higher-spin M_S values.

The implementation is performed by:

```
build_ms_resolved_soc_matrix(...)
```

The resulting matrix constitutes the central data structure used in all subsequent SOC analyses.

4.2.10 Effective, intermediate, and M_S -specific SOC

The effective spin-orbit coupling is evaluated directly from the complete M_S -resolved matrix according to the following equation,

$$H_{\text{SO}} = \sqrt{\sum_{M_S, M'_S} |\langle S, M_S | H_{\text{SO}} | S', M'_S \rangle|^2}.$$

The implementation is:

```
H_SO_all_matrix_elements_cm = compute_effective_soc_norm(SOC_Ms_matrix_cm1)
```

No empirical scaling factors or RMS averaging are applied.

SPINKS also identifies the intermediate SOC components used in the equation above and groups the individual M_S -resolved channels into dictionaries:

```
H_SO_channels_cm1  
H_SO_intermediate_components_cm1
```

These data are retained separately so that later workflow steps can evaluate Weak coupling probabilities using effective, intermediate, and fully M_S -resolved coupling representations independently.

The classified intermediate SOC components stored in the variable `H_SO_intermediate_components_cm1` are individual complex elements of the M_S -resolved SOC matrix. They should not be confused with the intermediate SOC values calculated later in Step 8. The latter are scalar couplings constructed from the norms of symmetry-related SOC-matrix rows for each unique $|M_{S,\text{low}}|$ value.

4.2.11 User override of the effective SOC

Although the ORCA-derived effective SOC is used by default, SPINKS optionally allows the user to replace it with an externally supplied value.

If selected, the user-defined scalar:

```
H_SO_cm
```

replaces the ORCA value in all subsequent probability and rate calculations, while the M_S -resolved analysis is disabled.

This option is primarily intended for benchmarking against literature values or experimental estimates.

4.2.12 Variables exported by step 2

The principal quantities transferred to the remaining workflow include:

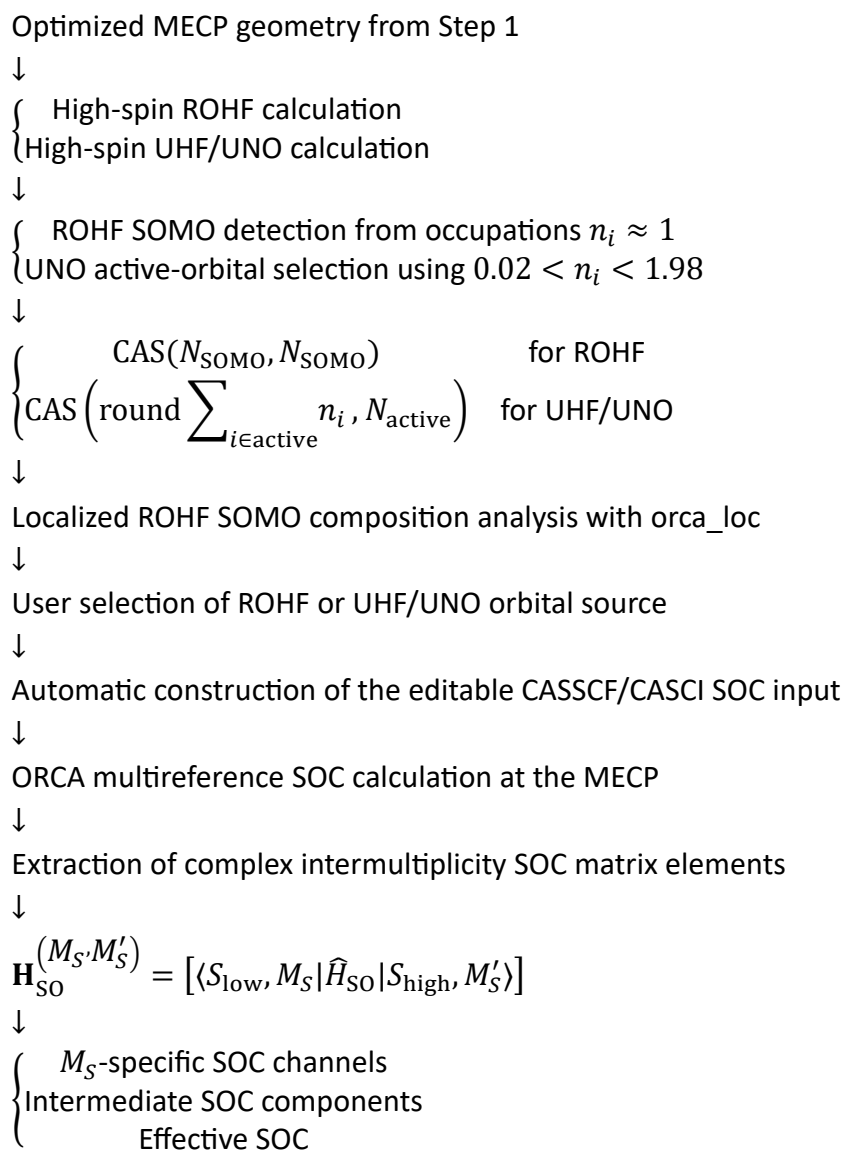
```
H_SO_cm  
SOC_Ms_matrix_cm1  
H_SO_channels_cm1
```

H_SO_intermediate_components_cm1
 rohf_somo_indices
 uhf_uno_active_indices
 soc_nel
 soc_norb
 soc_protocol

These variables provide all orbital, active-space, and spin–orbit coupling information required for the subsequent probability, density-of-states, and kinetic calculations.

4.2.13 Summary of the step 2 algorithm

The complete sequence implemented in Step 2 can be summarized as:



with the effective SOC evaluated as

$$H_{\text{SO}}^{\text{eff}} = \sqrt{\sum_{M_S, M'_S} |H_{\text{SO}}^{(M_S, M'_S)}|^2}.$$

The corresponding implementation flow is represented by the principal variables:

```

MECP_geometry
↓
roh_f_somo_indices
uhf_uno_active_indices
↓
soc_orbital_source
soc_active_indices
soc_nel
soc_norb
↓
SOC_Ms_matrix_cm1
↓
H_SO_channels_cm1
H_SO_intermediate_components_cm1
H_SO_ORCA_effective_cm
↓
H_SO_cm

```

If the ORCA-derived SOC is retained, the complete M_S -resolved matrix and all associated channel representations remain available for the subsequent probability and rate calculations. If the user supplies an external scalar SOC value, that value is assigned to $H_{\text{SO_cm}}$, and subsequent calculations are performed in effective-SOC-only mode.

Therefore, Step 2 transforms the optimized MECP geometry into the complete set of spin-orbit coupling quantities required by the later SPINKS calculations:

MECP geometry → reference orbitals → active space → SOC matrix → $\{H_{\text{SO}}^{\text{eff}}, H_{\text{SO}}^{\text{int}}, H_{\text{SO}}^{M_S, M'_S}\}$

These quantities are subsequently used in Step 8 to evaluate the Landau-Zener and weak coupling spin-crossing probabilities independently for the effective, intermediate, and M_S -specific SOC representations.

4.3 Step 3: MECP Energy Gradients

Step 3 evaluates the analytical energy gradients of the two spin-state potential-energy surfaces at the optimized MECP geometry obtained in Step 1. The resulting Cartesian gradient vectors provide the nuclear derivative information required to define the local crossing topology, the gradient-difference direction, and the reaction-coordinate quantities constructed formally in Step 4.

The principal operations performed in Step 3 are:

1. verification and import of the MECP geometry and computational settings;
2. construction of one ORCA Engrad input for each spin multiplicity;
3. propagation of the reviewed DFT settings from Step 1;
4. execution and monitoring of the two analytical-gradient calculations;
5. extraction of the $3N$ -dimensional Cartesian gradient vectors from the ORCA .engrad files;
6. assignment of the lower- and higher-spin gradients;
7. construction of the Cartesian gradient-difference vector;
8. normalization of the gradient-difference vector to obtain the crossing-seam normal direction; and
9. storage of the gradient vectors and associated diagnostics for Step 4.

The local-PC and HPC implementations use the same ORCA inputs, parsing procedures, and numerical definitions. They differ only in job submission, monitoring, and file transfer. The following description uses the local implementation as the representative code.

4.3.1 Import of quantities from the preceding steps

Step 3 requires the optimized MECP geometry and the electronic-structure settings established in Step 1. These dependencies are verified before any calculation is initiated:

```
required_vars_step3 = [  
    "jobname",  
    "MECP_geometry",  
    "charge",  
    "method",  
    "basis",  
    "nprocs",
```

```
"maxcore_mb",  
"multiplicities",  
"mult_main",  
"mult_other",  
...  
]
```

The most important imported quantity is:

```
MECP_geometry
```

which contains the optimized Cartesian geometry of the crossing point. Both spin-state gradients must be evaluated at exactly this same nuclear configuration. Step 3 therefore terminates if MECP_geometry is missing or empty.

The two multiplicities participating in the crossing are obtained from:

```
mult_main  
mult_other
```

and placed in numerical order:

```
engrad_mults = sorted([  
    int(mult_main),  
    int(mult_other)  
])
```

The smaller multiplicity is later identified as the low-spin state and the larger multiplicity as the high-spin state.

4.3.2 Preparation of the gradient-calculation method

The method string supplied in Step 1 may contain job-specific ORCA keywords such as Opt, Freq, SurfCrossOpt, or Engrad. These keywords must not be propagated indiscriminately into the new gradient input.

SPINKS removes incompatible job keywords using:

```
def clean_method_for_engrad(method_text):  
    excluded = {  
        "surfcrossopt",  
        "surfcrossnumfreq",  
        "numfreq",  
        "freq",  
        "opt",  
        "engrad"
```

```
}
```

The resulting electronic-structure method is stored as:

```
engrad_method
```

For example, a reviewed Step 1 header containing:

```
! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq
```

is adapted for Step 3 by removing Opt and Freq and introducing the required Engrad keyword.

This prevents incompatible calculation types from being included simultaneously while preserving the selected functional, dispersion correction, basis set, convergence settings, solvent model, and other transferable ORCA options.

4.3.3 Directory preparation

All Step 3 files are placed in a dedicated Engrad directory:

```
local_engrad = os.path.join(  
    local_base,  
    "Engrad"  
)
```

This directory contains:

- the two generated ORCA input files;
- the corresponding ORCA output files;
- the binary and text files generated by ORCA;
- the two .engrad files;
- revised inputs from any failed-job recovery; and
- the final gradient-summary file.

In the local implementation:

```
remote_engrad = local_engrad
```

is retained as a compatibility alias so that subsequent steps can use common variable names in both PC and HPC workflows.

4.3.4 Construction of the ORCA Engrad inputs

A separate analytical-gradient calculation is generated for each spin multiplicity. The preferred construction route uses the reviewed DFT settings stored by Step 1:

```
new_pre = make_dft_pre_xyz("engrad")
```

The `make_dft_pre_xyz()` function adapts the reviewed ORCA header to the requested job type. For an energy-gradient calculation, it introduces Engrad while removing incompatible optimization, frequency, single-point, and MECP-search keywords.

The final input is assembled as:

```
return f'''{new_pre.rstrip()}
* xyz {charge} {mult}
{MECP_geometry}
*
'''
```

Thus, both spin surfaces are evaluated using:

- the same electronic-structure method;
- the same basis set;
- the same SCF settings;
- the same solvent and dispersion treatment, where applicable;
- the same molecular charge;
- the same MECP geometry; and
- their respective spin multiplicities.

A representative header is:

```
! Engrad B3LYP D3 def2-TZVP TightSCF SlowConv
```

The consistency of the geometry and electronic-structure settings is important because the gradient difference must represent the local difference between the two potential-energy surfaces at one common nuclear configuration.

4.3.5 Generation of the two spin-state jobs

The program creates one job definition for each multiplicity:

```
for mult in engrad_mults:
    spin_name = multiplicity_name(mult)
```

```
engrad_jobname = (
    f"{jobname}_Engrad_{spin_name}"
)
```

For a singlet–triplet crossing, for example, the generated job names are:

```
<jobname>_Engrad_singlet
<jobname>_Engrad_triplet
```

Each job definition contains the multiplicity, spin-state name, input text, job name, and descriptive label:

```
engrad_job_definitions.append({
    "mult": mult,
    "spin_name": spin_name,
    "jobname": engrad_jobname,
    "inp_text": inp_text,
    "job_label":
        f"{spin_name} DFT Engrad at MECP"
})
```

This data structure preserves the correspondence between each generated file and its associated spin state.

4.3.6 Execution and protected input repair

The two ORCA calculations are executed using the job-management function established in Step 1:

```
submit_and_monitor_orca_job(...)
```

For Step 3, the repair policy is:

```
repair_policy = "engrad_restricted"
```

The protected quantities are:

```
locked = {
    "method": engrad_method,
    "basis": basis,
    "charge": charge,
    "mult": job["mult"],
    "geom": MECP_geometry
}
```

A revised input must therefore retain:

- the required Engrad keyword;
- the selected method;
- the selected basis set;

- the molecular charge;
- the appropriate multiplicity; and
- the optimized MECP geometry.

The user may modify convergence-related settings or add compatible ORCA options if a calculation fails, but cannot alter the quantities that define the physical gradient calculation.

After execution, each output is checked for normal ORCA termination. Step 3 does not proceed until both spin-state calculations have completed successfully.

4.3.7 Location and parsing of the ORCA .engrad files

ORCA writes analytical gradients to .engrad files. SPINKS first searches for the expected filename:

```
expected = (
    local_dir / f"{jobname_i}.engrad"
)
```

If that exact file is absent, the code searches for any matching .engrad file beginning with the job name.

The gradient is extracted using:

```
extract_gradient_from_engrad_text(...)
```

The parser first reads the number of atoms, N , from the section marked:

```
Number of atoms
```

and then locates the section:

```
The current gradient in Eh/bohr
```

Exactly $3N$ Cartesian gradient components are collected:

```
expected = 3 * natoms
```

The parser raises an error unless the number of extracted values is exactly $3N$. This validation prevents incomplete or malformed gradient files from being used in subsequent calculations.

The extracted gradient is returned as a one-dimensional NumPy array:

```
gradient = np.array(
    gradient_values,
    dtype=float
)
```

with the ordering

$$\mathbf{g} = \left(\frac{\partial E}{\partial x_1}, \frac{\partial E}{\partial y_1}, \frac{\partial E}{\partial z_1}, \dots, \frac{\partial E}{\partial z_N} \right).$$

The units are Hartree per bohr.

4.3.8 Storage of the spin-state gradients

The extracted gradients are stored by both multiplicity and spin-state name:

```
gradients_by_multiplicity[mult] = gradient  
gradients_by_name[spin_name] = gradient
```

They are also exported as dynamically named variables:

```
globals()[f"g_{spin_name}"] = gradient
```

For example, a singlet–triplet calculation produces variables equivalent to:

```
g_LS  
g_HS
```

The two general-purpose gradient variables used by later steps are assigned as:

```
g_LS = gradients_by_multiplicity[mult_LS]  
g_HS = gradients_by_multiplicity[mult_HS]
```

where:

```
mult_LS = available_mults[0]  
mult_HS = available_mults[-1]
```

The code verifies that both vectors have identical dimensions:

```
if g_LS.shape != g_HS.shape:  
    raise RuntimeError(...)
```

This confirms that both gradients correspond to the same number of atoms and the same Cartesian coordinate ordering.

4.3.9 Gradient norms

The Euclidean norms of the two Cartesian gradient vectors are evaluated as

$$\| \mathbf{g}_{\text{LS}} \| = \sqrt{\sum_{k=1}^{3N} g_{\text{LS},k}^2}$$

and

$$\|\mathbf{g}_{\text{HS}}\| = \sqrt{\sum_{k=1}^{3N} g_{\text{HS},k}^2}.$$

The corresponding implementation is:

```
g_LS_norm_Eh_per_Bohr = float(
    np.linalg.norm(g_LS)
)

g_HS_norm_Eh_per_Bohr = float(
    np.linalg.norm(g_HS)
)
```

These values are retained for the gradient analysis performed in Step 4.

Step 3 also evaluates the geometric mean of the two gradient norms:

```
projected_gradient_mean_Eh_per_Bohr = float(
    np.sqrt(
        g_LS_norm_Eh_per_Bohr *
        g_HS_norm_Eh_per_Bohr
    )
)
```

or

$$g_{\text{mean}} = \sqrt{\|\mathbf{g}_{\text{LS}}\| \|\mathbf{g}_{\text{HS}}\|}.$$

This quantity is stored as a diagnostic and is passed to Step 4, where the formal MECP gradient quantities and reaction-coordinate definitions are consolidated.

4.3.10 Cartesian gradient-difference vector

The gradient-difference vector is constructed as:

```
reverse_surface_gradient_difference_cart = g_HS - g_LS
```

corresponding to

$$\Delta\mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}.$$

This vector describes the first-order change in the energy separation between the two spin surfaces in Cartesian coordinate space. Its norm is calculated as:

```
GRADIENT_GAP_NORM_EH_PER_BOHR = float(
    np.linalg.norm(reverse_surface_gradient_difference_cart)
)
```

or

$$\|\Delta \mathbf{g}\| = \sqrt{\sum_{k=1}^{3N} (g_{\text{HS},k} - g_{\text{LS},k})^2}.$$

The code rejects nonfinite or nonpositive values:

```
if (
    not np.isfinite(
        GRADIENT_GAP_NORM_EH_PER_BOHR
    )
    or GRADIENT_GAP_NORM_EH_PER_BOHR <= 0.0
):
    raise RuntimeError(...)
```

A zero gradient difference would prevent definition of a unique crossing-normal direction and would indicate an invalid or numerically degenerate result.

4.3.11 Crossing-seam normal direction

The normalized Cartesian gradient-difference vector is defined as:

```
MECP_seam_normal_cart = (
    reverse_surface_gradient_difference_cart /
    GRADIENT_GAP_NORM_EH_PER_BOHR
)
```

corresponding to

$$\mathbf{n}_{\text{cart}} = \frac{\Delta \mathbf{g}}{\|\Delta \mathbf{g}\|}.$$

The normalized seam-normal vector is stored directly as:

```
MECP_seam_normal_cart = reverse_gradient_unit_vector.copy()
```

By construction,

$$\|\mathbf{n}_{\text{cart}}\| = 1.$$

SPINKS prints the norm as a diagnostic to verify numerical normalization.

This Cartesian vector identifies the local direction normal to the crossing seam associated with the difference between the two surface gradients. It is subsequently combined with the mean-gradient direction in Step 4 to define the reaction-coordinate vectors and classify the crossing as peaked or sloped.

4.3.12 Preliminary assignment of the gradient-gap magnitude

Step 3 assigns:

```
GRADIENT_GAP_NORM_EH_PER_BOHR = (  
  GRADIENT_GAP_NORM_EH_PER_BOHR  
)
```

so that

$$\Delta F_{\parallel} = \| \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \|.$$

This variable provides the gradient-difference magnitude required in the Landau-Zener probability calculation.

The uppercase variable name:

```
GRADIENT_GAP_NORM_EH_PER_BOHR
```

is retained throughout the remaining workflow to identify the active value and its units explicitly.

Although this quantity is first calculated in Step 3, Step 4 consolidates it with the complete MECF gradient analysis, reaction-coordinate construction, crossing classification, and effective-Hessian weighting parameter.

4.3.13 Files generated by step 3

The complete gradient information is written to:

```
<jobname>_Engrad_gradients_summary.txt
```

The file records:

- electronic-structure method;
- basis set;
- molecular charge;

- both multiplicities;
- identities of the low- and high-spin surfaces;
- $\| \mathbf{g}_{\text{LS}} \|$;
- $\| \mathbf{g}_{\text{HS}} \|$;
- the geometric mean of the gradient norms;
- $\| \Delta \mathbf{g} \|$;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- paths to both .engrad files;
- all $3N$ components of $\mathbf{g}_{\text{LS}}$;
- all $3N$ components of $\mathbf{g}_{\text{HS}}$;
- all components of $\Delta \mathbf{g}$; and
- all components of the normalized crossing-seam direction.

The individual gradient vectors are written using full numerical precision so that the subsequent gradient analysis can be reproduced independently.

4.3.14 Variables exported by step 3

The principal variables passed to Step 4 are:

```
g_LS
g_HS
reverse_surface_gradient_difference_cart
MECP_seam_normal_cart
n_cart
```

together with the scalar quantities:

```
g_LS_norm_Eh_per_Bohr
g_HS_norm_Eh_per_Bohr
GRADIENT_GAP_NORM_EH_PER_BOHR
GRADIENT_GAP_NORM_EH_PER_BOHR
projected_gradient_mean_Eh_per_Bohr
```

The code also exports the bookkeeping dictionaries:

```
gradients_by_multiplicity
gradients_by_name
natoms_by_multiplicity
engrad_files_remote
```

These variables preserve the relationship among the spin multiplicities, gradient vectors, source files, and atomic dimensions.

4.3.15 Summary of the step 3 algorithm

The complete scientific sequence implemented in Step 3 can be summarized as:

```

Optimized MECP geometry from Step 1
↓
{ DFT analytical gradient on the low-spin surface
  DFT analytical gradient on the high-spin surface
↓
{  $\mathbf{g}_{\text{LS}} = \nabla E_{\text{LS}}(\mathbf{R}_{\text{MECP}})$ 
   $\mathbf{g}_{\text{HS}} = \nabla E_{\text{HS}}(\mathbf{R}_{\text{MECP}})$ 
↓
Validation that both vectors contain exactly  $3N$  Cartesian components
↓
 $\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$ 
↓
 $\Delta F_{\parallel} = \|\Delta \mathbf{g}\|$ 
↓
 $\mathbf{n}_{\text{cart}} = \frac{\Delta \mathbf{g}}{\|\Delta \mathbf{g}\|}$ 

```

together with the diagnostic gradient magnitudes

$$\|\mathbf{g}_{\text{LS}}\|, \|\mathbf{g}_{\text{HS}}\|,$$

and

$$g_{\text{mean}} = \sqrt{\|\mathbf{g}_{\text{LS}}\| \|\mathbf{g}_{\text{HS}}\|}.$$

The corresponding variable flow is:

```

MECP_geometry
↓
Engrad input for mult_LS
Engrad input for mult_HS
↓
g_LS
g_HS

```

↓
 reverse_surface_gradient_difference_cart = $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$
 ↓
 GRADIENT_GAP_NORM_EH_PER_BOHR
 GRADIENT_GAP_NORM_EH_PER_BOHR
 ↓
 MECP_seam_normal_cart
 n_cart

Therefore, Step 3 transforms the optimized MECP geometry into the two complete Cartesian energy-gradient vectors and their normalized difference direction:

$$\mathbf{R}_{\text{MECP}} \rightarrow \{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}\} \rightarrow \Delta \mathbf{g} \rightarrow \{\Delta F_{\parallel}, \mathbf{n}_{\text{cart}}\}$$

These quantities are subsequently used in Step 4 to define the complete MECP gradient quantities, construct the reaction-coordinate directions, determine the effective-Hessian weighting parameter, and classify the crossing as peaked or sloped.

4.4 Step 4: Gradient Analysis and Crossing Classification

Step 4 reformulates the two Cartesian gradient vectors obtained in Step 3 into the gradient quantities and directional conventions required by the downstream statistical treatment of the crossing point. No additional ORCA calculation is performed in this step. Instead, Step 4 operates entirely on the previously extracted low-spin and high-spin gradients.

The principal operations performed in Step 4 are:

1. verification and reshaping of the two Cartesian gradient vectors;
2. construction of the surface-gradient-difference vector;
3. preservation of the opposite sign convention used by later workflow sections;
4. evaluation of the gradient-difference magnitude;
5. calculation of the individual gradient norms;
6. evaluation of a Cartesian geometric-mean gradient diagnostic;
7. classification of the crossing as peaked or sloped;
8. calculation of the effective-Hessian weighting parameter;
9. construction of normalized Cartesian crossing directions; and
10. export of the resulting quantities for the effective-Hessian and kinetic calculations.

A critical distinction is made between the Cartesian diagnostic

$$\sqrt{\| \mathbf{g}_{LS} \| \| \mathbf{g}_{HS} \|}$$

calculated in Step 4 and the final reaction-coordinate-projected projected quantity PROJECTED_GRADIENT_MEAN_EH_PER_BOHR, which is evaluated later after construction of the effective-Hessian reaction coordinate.

4.4.1 Import of the two MECP gradients

Step 4 requires the complete low-spin and high-spin Cartesian gradients generated in Step 3:

```
required_vars_step4 = [  
    "jobname",  
    "MECP_geometry",  
    "Reference_geometry",  
    "g_LS",
```

```

    "g_HS",
    "mult_LS",
    "mult_HS",
    "spin_name_LS",
    "spin_name_HS",
    ...
]

```

The two gradients are converted explicitly to one-dimensional floating-point arrays:

```

g_LS = np.asarray(g_LS, dtype=float).reshape(-1)
g_HS = np.asarray(g_HS, dtype=float).reshape(-1)

```

Their dimensions are then compared:

```

if g_LS.shape != g_HS.shape:
    raise RuntimeError(...)

```

This check ensures that both vectors contain the same $3N$ Cartesian components and correspond to the same MECP geometry and atom ordering.

4.4.2 Removal of stale projected-gradient variables

Before defining the Step 4 quantities, SPINKS removes any pre-existing projected gradient-mean variables:

```

globals().pop("PROJECTED_GRADIENT_MEAN_EH_PER_BOHR", None)
globals().pop("projected_gradient_mean_Eh_per_Bohr", None)

```

This safeguard prevents a value generated during a previous execution of a later step from being mistaken for a current Step 4 result.

This distinction is important because Step 4 evaluates only a Cartesian diagnostic, whereas the final PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is calculated later by projecting the gradients onto the effective-Hessian reaction coordinate.

4.4.3 Surface-gradient-difference convention

The SPINKS and effective-Hessian convention used in Step 4 defines the gradient-difference vector as

$$\Delta \mathbf{G}_{\text{diff}} = \mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}.$$

The implementation is:

```

surface_gradient_difference_cart = g_LS - g_HS

```

The vector `surface_gradient_difference_cart` is therefore the authoritative selected-sign gradient-difference vector.

Its Cartesian components are

$$\Delta G_k = g_{LS,k} - g_{HS,k}, k = 1, \dots, 3N.$$

The sign determines the orientation of the normalized vector but does not affect its norm.

4.4.4 Reverse surface-gradient convention

Earlier workflow sections and some later calculations use the opposite difference:

$$\Delta \mathbf{g} = \mathbf{g}_{HS} - \mathbf{g}_{LS}.$$

The reverse surface-gradient-difference vector is retained as:

`reverse_surface_gradient_difference_cart = g_HS - g_LS`

The two gradient-difference vectors have opposite orientations and equal norms.

$$\Delta \mathbf{g} = -\Delta \mathbf{G}_{\text{diff}}.$$

Thus,

$$\|\Delta \mathbf{g}\| = \|\Delta \mathbf{G}_{\text{diff}}\|.$$

Retaining both orientations makes the sign convention explicit while preserving the same gradient-gap magnitude.

4.4.5 Gradient-difference magnitude

The magnitude of the surface-gradient-difference vector is evaluated as:

```
gradient_gap_norm = float(
    np.linalg.norm(surface_gradient_difference_cart)
)
```

corresponding to

$$\|\Delta \mathbf{G}_{\text{diff}}\| = \sqrt{\sum_{k=1}^{3N} (g_{\text{LS},k} - g_{\text{HS},k})^2}.$$

The code verifies that this value is finite and positive:

```
if not np.isfinite(gradient_gap_norm) or gradient_gap_norm <= 0.0:
    raise RuntimeError(...)
```

The resulting scalar is stored using the following current variables:

```
gradient_gap_magnitude = gradient_gap_norm
GRADIENT_GAP_NORM_EH_PER_BOHR = gradient_gap_norm
gradient_gap_norm
```

Therefore,

$$\Delta F_{\parallel} = \|\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}\|.$$

The units are Hartree per bohr.

GRADIENT_GAP_NORM_EH_PER_BOHR is the principal variable passed to the Landau-Zener and weak coupling probability calculations.

4.4.6 Individual gradient norms

The magnitudes of the two surface gradients are recalculated as:

```
norm_grad_LS = float(np.linalg.norm(g_LS))
norm_grad_HS = float(np.linalg.norm(g_HS))
```

or

$$g_{\text{LS}}^{\text{norm}} = \|\mathbf{g}_{\text{LS}}\|$$

and

$$g_{\text{HS}}^{\text{norm}} = \|\mathbf{g}_{\text{HS}}\|.$$

The corresponding variables are:

```
g_LS_norm_Eh_per_Bohr
g_HS_norm_Eh_per_Bohr
```

These values are used in the Cartesian diagnostic and are also retained for later verification of the effective-Hessian projections.

4.4.7 Cartesian geometric-mean gradient diagnostic

Step 4 evaluates the geometric mean of the two Cartesian gradient norms:

```
cartesian_geometric_mean_gradient_Eh_per_Bohr = float(  
    np.sqrt(norm_grad_LS * norm_grad_HS)  
)
```

corresponding to

$$g_{\text{mean}}^{\text{Cartesian}} = \sqrt{\| \mathbf{g}_{\text{LS}} \| \| \mathbf{g}_{\text{HS}} \|}.$$

The value is stored as:

```
CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR
```

and:

```
cartesian_geometric_mean_gradient_Eh_per_Bohr
```

This quantity is retained only as a diagnostic of the full Cartesian gradients.

It is not the final projected gradient mean used in the weak coupling probability. The final value is obtained later from the gradient components projected along the effective-Hessian reaction coordinate.

This distinction is explicitly recorded in the output:

```
The final reaction-coordinate-projected  
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is calculated later.
```

4.4.8 Crossing classification

The local crossing topology is classified using the scalar product of the two surface gradients:

```
gradient_dot_product = float(  
    np.dot(g_LS, g_HS)  
)
```

or

$$G_{\text{slope}} = \mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}}.$$

The classification is then assigned as:

```

if gradslope < 0.0:
    intersection_type = "peaked"
else:
    intersection_type = "sloped"

```

Thus,

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} < 0 \Rightarrow \text{peaked crossing,}$$

whereas

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \geq 0 \Rightarrow \text{sloped crossing.}$$

At a peaked crossing, the two gradients point generally in opposing directions. At a sloped crossing, they possess a nonnegative mutual projection and therefore point more nearly in the same direction.

The classification is stored as:

```
intersection_type
```

for reporting and later diagnostics.

4.4.9 Effective-Hessian weighting parameter

Step 4 calculates the parameter used to combine the two spin-state Hessians in the effective-Hessian construction:

```

effective_hessian_mix_coefficient = float(
    np.dot(surface_gradient_difference_cart, g_LS)
    / (gradient_gap_norm ** 2)
)

```

In mathematical form,

$$\lambda = \frac{\Delta \mathbf{G}_{\text{diff}} \cdot \mathbf{g}_{\text{LS}}}{\|\Delta \mathbf{G}_{\text{diff}}\|^2}.$$

Since

$$\Delta \mathbf{G}_{\text{diff}} = \mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}},$$

the parameter may also be written as

$$\lambda = \frac{(\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}) \cdot \mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}\|^2}.$$

This value is exported as:

```
effective_hessian_mix_coefficient
```

and is used later to form the effective crossing-point Hessian from the two spin-state Hessians.

Step 4 only calculates the weighting parameter. The Hessians themselves are obtained in Step 5 and combined in Step 6.

4.4.10 Normalized gradient-difference direction

The normalized selected-sign gradient-difference vector is constructed as:

```
gradient_difference_unit_vector = (
    surface_gradient_difference_cart /
    gradient_gap_norm
)
```

corresponding to

$$\mathbf{n}_{\text{diff}} = \frac{\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}}{\|\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}\|}.$$

By construction,

$$\|\mathbf{n}_{\text{diff}}\| = 1.$$

The norm is printed as a numerical validation:

```
np.linalg.norm(gradient_difference_unit_vector)
```

This vector defines the SPINKS orientation of the Cartesian gradient-difference direction.

4.4.11 Reverse-gradient Cartesian direction

The opposite normalized direction is retained as:

```
n_cart = (
    reverse_surface_gradient_difference_cart /
    gradient_gap_norm
)
```

or

$$\mathbf{n}_{\text{cart}} = \frac{\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}\|}.$$

Therefore,

$$\mathbf{n}_{\text{cart}} = -\mathbf{n}_{\text{diff}}.$$

The code also assigns:

```
MECP_seam_normal_cart = n_cart.copy()
reaction_direction_cart = n_cart.copy()
```

These variables preserve the direction convention used by later parts of the workflow.

At this stage, `reaction_direction_cart` is a Cartesian gradient-difference direction. The final reaction-coordinate vector used for projected quantities is obtained later from the effective Hessian.

4.4.12 MECP-only workflow enforcement

Step 4 explicitly disables variables associated with transition-state and intrinsic-reaction-coordinate calculations:

```
IRC_frames = None
IRC_energies_hartree = None
coords_all = None
irc = None
symbols_irc = None
TS_geometry = None
irc_tangent = None
TS_index = None
```

This prevents an IRC tangent or transition-state geometry from being used accidentally in the MECP-only workflow.

The reaction-coordinate definitions in SPINKS are therefore based exclusively on:

- the two spin-state gradients at the MECP;
- the corresponding spin-state Hessians; and
- the effective-Hessian eigenvectors.

No conventional transition-state IRC is required.

4.4.13 Separation of step 4 and the final projected gradient mean

The code intentionally does not define:

`PROJECTED_GRADIENT_MEAN_EH_PER_BOHR`

in Step 4.

Instead, Step 4 defines only:

`CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR`

This separation prevents two physically different quantities from being confused.

The Step 4 Cartesian diagnostic is

$$g_{\text{mean}}^{\text{Cartesian}} = \sqrt{\| \mathbf{g}_{\text{LS}} \| \| \mathbf{g}_{\text{HS}} \|}.$$

The later projected projected quantity is based on the gradient components along the effective-Hessian reaction coordinate, $\mathbf{e}_{\text{RC}}$:

$$\begin{aligned} g_{\text{LS},\parallel} &= \mathbf{g}_{\text{LS}} \cdot \mathbf{e}_{\text{RC}}, \\ g_{\text{HS},\parallel} &= \mathbf{g}_{\text{HS}} \cdot \mathbf{e}_{\text{RC}}. \end{aligned}$$

The final projected gradient mean is then obtained from those directional components rather than from the full Cartesian norms.

Accordingly, the quantity used downstream in the validated weak coupling probability is calculated only after the effective-Hessian reaction coordinate has been determined.

4.4.14 Files generated by step 4

Step 4 creates the directory:

`MECP_gradient_definitions`

and writes the file:

`<jobname>_MECP_gradient_definitions.txt`

The file records:

- identities and multiplicities of both spin surfaces;
- the SPINKS and opposite gradient-difference conventions;
- $\| \mathbf{g}_{\text{LS}} \|^2$;

- $\| \mathbf{g}_{\text{HS}} \|$;
- $\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}}$;
- the peaked or sloped classification;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR;
- effective_hessian_mix_coefficient;
- all components of $\mathbf{g}_{\text{LS}}$;
- all components of $\mathbf{g}_{\text{HS}}$;
- all components of both sign conventions for the gradient difference; and
- all components of both normalized crossing directions.

The output also explicitly states that the projected PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is not defined in Step 4.

4.4.15 Variables exported by step 4

The principal vectors exported to later steps are:

```
surface_gradient_difference_cart
reverse_surface_gradient_difference_cart
surface_gradient_difference_cart
reverse_surface_gradient_difference_cart
gradient_difference_unit_vector
n_cart
MECP_seam_normal_cart
reaction_direction_cart
```

The principal scalar quantities are:

```
gradient_gap_norm
gradient_gap_magnitude
GRADIENT_GAP_NORM_EH_PER_BOHR
CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR
gradslope
intersection_type
effective_hessian_mix_coefficient
```

The original surface gradients and their norms also remain available:

```
g_LS
g_HS
g_LS_norm_Eh_per_Bohr
```

These quantities provide the complete gradient information required for the Hessian calculations and effective reaction-coordinate construction.

4.4.16 Summary of the step 4 algorithm

The complete sequence implemented in Step 4 can be summarized as:

$$\begin{array}{c} \{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}\} \\ \downarrow \\ \Delta \mathbf{G}_{\text{diff}} = \mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}} \end{array}$$

together with the reverse surface-gradient convention

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} = -\Delta \mathbf{G}_{\text{diff}}.$$

The common gradient-difference magnitude is

$$\Delta F_{\parallel} = \|\Delta \mathbf{G}_{\text{diff}}\| = \|\Delta \mathbf{g}\|.$$

The normalized directional vectors are

$$\mathbf{n}_{\text{diff}} = \frac{\Delta \mathbf{G}_{\text{diff}}}{\|\Delta \mathbf{G}_{\text{diff}}\|}$$

and

$$\mathbf{n}_{\text{cart}} = -\mathbf{n}_{\text{diff}}.$$

The crossing topology is classified through

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \begin{cases} < 0, & \text{peaked,} \\ \geq 0, & \text{sloped.} \end{cases}$$

The effective-Hessian weighting parameter is

$$\lambda = \frac{\Delta \mathbf{G}_{\text{diff}} \cdot \mathbf{g}_{\text{LS}}}{\|\Delta \mathbf{G}_{\text{diff}}\|^2}.$$

The Cartesian diagnostic gradient mean is

$$g_{\text{mean}}^{\text{Cartesian}} = \sqrt{\|\mathbf{g}_{\text{LS}}\| \|\mathbf{g}_{\text{HS}}\|},$$

but the final projected SPINKS PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is intentionally deferred until the effective-Hessian reaction coordinate has been obtained.

The corresponding variable flow is:

```

g_LS
g_HS
↓
surface_gradient_difference_cart = g_LS - g_HS
reverse_surface_gradient_difference_cart = g_HS - g_LS
↓
GRADIENT_GAP_NORM_EH_PER_BOHR
↓
gradient_difference_unit_vector
n_cart
↓
gradslope
intersection_type
effective_hessian_mix_coefficient
↓
CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR

```

Therefore, Step 4 transforms the raw Cartesian gradients from Step 3 into the selected-sign gradient difference, crossing-normal directions, crossing classification, and effective-Hessian weighting parameter:

$$\{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}\} \rightarrow \{\Delta \mathbf{G}_{\text{diff}}, \Delta F_{\parallel}, \mathbf{n}_{\text{diff}}, \lambda, \text{crossing type}\}$$

These quantities are subsequently combined with the two spin-state Hessians to construct the effective Hessian, identify the reaction-coordinate mode, calculate the reduced mass, and evaluate the final reaction-coordinate-projected PROJECTED_GRADIENT_MEAN_EH_PER_BOHR used in the weak coupling probability.

4.5 Step 5: MECP Hessians and Rotational Constants

Step 5 prepares the vibrational, rotational, symmetry, energy, and Hessian information required for the density-of-states treatment and for construction of the effective MECP Hessian in Step 6.

The reference-state vibrational data are reused directly from the optimized minimum obtained in Step 1. At the MECP, two separate SurfCrossNumFreq calculations are performed so that the Hessian information associated with both spin-surface orderings is available:

Job 1: xyz multiplicity = m_{main} , partner multiplicity = m_{other} ,

and

Job 2: xyz multiplicity = m_{other} , partner multiplicity = m_{main} .

Here, **main** denotes the multiplicity placed on the ORCA * xyz line, whereas **other** denotes the partner multiplicity specified in the %mecp block. The labels main/other and other/main therefore identify the two reversed spin-surface orderings used to obtain both MECP Hessians.

The raw ORCA PES2 frequencies generated by these calculations are retained only as diagnostics. They are not used as the final MECP frequencies in the downstream calculation. The final crossing-point frequencies are generated in Step 6 from the projected effective Hessian.

The principal operations performed in Step 5 are:

1. import of the reference minimum and MECP data from the preceding steps;
2. reuse of the reference electronic energy and zero-point energy from Step 1;
3. extraction of reference vibrational frequencies, rotational constants, and symmetry number;
4. construction of two SurfCrossNumFreq inputs at the MECP;
5. execution and validation of both MECP frequency calculations;
6. collection of the two MECP Hessian files;
7. extraction of MECP rotational constants and symmetry number;
8. extraction of the ORCA PES2 frequency block for diagnostic purposes;
9. calculation of the provisional electronic MECP barrier;
10. preparation of the downstream variables required by later steps; and
11. storage of all reference and MECP metadata.

4.5.1 Import of quantities from the preceding steps

Step 5 begins by verifying that the quantities needed from Steps 1–4 are available:

```
required_vars_step5 = [  
    "jobname",  
    "MECP_geometry",  
    "Reference_multiplicity",  
    "minima_data",  
    "local_base",  
    "method",  
    "basis",  
    "charge",  
    "mult_main",  
    "mult_other",  
    "nprocs",  
    "maxcore_mb",  
    "submit_and_monitor_orca_job",  
    "make_dft_pre_xyz",  
]
```

The most important imported quantities are:

```
Reference_multiplicity  
minima_data  
MECP_geometry  
mult_main  
mult_other
```

Reference_multiplicity identifies the lower-energy optimized minimum selected in Step 1, while minima_data contains its optimized geometry, electronic energy, zero-point energy, ORCA output path, and XYZ file path.

MECP_geometry contains the optimized crossing geometry at which the two Hessian calculations are performed.

4.5.2 Energy-unit conversion

Step 5 defines the conversion factor:

```
hartree_to_cm = 219474.6313705
```

which is used to convert electronic-energy differences from Hartree to wavenumbers:

$$\Delta E \text{ (cm}^{-1}\text{)} = \Delta E \text{ (E}_h\text{)} \times 219474.6313705.$$

This conversion is used later when the electronic MECP barrier is calculated relative to the selected reference minimum.

4.5.3 Reuse of reference-state data from step 1

The reference-state record is obtained directly from the minima_data dictionary:

```
ref_data = minima_data[Reference_multiplicity]
```

The optimized reference geometry is then assigned as:

```
Reference_geometry = ref_data["geometry"]
```

The reference electronic energy is extracted using either of the supported field names:

```
if "Electronic_Eh" in ref_data:  
    Ele_REF_hartree = float(ref_data["Electronic_Eh"])  
elif "Ele_hartree" in ref_data:  
    Ele_REF_hartree = float(ref_data["Ele_hartree"])
```

Similarly, the Step 1 zero-point energy is recovered as:

```
if "ZPE_Eh" in ref_data:  
    ZPE_REF_thermo_hartree = float(ref_data["ZPE_Eh"])  
elif "ZPE_hartree" in ref_data:  
    ZPE_REF_thermo_hartree = float(ref_data["ZPE_hartree"])
```

The corresponding aliases are:

```
E_ref_hartree = Ele_REF_hartree  
ZPE_REF_hartree = ZPE_REF_thermo_hartree  
ZPE_ref_hartree = ZPE_REF_thermo_hartree
```

Thus, Step 5 does not rerun the reference minimum frequency calculation. It reuses the already validated Step 1 output, avoiding unnecessary duplication and ensuring that the same optimized minimum defines the energetic and vibrational reference throughout the workflow.

4.5.4 Extraction of reference rotational constants and symmetry number

The Step 1 reference output is read from:

```
remote_ref_out = ref_data["out_path"]
```

and parsed using:

```
extract_symmetry_and_rot_constants_from_text(...)
```

The code searches for the ORCA lines containing:

```
Symmetry Number:
```

and:

```
Rotational constants in cm-1
```

The three rotational constants are stored as:

```
Reference_rot_constants_cm1
```

and the rotational symmetry number as:

```
Reference_symmetry_number
```

If ORCA does not explicitly print a symmetry number, the code assigns:

```
symmetry_number = 1
```

as the default.

The resulting quantities are used later in the rotational density-of-states correction:

$$\sigma_R = \text{reference rotational symmetry number,}$$

and

$$\mathbf{B}_R = (B_{R,A}, B_{R,B}, B_{R,C}).$$

4.5.5 Extraction and selection of the reference frequency block

An ORCA output can contain more than one vibrational-frequency section. SPINKS therefore identifies all blocks beginning with:

```
VIBRATIONAL FREQUENCIES
```

using:

```
extract_orca_frequency_blocks_from_text(...)
```

Each frequency block is separated into:

```
block["imag"]  
block["zero"]  
block["real"]
```

corresponding respectively to

$$\nu_i < 0, \nu_i \approx 0, \nu_i > 0.$$

The reference frequency block is selected using:

```
select_reference_frequency_block_from_text(...)
```

The selection procedure is:

1. identify all frequency blocks;
2. retain blocks containing no imaginary frequencies;

- among those, choose the block containing the largest number of real frequencies.

If all blocks contain imaginary frequencies, the code selects the block with the fewest imaginary modes and issues a warning.

The selected arrays are stored as:

```
freq_reactant_cm1_all
freq_reactant_real_cm1
freq_reactant_imag_cm1
freq_reactant_zero_cm1
```

The real frequencies are subsequently exported under the SPINKS name:

```
reference_vibrational_frequencies_cm1
```

and are used as the reactant vibrational frequencies in the density-of-states calculation.

4.5.6 Independent calculation of the reference ZPE from frequencies

Although the reference ZPE has already been obtained from the Step 1 thermochemistry output, Step 5 independently calculates it from the extracted positive frequencies:

```
def zpe_from_freqs_cm1(freqs_cm1):
    freqs = freqs[
        np.isfinite(freqs) & (freqs > 0.0)
    ]
    return 0.5 * float(np.sum(freqs))
```

Therefore,

$$\text{ZPE}_R^{(\text{cm}^{-1})} = \frac{1}{2} \sum_{\nu_i > 0} \nu_i.$$

The corresponding Hartree value is:

```
ZPE_REF_from_freq_hartree = (
    ZPE_REF_from_freq_cm1 / hartree_to_cm
)
```

or

$$\text{ZPE}_R^{(\text{E}_h)} = \frac{\text{ZPE}_R^{(\text{cm}^{-1})}}{219474.6313705}.$$

The two reference ZPE values are therefore kept separately:

```
ZPE_REF_thermo_hartree
```

```
ZPE_REF_from_freq_hartree
```

The first is the ZPE extracted from the Step 1 thermochemistry data, whereas the second is reconstructed directly from the selected positive vibrational frequencies.

This provides a useful internal consistency check.

4.5.7 Preparation of the MECP frequency directory

All Step 5 MECP frequency and Hessian files are stored in:

```
local_hess_dir = os.path.join(  
    local_base,  
    "Frequencies"  
)
```

The local implementation also assigns:

```
remote_hess_dir = local_hess_dir
```

as a compatibility alias.

The directory contains:

- both generated SurfCrossNumFreq inputs;
- both ORCA outputs;
- both Hessian files;
- copied reference information;
- diagnostic PES2 frequency information;
- rotational and symmetry metadata.

4.5.8 Construction of the SurfCrossNumFreq inputs

The MECP frequency inputs are generated using:

```
build_mecp_freq_input(  
    geom_text,  
    xyz_mult,  
    mec_p_mult  
)
```

The required MECP partner multiplicity is inserted through:

```
mecp_block = f"""%mecp Mult {int(mecp_mult)}  
end  
""
```

The reviewed Step 1 DFT header and all transferable percent blocks are then propagated using:

```
pre_xyz = make_dft_pre_xyz(  
    "freq",  
    job_specific_blocks=mecp_block  
)
```

The final ORCA input is constructed as:

```
return build_orca_input_from_pre_xyz(  
    pre_xyz,  
    charge,  
    xyz_mult,  
    geom_text  
)
```

This approach preserves user-reviewed settings such as:

- the DFT functional;
- the basis set;
- dispersion corrections;
- solvent models;
- custom basis or ECP blocks;
- integration-grid settings;
- SCF settings;
- parallelization settings; and
- memory settings.

At the same time, incompatible job keywords are removed and the required SurfCrossNumFreq specification is introduced.

4.5.9 Generation of both spin-surface orderings

Two separate inputs are generated:

mecp_main_other_inp = build_mecp_freq_input(

```
    MECP_geometry,  
    xyz_mult=mult_main,  
    mecpc_mult=mult_other  
)
```

and:

```
mecp_other_main_inp = build_mecp_freq_input(
    MECP_geometry,
    xyz_mult=mult_other,
    mecpc_mult=mult_main
)
```

The two job definitions are therefore:

```
freq_jobs = [
    {
        "xyz_mult": mult_main,
        "mecpc_mult": mult_other
    },
    {
        "xyz_mult": mult_other,
        "mecpc_mult": mult_main
    }
]
```

The use of both orderings ensures that Hessian information is available for both electronic surfaces at the same MECPC geometry.

The generated files are named:

```
<jobname>_Freq_MECPC_main_other.inp
<jobname>_Freq_MECPC_other_main.inp
```

This explicit naming preserves the relationship between the primary XYZ multiplicity and the partner MECPC multiplicity.

4.5.10 Execution and protected repair of the MECPC frequency jobs

Each frequency calculation is submitted using:

```
submit_and_monitor_orca_job(...)
```

with the repair policy:

```
repair_policy = "freq_restricted"
```

The locked quantities are:

```
locked = {
    "method": clean_method_for_frequency(method),
    "basis": basis,
    "charge": charge,
    "mult": job["xyz_mult"],
    "mecpc_mult": job["mecpc_mult"],
    "geom": MECPC_geometry,
}
```

Therefore, any repaired input must preserve:

- the selected method;

- the selected basis set;
- the molecular charge;
- the XYZ multiplicity;
- the partner MECP multiplicity;
- the optimized MECP geometry; and
- the required frequency job type.

The user may modify convergence-related settings if a calculation fails, but the scientifically defining quantities remain protected.

Step 5 proceeds only after both jobs have terminated normally.

4.5.11 Collection of the two Hessian files

After successful completion, the corresponding files are located explicitly:

```
local_mecp_main_other_hess = os.path.join(
    local_hess_dir,
    f"{freq_mecp_main_other_label}.hess"
)
```

and:

```
local_mecp_other_main_hess = os.path.join(
    local_hess_dir,
    f"{freq_mecp_other_main_label}.hess"
)
```

The primary Hessian paths are exported as:

```
MECP_hess_main_other_path
MECP_hess_other_main_path
```

These are the two central Step 5 outputs used in Step 6.

Conceptually, they provide the Hessian information associated with the two spin surfaces:

$$\mathbf{H}_{\text{main/other}}$$

and

$$\mathbf{H}_{\text{other/main}}$$

Step 6 subsequently combines the appropriate spin-surface Hessians using the effective-Hessian weighting parameter λ obtained in Step 4.

4.5.12 Extraction of MECP rotational constants and symmetry number

The rotational data are extracted from the main/other MECP output:

```
MECP_symmetry_number, MECP_rot_constants_cm1 = (
    extract_symmetry_and_rot_constants_from_text(
        mecp_main_other_out_text,
        source_label=local_mecp_main_other_out
    )
)
```

The resulting variables are:

```
MECP_symmetry_number
MECP_rot_constants_cm1
```

and are exported under the SPINKS names:

```
crossing_symmetry_factor
crossing_rotational_constants_cm1
```

Thus,

$$\sigma_X = \text{MECP rotational symmetry number,}$$

and

$$\mathbf{B}_X = (B_{X,A}, B_{X,B}, B_{X,C}).$$

These quantities are later used in evaluating the rotational contribution to the crossing-state density of states.

4.5.13 Extraction of diagnostic PES2 frequencies

The ORCA SurfCrossNumFreq output can contain several vibrational-frequency blocks. Step 5 searches specifically for blocks whose title contains:

```
PES2
```

using:

```
select_mecp_pes2_frequency_block_from_text(...)
```

If more than one PES2 block is present, the preferred block is the one with:

1. no imaginary frequencies; and
2. the largest number of real frequencies.

If all PES2 blocks contain imaginary modes, the block with the fewest imaginary frequencies is retained and a warning is printed.

The extracted arrays are stored as:

```
freq_MECP_PES2_real_cm1_diagnostic
freq_MECP_PES2_imag_cm1_diagnostic
freq_MECP_PES2_zero_cm1_diagnostic
```

These quantities are explicitly labeled as diagnostic. They are not assigned as the final MECP vibrational frequencies used in SPINKS:

NOTE: SPINKS `freX` must come from Step 6 projected effective Hessian.

This separation is important because the final crossing-point modes must be derived from the effective Hessian constructed from both spin surfaces, not from a single raw PES2 frequency block.

4.5.14 Electronic MECP barrier bookkeeping

Step 5 next determines the electronic energy of the optimized MECP.

The preferred source is the original Step 1 MECP optimization output:

```
candidate = os.path.join(
    local_mecp,
    f"{jobname}.out"
)
```

If that file is unavailable, the final single-point energy from the main/other frequency output is used instead.

The MECP electronic energy is extracted using:

```
extract_final_sp_energy_from_text(...)
```

which selects the final occurrence of:

```
FINAL SINGLE POINT ENERGY
```

The electronic MECP barrier relative to the reference minimum is then:

```
E_MECP_electronic_hartree = (
    Ele_MECP_hartree - Ele_REF_hartree
)
```

or

$$\Delta E_{\text{MECP}}^{\text{el}} = E_{\text{MECP}}^{\text{el}} - E_R^{\text{el}}.$$

The barrier in wavenumbers is:

```
VaG_MECP_electronic_cm1 = (  
    E_MECP_electronic_hartree *  
    hartree_to_cm  
)
```

or

$$V_{\text{MECP}}^{\text{el}} = (E_{\text{MECP}}^{\text{el}} - E_R^{\text{el}}) 219474.6313705.$$

The corresponding value in kJ mol^{-1} is calculated as:

```
VaG_MECP_kJmol = (  
    VaG_MECP_cm1 * 0.01196266  
)
```

At this point, the active barrier is provisionally assigned as:

```
VaG_MECP_cm1 = VaG_MECP_electronic_cm1
```

This is an electronic barrier only. The final zero-point-corrected barrier is determined later after the effective-Hessian frequencies have been obtained.

4.5.15 Preparation of downstream variables

The reference and MECP data are reorganized into the variable names used by the subsequent downstream calculations.

For the reactant:

```
reference_vibrational_frequencies_cm1 = freq_reactant_real_cm1  
reference_rotational_constants_cm1 = (  
    Reference_rot_constants_cm1  
)  
reference_symmetry_factor = float(Reference_symmetry_number)
```

For the crossing point:

```
crossing_rotational_constants_cm1 = (  
    MECP_rot_constants_cm1  
)  
crossing_symmetry_factor = float(MECP_symmetry_number)
```

The two Hessian paths are stored as:

```
MECP_hess_main_other_path  
MECP_hess_other_main_path
```

The resulting physical quantities are therefore:

$\{\nu_{R,i}\}$ = reference real vibrational frequencies,
 $\mathbf{B}_R$ = reference rotational constants,
 $\mathbf{B}_X$ = MECP rotational constants,
 σ_R = reference symmetry number,

and

σ_X = MECP symmetry number.

The final MECP effective frequencies, $\{\nu_{X,i}\}$, are not yet defined in Step 5.

4.5.16 Distinction between reference frequencies and MECP diagnostic frequencies

The reference frequencies:

[reference_vibrational_frequencies_cm1](#)

are final quantities because they originate from the validated optimized minimum frequency calculation.

In contrast, the MECP PES2 frequencies:

[freq_MECP_PES2_real_cm1_diagnostic](#)

are provisional diagnostics only.

This distinction can be summarized as:

Reference frequencies from Step 1 → used directly in SPINKS

whereas

Raw MECP PES2 frequencies from Step 5 → diagnostic only

and

Projected effective-Hessian frequencies from Step 6 → final MECP frequencies used in SPINKS

This prevents the density-of-states calculation from using frequencies belonging to only one raw crossing surface rather than the combined effective crossing-point Hessian.

4.5.17 Files generated by step 5

Step 5 creates the file:

<code><jobname>_rotational_constants_and_symmetry.txt</code>
--

which records:

- the reference output and XYZ paths;
- the reference multiplicity;
- the reference symmetry number;
- the reference rotational constants;
- both MECP output paths;
- both MECP Hessian paths;
- the XYZ and partner multiplicities for both jobs;
- the MECP symmetry number;
- the MECP rotational constants;
- the reference electronic energy;
- the MECP electronic energy;
- the source of the MECP energy;
- the electronic MECP barrier;
- the provisional active barrier;
- the numbers of reference real, imaginary, and zero frequencies;
- the reference ZPE reconstructed from frequencies; and
- the diagnostic MECP PES2 frequency counts.

The file explicitly notes that the final MECP frequencies are generated later from the projected effective Hessian.

Step 5 also attempts to copy the reference minimum output into the frequency directory under the name:

<code><jobname>_Reference_minimum.out</code>
--

to keep all vibrational and rotational source files together.

4.5.18 Variables exported by step 5

The principal reference-state variables exported are:

```
Ele_REF_hartree
ZPE_REF_thermo_hartree
ZPE_REF_from_freq_cm1
ZPE_REF_from_freq_hartree
freq_reactant_real_cm1
reference_vibrational_frequencies_cm1
Reference_rot_constants_cm1
Reference_symmetry_number
reference_symmetry_factor
```

The principal MECP variables are:

```
Ele_MECP_hartree
VaG_MECP_electronic_cm1
VaG_MECP_cm1
MECP_rot_constants_cm1
MECP_symmetry_number
crossing_symmetry_factor
MECP_hess_main_other_path
MECP_hess_other_main_path
```

The diagnostic MECP frequency variables are:

```
freq_MECP_PES2_real_cm1_diagnostic
freq_MECP_PES2_imag_cm1_diagnostic
freq_MECP_PES2_zero_cm1_diagnostic
```

The rotational constants used later are exported as:

```
reference_rotational_constants_cm1
crossing_rotational_constants_cm1
```

These quantities provide the complete reference and crossing-point metadata required for effective-Hessian construction, ZPE correction, and density-of-states calculations.

4.5.19 Summary of the step 5 algorithm

The complete sequence implemented in Step 5 can be summarized as:

Reference minimum data from Step 1
↓
 $\{E_R^{\text{el}}, \text{ZPE}_R, \{\nu_{R,i}\}, \mathbf{B}_R, \sigma_R\}$

where

$$\text{ZPE}_R = \frac{1}{2} \sum_{\nu_{R,i} > 0} \nu_{R,i}.$$

In parallel, the optimized MECP geometry is used to generate two Hessian calculations:

$$\begin{array}{c} \mathbf{R}_{\text{MECP}} \\ \downarrow \\ \left\{ \begin{array}{l} \text{SurfCrossNumFreq}(m_{\text{main}}, m_{\text{other}}), \\ \text{SurfCrossNumFreq}(m_{\text{other}}, m_{\text{main}}). \end{array} \right. \end{array}$$

These calculations produce

$$\{\mathbf{H}_{\text{main/other}}, \mathbf{H}_{\text{other/main}}\},$$

together with

$$\{\mathbf{B}_X, \sigma_X\}.$$

The provisional electronic crossing barrier is then calculated as

$$V_{\text{MECP}}^{\text{el}} = (E_{\text{MECP}}^{\text{el}} - E_R^{\text{el}}) 219474.6313705.$$

The raw ORCA PES2 frequencies are extracted only for diagnostic purposes:

$$\{\nu_{X,i}^{\text{PES2}}\} \text{diagnostic only.}$$

The final MECP frequencies remain undefined until Step 6:

$$\{\nu_{X,i}^{\text{eff}}\} = \text{eigenfrequencies of the projected effective Hessian.}$$

The corresponding variable flow is:

```
minima_data[Reference_multiplicity]
↓
Ele_REF_hartree
ZPE_REF_thermo_hartree
reference_vibrational_frequencies_cm1
Reference_rot_constants_cm1
```

```

reference_symmetry_factor
MECP_geometry
↓
SurfCrossNumFreq(main, other)
SurfCrossNumFreq(other, main)
↓
MECP_hess_main_other_path
MECP_hess_other_main_path
MECP_rot_constants_cm1
crossing_symmetry_factor
↓
Ele_MECP_hartree
VaG_MECP_electronic_cm1
↓
Step 6 effective-Hessian construction

```

Therefore, Step 5 transforms the validated reference minimum and optimized MECP geometry into the vibrational, rotational, symmetry, energy, and Hessian data required for the downstream statistical treatment:

$$\{\text{reference minimum, MECP geometry}\} \rightarrow \{\{\nu_{R,i}\}, \mathbf{B}_R, \sigma_R, \mathbf{H}_1, \mathbf{H}_2, \mathbf{B}_X, \sigma_X, V_{\text{MECP}}^{\text{el}}\}$$

The two MECP Hessians are subsequently combined in Step 6 using the effective-Hessian weighting parameter obtained in Step 4. The projected effective Hessian then provides the reaction-coordinate mode, reduced mass, final MECP vibrational frequencies, projected projected gradient mean, and zero-point correction used in the later probability and rate calculations.

4.6 Step 6: Effective Hessian and Reduced Mass

Step 6 combines the two MECP Hessians generated in Step 5 with the two spin-state gradients obtained in Step 3 to construct the SPINKS effective Hessian, identify the crossing reaction coordinate, calculate the corresponding reduced mass, and generate the transverse MECP vibrational frequencies used in the subsequent density-of-states calculation.

No additional ORCA calculation is performed in this step. All operations consist of numerical analysis of the previously generated Cartesian gradients, molecular geometry, atomic masses, and Hessian matrices.

The principal operations performed in Step 6 are:

1. parsing the MECP geometry and assigning atomic masses;
2. reconstructing the gradient-difference and force quantities;
3. evaluating the final projected `PROJECTED_GRADIENT_MEAN_EH_PER_BOHR`;
4. reading and validating the two ORCA Hessian files;
5. transforming both Hessians to mass-weighted coordinates;
6. constructing peaked- and sloped-crossing effective Hessians;
7. removing translational and rotational motion;
8. generating several candidate reaction-coordinate vectors;
9. selecting the validated reaction-coordinate convention;
10. evaluating alternative and final reduced masses;
11. projecting translation, rotation, and the reaction coordinate from the effective Hessian;
12. diagonalizing the transverse effective Hessian;
13. converting the eigenvalues into MECP frequencies; and
14. exporting the final gradient, mass, reaction-coordinate, and frequency quantities.

The principal downstream outputs are:

<code>GRADIENT_GAP_NORM_EH_PER_BOHR</code> <code>PROJECTED_GRADIENT_MEAN_EH_PER_BOHR</code> <code>reduced_mass_amu</code> <code>freq_MECP_effhess_real_cm1</code> <code>reaction_direction_hessian_cart</code> <code>reaction_direction_hessian_mw</code>
--

4.6.1 Import of gradient and Hessian data

Step 6 requires the two Cartesian gradient vectors, the optimized MECP geometry, and the two Hessian files generated in Step 5:

```
required_vars_step6 = [  
    "jobname",  
    "MECP_geometry",  
    "g_LS",  
    "g_HS",  
    "mult_LS",  
    "mult_HS",  
    "spin_name_LS",  
    "spin_name_HS",  
    "MECP_hess_main_other_path",  
    "MECP_hess_other_main_path",  
    ...  
]
```

The two Hessian paths are:

```
MECP_hess_main_other_path  
MECP_hess_other_main_path
```

and correspond to the two SurfCrossNumFreq calculations in which the multiplicities were interchanged between the primary XYZ state and the MECP partner state.

Step 6 terminates if any required gradient, geometry, or Hessian quantity is unavailable.

4.6.2 MECP geometry parsing and atomic-mass assignment

The Cartesian geometry is parsed atom by atom from:

```
MECP_geometry
```

to obtain the element symbols and coordinates:

```
symbols_mecp.append(sym)  
  
coords_mecp.append([  
    float(parts[1]),  
    float(parts[2]),  
    float(parts[3])  
])
```

The resulting arrays are:

```
symbols_mecp  
coords_mecp
```

For a system containing N atoms, the Cartesian dimension is

$$N_{\text{cart}} = 3N.$$

The code therefore defines:

```
natoms = len(symbols_mecp)
num_cart = 3 * natoms
```

Each atom is assigned a standard isotopic-average atomic mass from the internal atomic_masses dictionary. The atomic masses are stored as:

```
masses
```

and expanded into a Cartesian mass vector:

```
mass_vector = np.repeat(masses, 3)
```

Thus,

$$\mathbf{m} = (m_1, m_1, m_1, m_2, m_2, m_2, \dots, m_N, m_N, m_N).$$

The square-root mass vector is also generated:

```
sqrt_mass_vector = np.sqrt(mass_vector)
```

These quantities are used for mass weighting, projector construction, and reduced-mass evaluation.

4.6.3 Validation of the gradient dimensions

The low-spin and high-spin gradients are converted to one-dimensional NumPy arrays:

```
g_LS = np.asarray(g_LS, dtype=float).reshape(-1)
g_HS = np.asarray(g_HS, dtype=float).reshape(-1)
```

Each gradient must contain exactly $3N$ components:

```
if g_LS.size != num_cart:
    raise RuntimeError(...)

if g_HS.size != num_cart:
    raise RuntimeError(...)
```

This validation ensures that both gradients correspond exactly to the parsed MECP geometry and the Hessian dimensions.

The individual norms are then calculated as:

```
gLS_norm = float(np.linalg.norm(g_LS))
gHS_norm = float(np.linalg.norm(g_HS))
```

or

$$\| \mathbf{g}_{\text{LS}} \| = \sqrt{\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{LS}}},$$

$$\| \mathbf{g}_{\text{HS}} \| = \sqrt{\mathbf{g}_{\text{HS}} \cdot \mathbf{g}_{\text{HS}}}.$$

4.6.4 Gradient-difference and gradient-sum vectors

Step 6 reconstructs both the difference and sum of the two surface gradients:

```
reverse_surface_gradient_difference_cart = g_HS - g_LS
surface_gradient_sum_cart = g_HS + g_LS
```

The gradient-difference vector is

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}},$$

whereas the gradient-sum vector is

$$\mathbf{g}_{\text{sum}} = \mathbf{g}_{\text{HS}} + \mathbf{g}_{\text{LS}}$$

The selected-sign convention is also retained:

```
surface_gradient_difference_cart = g_LS - g_HS
```

such that

$$\Delta \mathbf{G}_{\text{diff}} = -\Delta \mathbf{g}.$$

As in Step 4, both sign conventions are preserved explicitly because different portions of the workflow use different vector orientations.

4.6.5 Gradient-difference magnitude and normalized crossing direction

The magnitude of the Cartesian gradient difference is calculated as:

```
GRADIENT_GAP_NORM_EH_PER_BOHR = float(
    np.linalg.norm(reverse_surface_gradient_difference_cart)
)
```

or

$$\Delta F_{\parallel} = \| \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \|.$$

The principal exported variable is:

```
GRADIENT_GAP_NORM_EH_PER_BOHR
```

with units of Hartree per bohr.

The corresponding normalized vector is:

```
n_cart = (  
    reverse_surface_gradient_difference_cart /  
    GRADIENT_GAP_NORM_EH_PER_BOHR  
)
```

or

$$\mathbf{n} = \frac{\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}\|}$$

The opposite SPINKS orientation is:

```
gradient_difference_unit_vector = (  
    surface_gradient_difference_cart /  
    GRADIENT_GAP_NORM_EH_PER_BOHR  
)
```

and therefore

$$\mathbf{n}_{\text{diff}} = -\mathbf{n}.$$

At this point, the aliases:

```
reaction_direction_cart  
MECP_seam_normal_cart
```

are assigned to the normalized Cartesian gradient-difference direction.

4.6.6 Projected gradient components and the final projected gradient mean

Step 6 projects each spin-state gradient onto the normalized gradient-difference direction:

```
low_spin_projected_gradient_signed = float(np.dot(g_LS, n_cart))  
high_spin_projected_gradient_signed = float(np.dot(g_HS, n_cart))
```

Thus,

$$\begin{aligned} F_{\text{LS}}^{\text{signed}} &= \mathbf{g}_{\text{LS}} \cdot \mathbf{n}, \\ F_{\text{HS}}^{\text{signed}} &= \mathbf{g}_{\text{HS}} \cdot \mathbf{n}. \end{aligned}$$

The magnitudes of the projected force components are then:

```
low_spin_projected_gradient_Eh_per_Bohr = abs(low_spin_projected_gradient_signed)  
high_spin_projected_gradient_Eh_per_Bohr = abs(high_spin_projected_gradient_signed)
```

or

$$F_{LS,\parallel} = |\mathbf{g}_{LS} \cdot \mathbf{n}|,$$
$$F_{HS,\parallel} = |\mathbf{g}_{HS} \cdot \mathbf{n}|.$$

The authoritative MECP gradient mean is then calculated as:

```
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR = float(  
    np.sqrt(  
        low_spin_projected_gradient_Eh_per_Bohr *  
        high_spin_projected_gradient_Eh_per_Bohr  
    )  
)
```

corresponding to

$$\text{GRADMEAN} = \sqrt{F_{LS,\parallel} F_{HS,\parallel}}.$$

This quantity differs from the Cartesian diagnostic calculated in Step 4:

$$\sqrt{\|\mathbf{g}_{LS}\| \|\mathbf{g}_{HS}\|}.$$

The Step 6 value uses only the gradient components parallel to the crossing direction and is therefore the authoritative value passed to the weak coupling probability calculation.

The backward-compatible alias is:

```
projected_gradient_mean_Eh_per_Bohr = PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
```

4.6.7 Direct Cartesian gradient-direction reduced mass

Before the effective-Hessian reaction coordinate is constructed, Step 6 calculates a diagnostic reduced mass directly from the normalized Cartesian gradient-difference vector:

```
inverse_mass_along_gradient_gap = float(  
    np.sum((n_cart ** 2) / mass_vector)  
)  
  
gradient_gap_reduced_mass_amu = (  
    1.0 / inverse_mass_along_gradient_gap  
)
```

Mathematically,

$$\frac{1}{\mu_{\Delta g}} = \sum_{i=1}^{3N} \frac{n_i^2}{m_i},$$

and therefore

$$\mu_{\Delta g} = \left[\sum_{i=1}^{3N} \frac{n_i^2}{m_i} \right]^{-1}.$$

This mass is retained as a diagnostic. It is not automatically assigned as the final reduced mass used downstream.

4.6.8 Crossing classification

The gradient dot product is evaluated as:

```
grad_dot = float(np.dot(g_LS, g_HS))
```

The crossing is classified using:

```
if grad_dot < 0.0:
    intersection_type = "peaked"
else:
    intersection_type = "sloped"
```

Therefore,

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} < 0 \Rightarrow \text{peaked},$$

whereas

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \geq 0 \Rightarrow \text{sloped}.$$

This classification determines which effective-Hessian expression is selected later in the step.

4.6.9 ORCA Hessian parsing

Each ORCA .hess file is read using:

```
read_orca_hessian(
    hess_path,
    expected_dim
)
```

The parser first attempts to interpret the file as a plain numerical matrix. If that fails, it locates the ORCA

\$Hessian

block and reconstructs the matrix from the blockwise column format.

The parser verifies that the Hessian dimension is exactly

$$3N \times 3N.$$

The reconstructed matrix is symmetrized as:

```
H = 0.5 * (H + H.T)
```

which enforces

$$\mathbf{H} = \frac{1}{2}(\mathbf{H} + \mathbf{H}^T).$$

The parser also rejects a matrix containing only zeros.

The two Cartesian Hessians are stored as:

```
H_mecp_main_other_cart
H_mecp_other_main_cart
```

4.6.10 Mass weighting of the Hessians

The mass-weighting matrix is constructed as:

```
mass_weight_matrix = (
    1.0 /
    np.sqrt(
        np.outer(mass_vector, mass_vector)
    )
)
```

For Cartesian coordinates i and j ,

$$W_{ij} = \frac{1}{\sqrt{m_i m_j}}.$$

The two mass-weighted Hessians are therefore:

```
H_mecp_main_other_mw = (
  H_mecp_main_other_cart *
  mass_weight_matrix
)

H_mecp_other_main_mw = (
  H_mecp_other_main_cart *
  mass_weight_matrix
)
```

corresponding to

$$H_{ij}^{\text{mw}} = \frac{H_{ij}^{\text{cart}}}{\sqrt{m_i m_j}}.$$

All effective-Hessian construction, projection, and diagonalization are subsequently performed in mass-weighted coordinates.

4.6.11 Construction of the peaked-crossing effective Hessian

For a peaked crossing, Step 6 constructs the gradient-norm-weighted sum:

```
H_eff_mw_plus = (
  gHS_norm * H_mecp_main_other_mw
  + gLS_norm * H_mecp_other_main_mw
) / (gLS_norm + gHS_norm)
```

or

$$\mathbf{H}_{\text{eff}}^{(+)} = \frac{\|\mathbf{g}_{\text{HS}}\| \mathbf{H}_1 + \|\mathbf{g}_{\text{LS}}\| \mathbf{H}_2}{\|\mathbf{g}_{\text{LS}}\| + \|\mathbf{g}_{\text{HS}}\|}.$$

This expression provides a weighted combination of the two surface Hessians in which the weights are determined by the corresponding gradient magnitudes.

4.6.12 Construction of the sloped-crossing effective Hessian

For a sloped crossing, Step 6 constructs the weighted difference:

```
H_eff_mw_minus = (
  gHS_norm * H_mecp_main_other_mw
  - gLS_norm * H_mecp_other_main_mw
) / (gHS_norm - gLS_norm)
```

or

$$\mathbf{H}_{\text{eff}}^{(-)} = \frac{\|\mathbf{g}_{\text{HS}}\| \mathbf{H}_1 - \|\mathbf{g}_{\text{LS}}\| \mathbf{H}_2}{\|\mathbf{g}_{\text{HS}}\| - \|\mathbf{g}_{\text{LS}}\|}.$$

If the two gradient norms are numerically equal, the denominator would approach zero. In that case, the implementation falls back to the peaked-form Hessian:

```
if abs(gHS_norm - gLS_norm) > 1.0e-14:  
    ...  
else:  
    H_eff_mw_minus = H_eff_mw_plus.copy()
```

This avoids numerical instability.

4.6.13 Selection of the active effective Hessian

The final effective Hessian is selected according to the crossing classification:

```
if intersection_type == "peaked":  
    H_eff_mw_selected = H_eff_mw_plus.copy()  
    effective_hessian_choice = "plus/peaked"  
else:  
    H_eff_mw_selected = H_eff_mw_minus.copy()  
    effective_hessian_choice = "minus/sloped"
```

Thus,

$$\mathbf{H}_{\text{eff}} = \begin{cases} \mathbf{H}_{\text{eff}}^{(+)}, & \text{peaked crossing,} \\ \mathbf{H}_{\text{eff}}^{(-)}, & \text{sloped crossing.} \end{cases}$$

For diagnostic purposes, the arithmetic average is also stored:

```
H_eff_mw_average = 0.5 * (  
    H_mecp_main_other_mw +  
    H_mecp_other_main_mw  
)
```

However, the selected gradient-weighted effective Hessian is the matrix used for reaction-coordinate and frequency analysis.

4.6.14 Construction of the translational and rotational subspace

The center of mass is calculated as:

```
R_cm = (
```

```

np.sum(
    coords_mecp * masses[:, None],
    axis=0
) / np.sum(masses)

```

and the geometry is shifted to the center-of-mass frame:

```
coords_centered = coords_mecp - R_cm
```

A $3N \times 6$ matrix, Z_{raw} , is then constructed from three mass-weighted translational vectors and three mass-weighted rotational vectors.

The translational vectors have components proportional to

$$\sqrt{m_a}$$

along each Cartesian axis.

The rotational vectors are generated from the centered atomic coordinates and atomic masses.

The independent rigid-body vectors are orthonormalized using:

```
Z = orth(Z_raw)
```

The translational/rotational projector is:

```
P_TR = Z @ Z.T
```

and the complementary vibrational-space projector is:

```
P_RT = np.eye(num_cart) - P_TR
```

Thus,

$$\mathbf{P}_{\text{TR}} = \mathbf{Z}\mathbf{Z}^T,$$

and

$$\mathbf{P}_{\text{RT}} = \mathbf{I} - \mathbf{P}_{\text{TR}}.$$

The code prints the projector idempotency errors

$$\| \mathbf{P}^2 - \mathbf{P} \|$$

as numerical diagnostics.

For a nonlinear molecule, the rigid-body subspace normally has dimension six. For a linear molecule, linear dependence among the rotational vectors may reduce this dimension to five automatically through orthogonalization.

4.6.15 Generation of candidate reaction-coordinate seeds

Step 6 evaluates four Cartesian seed definitions:

```
reverse_surface_gradient_difference_cart
surface_gradient_sum_cart
g_HS_unit - g_LS_unit
g_HS_unit + g_LS_unit
```

These correspond to

$$\begin{aligned} \mathbf{s}_1 &= \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}, \\ \mathbf{s}_2 &= \mathbf{g}_{\text{HS}} + \mathbf{g}_{\text{LS}}, \\ \mathbf{s}_3 &= \frac{\mathbf{g}_{\text{HS}}}{\|\mathbf{g}_{\text{HS}}\|} - \frac{\mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{LS}}\|}, \end{aligned}$$

and

$$\mathbf{s}_4 = \frac{\mathbf{g}_{\text{HS}}}{\|\mathbf{g}_{\text{HS}}\|} + \frac{\mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{LS}}\|}.$$

Each seed is tested under two mass-weighting conventions:

```
seed_mw_div = seed_cart / sqrt_mass_vector
seed_mw_mul = seed_cart * sqrt_mass_vector
```

or

$$\mathbf{s}_{\text{div}}^{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{s},$$

and

$$\mathbf{s}_{\text{mul}}^{\text{mw}} = \mathbf{M}^{1/2} \mathbf{s}.$$

This produces eight candidate reaction-coordinate definitions for diagnostic comparison.

4.6.16 Removal of rigid-body contamination from each seed

Each mass-weighted seed is projected into the nonrigid subspace:

```
seed_mw = P_RT @ seed_mw_raw
```

and normalized:

```
seed_mw = normalize_vector(...)
```

This removes any translational or rotational contamination from the candidate direction.

The reaction-coordinate projector associated with the seed is then:

```
P_RC_seed = np.outer(seed_mw, seed_mw)
```

or

$$\mathbf{P}_{\text{RC}} = \mathbf{s}_{\text{mw}} \mathbf{s}_{\text{mw}}^T.$$

The combined projector over translation, rotation, and the reaction coordinate is:

```
P_TOTAL = P_TR + P_RC_seed
```

or

$$\mathbf{P}_{\text{TOTAL}} = \mathbf{P}_{\text{TR}} + \mathbf{P}_{\text{RC}}.$$

4.6.17 Projected zero-mode reaction-coordinate analysis

For each candidate, the effective Hessian is projected to remove translation, rotation, and the seed reaction-coordinate direction:

```
H_proj = (  
    (I - P_TOTAL)  
    @ H_eff_mw_selected  
    @ (I - P_TOTAL)  
)
```

or

$$\mathbf{H}_{\text{proj}} = (\mathbf{I} - \mathbf{P}_{\text{TOTAL}}) \mathbf{H}_{\text{eff}} (\mathbf{I} - \mathbf{P}_{\text{TOTAL}}).$$

The projected Hessian is diagonalized:

```
eigvals, eigvecs = np.linalg.eigh(H_proj)
```

The eigenvectors associated with the smallest absolute eigenvalues are examined to identify the removed reaction-coordinate zero mode.

Each candidate eigenvector is scored using:

- overlap with the seed reaction coordinate;
- overlap with the translation/rotation subspace; and
- magnitude of its eigenvalue.

The preferred zero-mode vector has high reaction-coordinate overlap, low rigid-body overlap, and a near-zero eigenvalue.

The corresponding reduced mass is stored as:

```
mu_zero_amu
```

and described in the output as zero-mode reduced mass.

4.6.18 Direct reaction-coordinate reduced mass

Step 6 also constructs the one-dimensional reaction-coordinate Hessian block:

```
H_RC = (  
    P_RC_seed  
    @ H_eff_mw_selected  
    @ P_RC_seed  
)
```

or

$$\mathbf{H}_{\text{RC}} = \mathbf{P}_{\text{RC}} \mathbf{H}_{\text{eff}} \mathbf{P}_{\text{RC}}.$$

The corresponding reaction-coordinate eigenvector is identified as the eigenvector with the largest overlap with the seed:

```
idx_rc = int(  
    np.argmax(  
        np.abs(eigvecs_rc.T @ seed_mw)  
    )  
)
```

The resulting vector is stored as:

```
k_rc_direct
```

and converted into a Cartesian-like displacement vector by:

```
k_cart_mass_unweighted = (  
    k_mw / sqrt_mass_vector  
)
```

The associated reduced mass is:

```
mu_direct = 1.0 / np.dot(
    k_cart_mass_unweighted,
    k_cart_mass_unweighted
)
```

or

$$\mu_{\text{RC}} = \left[\sum_{i=1}^{3N} \left(\frac{k_i^{\text{mw}}}{\sqrt{m_i}} \right)^2 \right]^{-1}.$$

This value is described in the workflow as direct reaction-coordinate mass.

4.6.19 Automatic selection of the final reaction coordinate

Although eight candidate definitions are evaluated, the final selection rule is fixed explicitly:

```
if (
    cand["label"].startswith("Δg")
    and cand["convention"] == "seed/sqrt(m)"
):
    selected_rc = cand
```

Therefore, SPINKS selects:

$$\mathbf{s} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$$

using the mass-weighted convention

$$\mathbf{s}^{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{s}.$$

The final reduced mass is assigned from the direct reaction-coordinate result:

```
reduced_mass_amu = float(
    selected_rc["mu_direct_amu"]
)
```

Thus,

$$\mu_{\text{final}} = \mu_{\text{direct}}[\Delta \mathbf{g} / \sqrt{\mathbf{m}}]$$

The selected mass-weighted and Cartesian reaction-coordinate vectors are exported as:

```
reaction_direction_hessian_mw
reaction_direction_hessian_cart
```

The Cartesian vector is normalized before export.

The alternative quantities:

`reduced_mass_effhess_zero_selected_amu`
`reduced_mass_effhess_direct_selected_amu`

are retained separately for validation and reporting.

4.6.20 Distinction among the three reduced masses

Step 6 produces three related reduced-mass quantities:

Cartesian gradient-direction mass:

`gradient_gap_reduced_mass_amu`

calculated directly from the normalized Cartesian $\Delta\mathbf{g}$ direction.

Effective-Hessian zero-mode mass:

`reduced_mass_effhess_zero_selected_amu`

calculated from the zero eigenvector recovered after projection of the selected reaction-coordinate direction.

Effective-Hessian direct mass:

`reduced_mass_effhess_direct_selected_amu`

calculated directly from the selected reaction-coordinate block of the effective Hessian.

The final downstream value is:

`reduced_mass_amu`

and is assigned as

$$\text{reduced_mass_amu} = \text{reduced_mass_effhess_direct_selected_amu}.$$

The other two values remain available as diagnostics.

4.6.21 Projection of the effective Hessian into the transverse subspace

After selection of the final reaction coordinate, Step 6 removes translation, rotation, and reaction-coordinate motion from the effective Hessian.

The retained transverse-space projector is:

```
P_KEEP = np.eye(num_cart) - P_TOTAL
```

or

$$\mathbf{P}_{\text{keep}} = \mathbf{I} - \mathbf{P}_{\text{TR}} - \mathbf{P}_{\text{RC}}.$$

The transverse effective Hessian is then:

```
H_eff_projected_transverse = (  
    P_KEEP  
    @ H_eff_mw_selected  
    @ P_KEEP  
)
```

or

$$\mathbf{H}_{\text{eff}}^{\perp} = \mathbf{P}_{\text{keep}} \mathbf{H}_{\text{eff}} \mathbf{P}_{\text{keep}}.$$

The matrix is symmetrized before diagonalization.

This projected Hessian contains only vibrational motion transverse to the crossing reaction coordinate.

4.6.22 Diagonalization of the projected effective Hessian

The transverse effective Hessian is diagonalized using:

```
eigvals_eff, eigvecs_eff = np.linalg.eigh(  
    H_eff_projected_transverse  
)
```

yielding

$$\mathbf{H}_{\text{eff}}^{\perp} \mathbf{v}_i = \lambda_i \mathbf{v}_i.$$

The eigenvalues are sorted in ascending order.

Each eigenvector is then tested for residual overlap with:

- the rigid-body basis, $\mathbf{Z}$; and
- the selected reaction-coordinate vector

The overlaps are calculated as:

```
rt_overlap_eff  
rc_overlap_eff
```

A mode is classified as zero-like if either overlap exceeds the numerical threshold:

```
zero_like_mask = (  
    (rt_overlap_eff > 1.0e-6)  
    | (rc_overlap_eff > 1.0e-6)  
)
```

This prevents translational, rotational, or reaction-coordinate modes from being misclassified as physical transverse vibrations.

4.6.23 Conversion of effective-Hessian eigenvalues to frequencies

The nonzero eigenvalues are converted to wavenumbers using:

```
conversion_factor = 5140.48
```

and:

```
freq_eff_cm1[j] = (  
    np.sign(val)  
    * np.sqrt(abs(val))  
    * conversion_factor  
)
```

Thus,

$$\nu_i = \text{sgn}(\lambda_i) \sqrt{|\lambda_i|} \times 5140.48.$$

Positive eigenvalues produce real frequencies:

$$\lambda_i > 0 \Rightarrow \nu_i > 0,$$

whereas negative eigenvalues produce signed imaginary frequencies:

$$\lambda_i < 0 \Rightarrow \nu_i < 0.$$

Modes identified as translational, rotational, or reaction-coordinate-like are assigned zero frequency.

The final arrays are:

```
freq_MECP_effhess_real_cm1  
freq_MECP_effhess_imag_cm1  
freq_MECP_effhess_zero_cm1
```

The aliases used in later steps are:

```
freq_MECP_real_cm1
```

freq_MECP_imag_cm1 freq_MECP_zero_cm1
--

The real transverse frequencies are the authoritative MECP frequencies used in the density-of-states calculation.

4.6.24 Expected number of effective-Hessian frequencies

For a nonlinear molecule with N atoms, the full Cartesian space contains $3N$ coordinates.

After removing:

- three translations;
- three rotations; and
- one reaction-coordinate direction,

the expected number of transverse vibrational modes is

$$3N - 7.$$

For a linear molecule, the expected number is

$$3N - 6,$$

because only two independent rotational modes are present.

The implementation does not hard-code these numbers. Instead, it constructs the rigid-body subspace numerically and identifies zero-like modes through projector overlaps. This allows the same procedure to accommodate both linear and nonlinear geometries.

4.6.25 Storage of the two original Hessians

The Cartesian Hessians are stored together in the three-dimensional array:

Hessian_data_MECP

with dimensions

$$(3N) \times (3N) \times 2.$$

The first slice contains:

[H_mecp_main_other_cart](#)

and the second contains:

[H_mecp_other_main_cart.](#)

The mass-weighted forms are stored similarly in:

[mw_Hessian_data_MECp](#)

These arrays preserve the original surface-specific Hessian information for later diagnostics or independent analysis.

4.6.26 Reaction-coordinate candidate diagnostics

All eight candidate reaction-coordinate analyses are retained in:

[rc_candidates](#)

For each candidate, SPINKS records:

- seed definition;
- mass-weighting convention;
- zero-mode reduced mass;
- direct reduced mass;
- selected near-zero eigenvalue;
- overlap with the seed reaction coordinate; and
- overlap with the translation/rotation subspace.

A typical diagnostic record contains:

```
{  
  "label": ...,  
  "convention": ...,  
  "mu_zero_amu": ...,  
  "mu_direct_amu": ...,  
  "best_zero_eigval": ...,  
  "rc_overlap_zero": ...,  
  "rt_overlap_zero": ...  
}
```

Although only one convention is used downstream, retaining all candidates makes the reaction-coordinate selection transparent and allows comparison among alternative definitions.

4.6.27 Atomic contributions to the selected reaction coordinate

The Cartesian reaction-coordinate vector is divided into three-component atomic displacements. The amplitude associated with atom a is calculated as

$$A_a = \sqrt{k_{ax}^2 + k_{ay}^2 + k_{az}^2}.$$

The implementation is:

```
amp = float(
    np.linalg.norm(
        k_cart_like[idx:idx + 3]
    )
)
```

The atoms are ranked by decreasing amplitude.

This analysis identifies which atoms contribute most strongly to the effective crossing reaction coordinate and provides a chemically interpretable description of the spin-crossing motion.

4.6.28 Files generated by step 6

Step 6 creates the directory:

[Effective_Hessian_RC](#)

and writes three principal output files.

Effective-Hessian and reduced-mass summary:

[<jobname>_Step6_effective_Hessian_RC_summary.txt](#)

This file contains:

- both gradient norms;
- the gradient dot product;
- crossing classification;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- both projected force magnitudes;
- PROJECTED_GRADIENT_MEAN_EH_PER_BOHR;
- both Hessian paths;
- the selected effective-Hessian expression;

- all three reduced masses;
- the selected reaction-coordinate definition;
- zero-mode and overlap diagnostics;
- frequency counts;
- all reaction-coordinate candidates; and
- ranked atomic contributions.

Effective-Hessian frequency file:

`<jobname>_Step6_effective_Hessian_frequencies.txt`

This file lists separately:

- real transverse frequencies;
- imaginary transverse frequencies; and
- zero-like frequencies.

Reaction-coordinate vector file:

`<jobname>_Step6_reaction_coordinate_vectors.txt`

This file contains:

- the selected Cartesian effective-Hessian reaction coordinate;
- the selected mass-weighted reaction coordinate;
- the normalized $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$ direction; and
- the opposite selected-sign direction.

4.6.29 Variables exported by step 6

The primary gradient variables exported are:

`GRADIENT_GAP_NORM_EH_PER_BOHR`
`low_spin_projected_gradient_Eh_per_Bohr`
`high_spin_projected_gradient_Eh_per_Bohr`
`PROJECTED_GRADIENT_MEAN_EH_PER_BOHR`

The primary reduced-mass and reaction-coordinate variables are:

`reduced_mass_amu`
`reaction_direction_hessian_cart`
`reaction_direction_hessian_mw`

The diagnostic mass variables are:

```
gradient_gap_reduced_mass_amu  
reduced_mass_effhess_zero_selected_amu  
reduced_mass_effhess_direct_selected_amu
```

The primary Hessian variables are:

```
H_eff_mw_selected  
H_eff_mw_plus  
H_eff_mw_minus  
H_eff_projected_transverse
```

The final MECP frequency variables are:

```
freq_MECP_effhess_real_cm1  
freq_MECP_effhess_imag_cm1  
freq_MECP_effhess_zero_cm1
```

The associated projectors and bases are also retained:

```
Z  
P_TR  
P_RT  
P_RC  
P_TOTAL
```

These quantities provide all gradient, effective-Hessian, reaction-coordinate, mass, and MECP frequency information required by the subsequent energetic, probability, and density-of-states calculations.

4.6.30 Summary of the step 6 algorithm

The complete sequence implemented in Step 6 can be summarized as follows.

The two Cartesian gradients and Hessians are imported:

$$\{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}, \mathbf{H}_1, \mathbf{H}_2\}.$$

The gradient-difference direction is constructed:

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}},$$
$$\mathbf{n} = \frac{\Delta \mathbf{g}}{\|\Delta \mathbf{g}\|},$$

and

$$\Delta F_{\parallel} = \|\Delta \mathbf{g}\|.$$

The two surface gradients are projected onto this direction:

$$F_{LS,\parallel} = | \mathbf{g}_{LS} \cdot \mathbf{n} |,$$

$$F_{HS,\parallel} = | \mathbf{g}_{HS} \cdot \mathbf{n} |,$$

giving the final MECP gradient mean:

$$\text{GRADMEAN} = \sqrt{F_{LS,\parallel} F_{HS,\parallel}}.$$

The two Cartesian Hessians are mass weighted:

$$\mathbf{H}_i^{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{H}_i \mathbf{M}^{-1/2}.$$

The effective Hessian is selected according to the crossing topology:

$$\mathbf{H}_{\text{eff}} = \begin{cases} \frac{\| \mathbf{g}_{HS} \| \mathbf{H}_1 + \| \mathbf{g}_{LS} \| \mathbf{H}_2}{\| \mathbf{g}_{HS} \| + \| \mathbf{g}_{LS} \|}, & \text{peaked,} \\ \frac{\| \mathbf{g}_{HS} \| \mathbf{H}_1 - \| \mathbf{g}_{LS} \| \mathbf{H}_2}{\| \mathbf{g}_{HS} \| - \| \mathbf{g}_{LS} \|}, & \text{sloped.} \end{cases}$$

The translational and rotational subspace is removed:

$$\mathbf{P}_{\text{RT}} = \mathbf{I} - \mathbf{P}_{\text{TR}}.$$

The selected reaction-coordinate seed is

$$\mathbf{s}_{\text{RC}} = \mathbf{M}^{-1/2} (\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}).$$

Its projector is

$$\mathbf{P}_{\text{RC}} = \mathbf{s}_{\text{RC}} \mathbf{s}_{\text{RC}}^{\text{T}}.$$

The final reduced mass is obtained from the direct effective-Hessian reaction-coordinate vector:

$$\mu_{\text{RC}} = \left[\sum_i \left(\frac{k_i^{\text{mw}}}{\sqrt{m_i}} \right)^2 \right]^{-1}.$$

The effective Hessian is then projected into the transverse subspace:

$$\mathbf{H}_{\text{eff}}^{\perp} = (\mathbf{I} - \mathbf{P}_{\text{TR}} - \mathbf{P}_{\text{RC}}) \mathbf{H}_{\text{eff}} (\mathbf{I} - \mathbf{P}_{\text{TR}} - \mathbf{P}_{\text{RC}}).$$

Diagonalization gives

$$\mathbf{H}_{\text{eff}}^{\perp} \mathbf{v}_i = \lambda_i \mathbf{v}_i,$$

and the transverse MECP frequencies are obtained from

$$\nu_i = \text{sgn}(\lambda_i) \sqrt{|\lambda_i|} \times 5140.48.$$

The corresponding variable flow is:

```

MECP_geometry
g_LS
g_HS
MECP_hess_main_other_path
MECP_hess_other_main_path
↓
masses
mass_vector
reverse_surface_gradient_difference_cart
n_cart
↓
GRADIENT_GAP_NORM_EH_PER_BOHR
low_spin_projected_gradient_Eh_per_Bohr
high_spin_projected_gradient_Eh_per_Bohr
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
↓
H_mecp_main_other_mw
H_mecp_other_main_mw
↓
H_eff_mw_selected
↓
P_TR
P_RT
P_RC
P_TOTAL
↓
reaction_direction_hessian_mw
reaction_direction_hessian_cart
reduced_mass_amu
↓
H_eff_projected_transverse
↓
freq_MECP_effhess_real_cm1
freq_MECP_effhess_imag_cm1

```

Therefore, Step 6 transforms the two MECP gradients and the two surface Hessians into the authoritative MECP gradient mean, effective-Hessian reaction coordinate, reduced mass, and transverse crossing-point frequencies:

$$\{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}, \mathbf{H}_1, \mathbf{H}_2\} \rightarrow \{\Delta F_{\parallel}, \text{GRADMEAN}, \mathbf{k}_{\text{RC}}, \mu_{\text{RC}}, \{v_{X,i}^{\text{eff}}\}\}$$

These quantities are subsequently used to finalize the zero-point-corrected MECP barrier, calculate the energy-dependent crossing velocity, evaluate Landau-Zener and weak coupling probabilities, and construct the MECP density and sum of states.

4.7 Step 7: Reaction Coordinate Energy and Velocity Grid

Step 7 uses the effective-Hessian frequencies and reduced mass obtained in Step 6 to finalize the zero-point-energy-corrected MECP barrier and construct the common microcanonical energy and velocity grids used in the subsequent probability and rate calculations.

No additional electronic-structure calculation is performed in this step. The calculations are numerical and use:

- the electronic MECP barrier from Step 5;
- the reference-state vibrational frequencies from Step 5;
- the projected effective-Hessian MECP frequencies from Step 6;
- the reaction-coordinate reduced mass from Step 6;
- GRADIENT_GAP_NORM_EH_PER_BOHR; and
- PROJECTED_GRADIENT_MEAN_EH_PER_BOHR.

The principal operations performed in Step 7 are:

1. calculation of the effective-Hessian MECP zero-point energy;
2. calculation of the MECP–reference ZPE difference;
3. replacement of the provisional electronic barrier with the active ZPE-corrected MECP barrier;
4. user definition of the maximum reaction-coordinate excess energy;
5. construction of a common 1 cm^{-1} total-energy grid;
6. identification of the MECP threshold bin;
7. calculation of the energy available along the reaction coordinate;
8. conversion of the reaction-coordinate excess energy to SI units;
9. evaluation of the corresponding crossing velocity;
10. generation of the velocity plot;
11. storage of the complete energy and velocity arrays; and
12. export of the common grid for the probability and rate calculations.

4.7.1 Import of quantities from the preceding steps

Step 7 verifies that the principal energetic, gradient, frequency, and reduced-mass quantities are available:

```
required_vars_step7 = [  
    "jobname",  
    "VaG_MECP_electronic_cm1",  
    "reduced_mass_amu",  
    "GRADIENT_GAP_NORM_EH_PER_BOHR",  
    "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR",  
    "freq_MECP_effhess_real_cm1",  
    "local_base",  
    "remote_base"  
]
```

The most important inputs are:

```
VaG_MECP_electronic_cm1  
freq_MECP_effhess_real_cm1  
ZPE_REF_from_freq_cm1  
reduced_mass_amu
```

where:

$$V_{\text{MECP}}^{\text{el}}$$

is the electronic barrier relative to the selected reference minimum,

$$\{\nu_{X,i}^{\text{eff}}\}$$

are the real transverse frequencies generated from the projected effective Hessian,

$$\text{ZPE}_R$$

is the reference zero-point energy, and

$$\mu_{\text{RC}}$$

is the reaction-coordinate reduced mass.

4.7.2 Recovery of the reference zero-point energy

The preferred reference ZPE variable is:

```
ZPE_REF_from_freq_cm1
```

which was calculated in Step 5 directly from the positive reference-state frequencies.

If this variable is unavailable, Step 7 reconstructs it from one of the available Hartree values:

```
if "ZPE_REF_thermo_hartree" in globals():
    ZPE_REF_from_freq_cm1 = (
        float(ZPE_REF_thermo_hartree)
        * hartree_to_cm
    )
elif "ZPE_REF_hartree" in globals():
    ZPE_REF_from_freq_cm1 = (
        float(ZPE_REF_hartree)
        * hartree_to_cm
    )
```

Therefore,

$$\text{ZPE}_R(\text{cm}^{-1}) = \text{ZPE}_R(E_h) \times 219474.6313705.$$

This fallback ensures that the ZPE correction can still be evaluated if the frequency-derived reference variable is absent but the thermochemical ZPE is available.

4.7.3 Validation of the effective-Hessian MECP frequencies

The Step 6 frequencies are converted to a one-dimensional numerical array:

```
freq_MECP_effhess_real_cm1 = np.asarray(
    freq_MECP_effhess_real_cm1,
    dtype=float
).reshape(-1)
```

Only finite, positive values are retained:

```
freq_MECP_effhess_real_cm1 = (
    freq_MECP_effhess_real_cm1[
        np.isfinite(freq_MECP_effhess_real_cm1)
        & (freq_MECP_effhess_real_cm1 > 0.0)
    ]
)
```

Thus, translational, rotational, reaction-coordinate, imaginary, and invalid modes do not contribute to the MECP ZPE.

The code terminates if no usable positive frequencies remain:

```
if freq_MECP_effhess_real_cm1.size == 0:
    raise RuntimeError(...)
```

This safeguard prevents construction of the final barrier from an invalid effective-Hessian analysis.

4.7.4 Effective-Hessian MECP zero-point energy

The crossing-point ZPE is calculated from the positive transverse effective-Hessian frequencies:

```
ZPE_MECP_effhess_cm1 = (  
    0.5  
    * float(  
        np.sum(freq_MECP_effhess_real_cm1)  
    )  
)
```

In mathematical form,

$$\text{ZPE}_X^{\text{eff}} = \frac{1}{2} \sum_i \nu_{X,i}^{\text{eff}}.$$

Only the real transverse modes are included.

This quantity differs from the raw ORCA PES2 ZPE because the frequencies originate from the projected effective Hessian rather than from one uncombined spin surface.

4.7.5 MECP–reference zero-point-energy difference

The ZPE correction relative to the reference minimum is:

```
Delta_ZPE_MECP_minus_REF_cm1 = (  
    ZPE_MECP_effhess_cm1  
    - float(ZPE_REF_from_freq_cm1)  
)
```

or

$$\Delta\text{ZPE}_{X-R} = \text{ZPE}_X^{\text{eff}} - \text{ZPE}_R.$$

A positive value raises the active barrier relative to the electronic barrier, whereas a negative value lowers it.

The variable is stored as:

```
Delta_ZPE_MECP_minus_REF_cm1
```

for later reporting and validation.

4.7.6 Final ZPE-corrected MECP barrier

The active MECP barrier is calculated as:

```
VaG_MECP_ZPE_corrected_cm1 = (
    float(VaG_MECP_electronic_cm1)
    + Delta_ZPE_MECP_minus_REF_cm1
)
```

corresponding to

$$V_{\text{MECP}}^{\text{ZPE}} = V_{\text{MECP}}^{\text{el}} + \text{ZPE}_X^{\text{eff}} - \text{ZPE}_R.$$

The result replaces the provisional electronic barrier:

```
VaG_MECP_cm1 = float(
    VaG_MECP_ZPE_corrected_cm1
)
```

The active barrier is therefore

$$V_{\text{MECP}} = V_{\text{MECP}}^{\text{el}} + \Delta\text{ZPE}_{X-R}$$

The corresponding value in kJ mol^{-1} is:

```
VaG_MECP_kJmol = (
    VaG_MECP_cm1 * 0.01196266
)
```

and the general MECP-energy aliases are updated:

```
E_MECP = VaG_MECP_cm1
E_MECP_cm1 = VaG_MECP_cm1
```

From this point onward, VaG_MECP_cm1 refers to the ZPE-corrected barrier rather than the electronic barrier.

4.7.7 Barrier-source labeling

The definition of the active barrier is recorded explicitly in:

```
VaG_MECP_barrier_label
```

as:

```
ZPE-corrected MECP barrier = electronic barrier
+ ZPE_effhess(MECP) - ZPE(reference)
```

This label is propagated to the final reports so that the energetic origin of the threshold is unambiguous.

The electronic and ZPE-corrected barriers remain available separately:

```
VaG_MECP_electronic_cm1
VaG_MECP_ZPE_corrected_cm1
```

```
VaG_MECP_cm1
```

This separation allows the contribution of the ZPE correction to be examined directly.

4.7.8 User definition of the maximum excess energy

The user is asked to specify the highest reaction-coordinate energy above the ZPE-corrected MECP barrier:

```
epsilon_rc_max_cm1 = ask_float(  
    "Maximum reaction-coordinate energy "  
    "above MECP, cm^-1, e.g. 5000: ",  
    minimum=1.0e-12  
)
```

This value is not the absolute maximum total energy. It is the maximum excess energy relative to the MECP:

$$\epsilon_{\text{RC,max}} = E_{\text{max}} - E_{\text{MECP}}.$$

The absolute requested maximum energy is then:

```
E_max_requested_cm1 = (  
    VaG_MECP_cm1  
    + epsilon_rc_max_cm1  
)
```

or

$$E_{\text{max}}^{\text{requested}} = V_{\text{MECP}} + \epsilon_{\text{RC,max}}.$$

4.7.9 Definition of the common energy spacing

The microcanonical energy spacing is fixed as:

```
energy_step_cm1 = 1.0
```

so that

$$\Delta E = 1 \text{ cm}^{-1}.$$

This is the common energy resolution used by the subsequent probability, density-of-states, sum-of-states, microcanonical-rate, and canonical-rate calculations.

Using a common integer-spaced grid ensures consistent indexing and direct alignment with the SPINKS implementation.

4.7.10 Construction of the total-energy grid

The number of the highest required energy bin is calculated as:

```
maximum_energy_bin_index = int(
    np.ceil(
        E_max_requested_cm1 / energy_step_cm1
    )
)
```

The complete total-energy grid is then:

```
total_energy_grid_cm1 = (
    np.arange(maximum_energy_bin_index + 1, dtype=float)
    * energy_step_cm1
)
```

Therefore,

$$E_j = j\Delta E, \quad j = 0, 1, \dots, \text{maxn}_{\text{py}},$$

with

$$\Delta E = 1 \text{ cm}^{-1}.$$

The grid begins at zero energy relative to the selected reference minimum and extends slightly beyond the requested maximum when the requested value does not fall exactly on an integer bin.

The actual maximum grid energy is:

```
E_max_cm1 = float(total_energy_grid_cm1[-1])
```

and satisfies

$$E_{\text{max}} = \left\lceil E_{\text{max}}^{\text{requested}} \right\rceil.$$

4.7.11 Identification of the MECP threshold bin

The MECP bin is defined as:

```
crossing_energy_bin_index = int(
    np.ceil(E_MECP / energy_step_cm1)
)
```

With a 1 cm⁻¹ spacing,

$$\text{binX} = \lceil V_{\text{MECP}} \rceil.$$

Because the ZPE-corrected MECP energy is generally noninteger, the first grid point above the exact MECP energy is the threshold bin.

For example, if

$$V_{\text{MECP}} = 10663.867 \text{ cm}^{-1},$$

then

$$\text{binX} = 10664.$$

This distinction is important because the exact MECP energy itself is not necessarily a member of the integer grid.

4.7.12 Reaction-coordinate excess-energy grid

The energy available above the crossing barrier is calculated as:

```
epsilon_rc_cm1 = (
    total_energy_grid_cm1 - E_MECP
)
```

or

$$\epsilon_{\text{RC}}(E) = E - V_{\text{MECP}}.$$

The resulting array is:

- negative below the MECP;
- zero only if the exact barrier lies on a grid point; and
- positive above the MECP.

For operations requiring nonnegative energy, Step 7 defines:

```
epsilon_rc_positive_cm1 = np.maximum(
```

```
epsilon_rc_cm1,  
0.0  
)
```

or

$$\epsilon_{\text{RC}}^+ = \max(E - V_{\text{MECP}}, 0).$$

The aliases:

```
E_above_MECP_cm1  
dE_cm1
```

are also assigned to the signed excess-energy grid.

4.7.13 Why the velocity is zero at and below the MECP

The classical reaction-coordinate velocity is evaluated only when the energy exceeds the crossing barrier.

For

$$E \leq V_{\text{MECP}},$$

the available reaction-coordinate kinetic energy is nonpositive:

$$\epsilon_{\text{RC}} \leq 0.$$

The implementation therefore assigns

$$v(E) = 0.$$

For

$$E > V_{\text{MECP}},$$

the velocity is

$$v(E) = \sqrt{\frac{2\epsilon_{\text{RC}}}{\mu_{\text{RC}}}}.$$

Consequently, if the exact MECP energy lies between integer grid points, the first nonzero velocity occurs at the first grid bin above the MECP.

4.7.14 Conversion of excess energy to joules

The signed excess energy in wavenumbers is converted to joules using:

```
E_excess_J = (  
    (total_energy_grid_cm1 - E_MECP)  
    * 100.0  
    * h_SI  
    * c_SI  
)
```

The conversion follows

$$\epsilon_{\text{RC}}(\text{J}) = \epsilon_{\text{RC}}(\text{cm}^{-1}) \times 100 \times h \times c.$$

The factor of 100 converts inverse centimeters to inverse meters.

The constants are:

```
h_SI = 6.62607015e-34  
c_SI = 2.99792458e8
```

where h is Planck's constant and c is the speed of light.

4.7.15 Conversion of the reduced mass to kilograms

The reaction-coordinate reduced mass from Step 6 is converted from atomic mass units to kilograms:

```
mu_kg = (  
    reduced_mass_amu  
    * amu_to_kg  
)
```

with:

```
amu_to_kg = 1.66053906660e-27
```

Thus,

$$\mu_{\text{kg}} = \mu_{\text{amu}} \times 1.66053906660 \times 10^{-27}.$$

The code verifies that the converted mass is finite and positive before calculating the velocity.

4.7.16 Reaction-coordinate velocity in meters per second

The velocity array is initialized as zero:

```
v_m_s = np.zeros_like(  
    total_energy_grid_cm1,  
    dtype=float  
)
```

The positive-energy mask is:

```
mask_energy = E_excess_J > 0.0
```

and the velocity is evaluated only for those bins:

```
v_m_s[mask_energy] = np.sqrt(  
    2.0  
    * E_excess_J[mask_energy]  
    / mu_kg  
)
```

Therefore,

$$v(E) = \begin{cases} 0, & E \leq V_{\text{MECP}}, \\ \sqrt{\frac{2(E - V_{\text{MECP}})}{\mu_{\text{RC}}}}, & E > V_{\text{MECP}}. \end{cases}$$

The velocity increases as

$$v \propto \sqrt{E - V_{\text{MECP}}}.$$

It is therefore largest at the highest energy included in the selected grid.

4.7.17 Velocity in bohr per second

The SI velocity is converted to bohr per second using:

```
v_bohr_s = v_m_s / bohr_m
```

with:

```
bohr_m = 5.29177210903e-11
```

Thus,

$$v_{\text{bohr/s}} = \frac{v_{\text{m/s}}}{5.29177210903 \times 10^{-11}}.$$

The `v_bohr_s` array is used directly in the probability equations because the gradient quantities are expressed in atomic units involving bohr.

4.7.18 Numerically safe velocity array

At and below the crossing threshold, the velocity is exactly zero. Some subsequent probability equations contain inverse powers of velocity, so direct division by zero must be prevented.

Step 7 therefore creates:

```
v_bohr_s_safe = np.copy(v_bohr_s)
```

and replaces all values below the floor with an extremely small positive number:

```
velocity_floor = 1.0e-12  
v_bohr_s_safe[  
    v_bohr_s_safe <= velocity_floor  
] = 1.0e-300
```

This array is used only for numerically safe evaluation of expressions involving division by velocity.

The physical velocity remains stored separately in:

```
v_bohr_s
```

and is still exactly zero at and below the MECP.

The probability code subsequently applies the appropriate energy masks so that subthreshold bins remain physically inactive.

4.7.19 Common-grid consistency with later steps

Step 7 exports identical energy and velocity arrays to the subsequent probability and rate calculations:

```
total_energy_grid_cm1  
probability_energy_grid_cm1  
epsilon_rc_cm1  
v_m_s  
v_bohr_s  
v_bohr_s_safe
```

The alias:

```
probability_energy_grid_cm1 = total_energy_grid_cm1.copy()
```

ensures that the Landau-Zener probabilities are evaluated on exactly the same total-energy grid used for:

- weak coupling probabilities;
- intermediate SOC channels;
- M_S -specific SOC channels;
- densities of states;
- sums of states;
- microcanonical rates; and
- canonical integrations.

This prevents interpolation errors or bin misalignment between different parts of the workflow.

4.7.20 Velocity-grid plot

Step 7 generates a plot of velocity against total energy:

```
plt.plot(
    total_energy_grid_cm1,
    v_m_s,
    linewidth=2
)
```

The ZPE-corrected MECP barrier is marked by a vertical dashed line:

```
plt.axvline(
    E_MECP,
    linestyle="--",
    linewidth=1.5,
    label="ZPE-corrected MECP"
)
```

The axes are

x = total energy relative to the reference minimum,

and

y = reaction-coordinate velocity at the MECP.

Below the barrier, the plot remains at zero. Above the barrier, it follows the square-root dependence

$$v(E) \propto \sqrt{E - V_{\text{MECP}}}.$$

The plot is saved as

<jobname>_Step7_velocity_grid.png

at 300 dpi.

4.7.21 Step 7 output directory

The local output directory is:

```
local_step7 = os.path.join(
    local_base,
    "RC_energy_velocity_grid"
)
```

In the cluster implementation, a corresponding remote directory is defined as:

```
remote_step7 = posixpath.join(
    remote_base,
    "RC_energy_velocity_grid"
)
```

If an active SFTP connection is available, the summary, grid, and plot files are uploaded to the remote project directory.

Failure of the optional upload does not invalidate the local Step 7 calculation.

4.7.22 Energy and velocity summary file

The file:

```
<jobname>_Step7_energy_velocity_summary.txt
```

records:

- the electronic MECP barrier;
- the reference ZPE;
- the effective-Hessian MECP ZPE;
- the MECP–reference ZPE difference;
- the final ZPE-corrected barrier;
- the barrier in kJ mol^{−1};

- the barrier-source label;
- energy spacing;
- number of bins;
- MECP bin index;
- requested maximum excess energy;
- actual maximum grid energy;
- reduced mass;
- reduced mass in kilograms;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- PROJECTED_GRADIENT_MEAN_EH_PER_BOHR;
- minimum and maximum velocities; and
- paths to the generated files.

This file provides a compact record of the energy-grid construction.

4.7.23 Complete energy and velocity grid file

The file:

<jobname>_Step7_energy_velocity_grid.txt

contains one row for every 1 cm^{-1} energy bin.

The columns are:

total_energy_grid_cm1
 epsilon_rc_cm1
 epsilon_rc_positive_cm1
 E_excess_J
 v_m_s
 v_bohr_s
 v_bohr_s_safe

For every total energy E_j , the file therefore records:

$$\begin{aligned}
 &E_j, \\
 &E_j - V_{\text{MECP}}, \\
 &\max(E_j - V_{\text{MECP}}, 0),
 \end{aligned}$$

the excess energy in joules, and the corresponding velocities in both SI and atomic-distance units.

This complete tabulation permits independent reproduction of the probability calculations.

4.7.24 Variables exported by step 7

The principal energetic variables exported are:

```
VaG_MECP_electronic_cm1  
ZPE_REF_from_freq_cm1  
ZPE_MECP_effhess_cm1  
Delta_ZPE_MECP_minus_REF_cm1  
VaG_MECP_ZPE_corrected_cm1  
VaG_MECP_cm1  
VaG_MECP_kJmol
```

The principal grid variables are:

```
energy_step_cm1  
maximum_energy_bin_index  
crossing_energy_bin_index  
total_energy_grid_cm1  
probability_energy_grid_cm1  
epsilon_rc_cm1  
epsilon_rc_positive_cm1  
epsilon_rc_max_cm1
```

The principal velocity variables are:

```
mu_kg  
E_excess_J  
v_m_s  
v_bohr_s  
v_bohr_s_safe
```

The validated gradient quantities are retained as:

```
GRADIENT_GAP_NORM_EH_PER_BOHR  
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
```

These arrays and scalar quantities are consumed directly by the probability calculations in Step 8 and the rate calculations in the following steps.

4.7.25 Summary of the step 7 algorithm

The complete sequence implemented in Step 7 can be summarized as follows.

The effective-Hessian MECP ZPE is calculated from the positive transverse frequencies:

$$\text{ZPE}_X^{\text{eff}} = \frac{1}{2} \sum_i v_{X,i}^{\text{eff}}.$$

The ZPE difference relative to the reference minimum is

$$\Delta\text{ZPE}_{X-R} = \text{ZPE}_X^{\text{eff}} - \text{ZPE}_R.$$

The active crossing barrier is then

$$V_{\text{MECP}} = V_{\text{MECP}}^{\text{el}} + \Delta\text{ZPE}_{X-R}.$$

The user supplies a maximum excess energy,

$$\epsilon_{\text{RC,max}},$$

and Step 7 constructs the total-energy grid

$$E_j = j, j = 0, 1, \dots, [V_{\text{MECP}} + \epsilon_{\text{RC,max}}],$$

in units of cm^{-1} .

The reaction-coordinate excess energy is

$$\epsilon_{\text{RC}}(E_j) = E_j - V_{\text{MECP}}.$$

The velocity is evaluated as

$$v(E_j) = \begin{cases} 0, & \epsilon_{\text{RC}} \leq 0, \\ \sqrt{\frac{2\epsilon_{\text{RC}}}{\mu_{\text{RC}}}}, & \epsilon_{\text{RC}} > 0. \end{cases}$$

After unit conversion,

$$\epsilon_{\text{RC}}(\text{J}) = \epsilon_{\text{RC}}(\text{cm}^{-1}) \times 100hc,$$

and

$$\mu_{\text{kg}} = \mu_{\text{amu}} \times 1.66053906660 \times 10^{-27}.$$

The corresponding variable flow is:

```

VaG_MECP_electronic_cm1
freq_MECP_effhess_real_cm1
ZPE_REF_from_freq_cm1
  ↓
ZPE_MECP_effhess_cm1
Delta_ZPE_MECP_minus_REF_cm1
  ↓
VaG_MECP_ZPE_corrected_cm1
VaG_MECP_cm1
  ↓
epsilon_rc_max_cm1
energy_step_cm1 = 1 cm-1
  ↓
total_energy_grid_cm1
crossing_energy_bin_index
epsilon_rc_cm1
  ↓
E_excess_J
reduced_mass_amu
  ↓
v_m_s
v_bohr_s
v_bohr_s_safe
  ↓
Step 8 probabilities
subsequent microcanonical rates

```

Therefore, Step 7 transforms the electronic barrier, effective-Hessian frequencies, reference ZPE, and reduced mass into the final active MECP threshold and a common energy-dependent velocity grid:

$$\{V_{\text{MECP}}^{\text{el}}, ZPE_X^{\text{eff}}, ZPE_R, \mu_{\text{RC}}\} \rightarrow \{V_{\text{MECP}}^{\text{ZPE}}, E_j, \epsilon_{\text{RC}}(E_j), v(E_j)\}$$

These quantities are subsequently used in Step 8 to calculate the effective, intermediate, and M_S -specific Landau-Zener and weak coupling spin-crossing probabilities on the same 1 cm⁻¹ common energy grid.

4.8 Step 8: Landau-Zener and Weak Coupling Probabilities

Step 8 evaluates the energy-dependent spin-crossing probabilities on the common 1 cm^{-1} total-energy grid generated in Step 7. Both the Landau-Zener and weak coupling expressions are evaluated using the same active MECP barrier, reduced mass, gradient quantities, and velocity array.

Three levels of spin-orbit-coupling resolution are considered:

1. Effective probabilities, calculated using the scalar effective SOC, H_{SO} ;
2. Intermediate probabilities, calculated from SOC values grouped according to $|M_S|$ on the lower-spin surface; and
3. Individual M_S -resolved probabilities, calculated from every nonzero element of the spin-orbit-coupling matrix.

No separate energy or velocity grid is created in Step 8. The arrays generated in Step 7 are imported directly and independently verified before probability evaluation. In addition, individual M_S -specific probabilities are not added together in this step. Channels with equal $|H_{SO}|$ are grouped only for plotting because they generate identical curves; the individual channels remain stored separately.

The principal operations performed in Step 8 are:

1. import and verification of the Step 7 energy and velocity grids;
2. validation of the active MECP, gradient, SOC, and reduced-mass quantities;
3. evaluation of the effective Landau-Zener probability;
4. evaluation of the effective weak coupling probability;
5. extraction of individual nonzero SOC matrix elements;
6. calculation of one LZ and WC probability curve for every nonzero M_S -specific channel;
7. grouping of symmetry-equivalent or equal- $|H_{SO}|$ curves for plotting only;
8. construction of intermediate SOC quantities grouped by $|M_S|$;
9. calculation of intermediate LZ and WC probability curves;
10. generation of probability plots;
11. storage of complete numerical probability arrays; and
12. export of Step 9-compatible probability variables.

4.8.1 Import of active quantities

Step 8 verifies the availability of the effective SOC, MECP threshold, gradient quantities, reduced mass, energy grid, and velocity arrays:

```
required_vars_step8 = [  
    "jobname",  
    "H_SO_cm",  
    "GRADIENT_GAP_NORM_EH_PER_BOHR",  
    "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR",  
    "VaG_MECP_cm1",  
    "total_energy_grid_cm1",  
    "v_m_s",  
    "v_bohr_s",  
    "energy_step_cm1",  
    "maximum_energy_bin_index",  
    "crossing_energy_bin_index",  
    "reduced_mass_amu",  
    ...  
]
```

The principal physical inputs are:

```
H_SO_cm  
GRADIENT_GAP_NORM_EH_PER_BOHR  
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR  
VaG_MECP_cm1  
reduced_mass_amu
```

where

$$H_{\text{SO}}$$

is the effective spin-orbit coupling obtained in Step 2,

$$\Delta F_{\parallel} = \| \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \|$$

is the gradient-difference magnitude,

$$\bar{g} = \text{GRADMEAN}$$

is the projected gradient mean obtained in Step 6,

$$V_{\text{MECP}}$$

is the ZPE-corrected crossing barrier from Step 7, and

$$\mu$$

is the selected reaction-coordinate reduced mass.

4.8.2 Reuse of the common Step 7 grid

The total-energy grid is imported and converted to a one-dimensional array:

```
total_energy_grid_cm1 = np.asarray(
    total_energy_grid_cm1,
    dtype=float
).reshape(-1)
```

The probability-grid alias is defined as:

```
probability_energy_grid_cm1 = total_energy_grid_cm1.copy()
```

Therefore,

$$E_{\text{LZ},j} = E_j,$$

and no separate Landau-Zener grid is constructed.

The Step 7 velocity arrays are imported as:

```
v_m_s_step7
v_bohr_s_step7
```

and later copied into the authoritative Step 8 arrays:

```
v_m_s
v_bohr_s.
```

Consequently, every effective, intermediate, and M_S -specific probability is evaluated at the same total energies and velocities used in the subsequent microcanonical-rate calculation.

4.8.3 Validation of the energy-grid dimensions

Step 8 checks that the total-energy array contains exactly

$$\text{maxn}_{\text{py}} + 1$$

points:

```
if total_energy_grid_cm1.size != maximum_energy_bin_index + 1:
    raise RuntimeError(...)
```

It also verifies that both imported velocity arrays have the same length as the energy grid:

```
if v_m_s_step7.size != total_energy_grid_cm1.size:
    raise RuntimeError(...)

if v_bohr_s_step7.size != total_energy_grid_cm1.size:
    raise RuntimeError(...)
```

These checks prevent probabilities from being paired with velocities belonging to different energy bins.

4.8.4 Verification of the uniform common energy grid

The expected grid is reconstructed as:

```
expected_grid = (
    np.arange(maximum_energy_bin_index + 1, dtype=float)
    * energy_step_cm1
)
```

and compared with the imported array:

```
if not np.allclose(
    total_energy_grid_cm1,
    expected_grid,
    rtol=0.0,
    atol=1.0e-12
):
    raise RuntimeError(...)
```

Thus, Step 8 requires

$$E_j = j\Delta E,$$

with a uniform spacing

$$\Delta E = E_{\text{step}}.$$

For the default common grid,

$$\Delta E = 1 \text{ cm}^{-1}.$$

4.8.5 Verification of the MECP threshold bin

The expected MECP bin is recalculated as:

```
expected_crossing_energy_bin_index = int(
    np.ceil(E_MECP_cm1 / energy_step_cm1)
)
```

and compared with the value imported from Step 7:

```
if crossing_energy_bin_index != expected_crossing_energy_bin_index:
    raise RuntimeError(...)
```

Therefore,

$$\text{binX} = \left\lceil \frac{V_{\text{MECP}}}{\Delta E} \right\rceil.$$

This ensures that Step 8 and Step 7 use the same crossing threshold and the same first integer energy bin above the MECP.

4.8.6 Active gradient quantities

The two active gradient quantities are converted to positive scalar magnitudes:

```
gradient_gap_magnitude = abs(
    float(GRADIENT_GAP_NORM_EH_PER_BOHR)
)
gradmean = abs(
    float(PROJECTED_GRADIENT_MEAN_EH_PER_BOHR)
)
```

The code verifies that both are finite and positive.

The active Landau-Zener gradient quantity is

$$\Delta F_{\parallel} = \| \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \parallel.$$

The active weak coupling gradient quantity is the projected value generated in Step 6:

$$\tilde{g} = \sqrt{F_{\text{LS},\parallel} F_{\text{HS},\parallel}}.$$

Step 8 therefore uses the authoritative projected PROJECTED_GRADIENT_MEAN_EH_PER_BOHR, not the Cartesian diagnostic calculated in Step 4.

4.8.7 Reduced-mass conversions

The reduced mass is converted to both kilograms and atomic units:

```

mu_kg = (
    float(reduced_mass_amu)
    * amu_to_kg
)

mu_au = (
    float(reduced_mass_amu)
    * amu_to_au
)

```

with:

```

amu_to_kg = 1.66053906660e-27
amu_to_au = 1822.8884853323708

```

Thus,

$$\mu_{\text{kg}} = \mu_{\text{amu}} \times 1.66053906660 \times 10^{-27},$$

and

$$\mu_{\text{au}} = \mu_{\text{amu}} \times 1822.8884853323708.$$

The SI mass is used for the independent velocity check, whereas the atomic-unit mass is used in the weak coupling probability.

4.8.8 Independent velocity consistency check

Although Step 8 uses the Step 7 velocity arrays directly, it independently reconstructs the velocity according to the Step 9 definition.

The excess energy is recalculated as:

```

E_excess_J_check = (
    (total_energy_grid_cm1 - E_MECP_cm1)
    * 100.0
    * h_SI
    * c_SI
)

```

and:

```

v_m_s_check[mask_velocity_check] = np.sqrt(
    2.0
    * E_excess_J_check[mask_velocity_check]
    / mu_kg
)

```

Therefore,

$$v(E) = \begin{cases} 0, & E \leq V_{\text{MECP}}, \\ \sqrt{\frac{2(E - V_{\text{MECP}})}{\mu}}, & E > V_{\text{MECP}}. \end{cases}$$

The reconstructed arrays are compared with the imported Step 7 arrays using strict numerical tolerances.

Step 8 terminates if either:

`v_m_s_step7`

or:

`v_bohr_s_step7`

does not agree with the independently reconstructed velocity.

This guarantees that the probability and rate sections use an identical velocity definition.

4.8.9 Effective spin-orbit coupling

The effective SOC used for the principal probability curves is:

`H_SO_cm`

This scalar may originate from:

- the complete M_S -resolved SOC matrix;
- a scaled SOC matrix;
- an effective ORCA SOC value; or
- a user-supplied replacement.

The probability functions use the absolute SOC magnitude:

`H_cm = float(abs(H_cm))`

because both implemented probability expressions depend on H_{SO}^2 . The complex phase or sign of a matrix element therefore does not affect its individual probability curve.

4.8.10 Conversion of the SOC to joules

For the Landau-Zener calculation, an SOC supplied in cm^{-1} is converted to joules:

`H_J = (`

```

h_SI
* c_SI
* H_cm
* 100.0
)

```

Thus,

$$H_{\text{SO}}(\text{J}) = hcH_{\text{SO}}(\text{cm}^{-1}) \times 100.$$

The factor of 100 converts inverse centimeters to inverse meters.

4.8.11 Energy mask used for the LZ probability

The Landau-Zener probability is evaluated only for bins satisfying:

```

mask = (
    np.isfinite(E_grid_cm1)
    & np.isfinite(v_bohr_s_grid)
    & (E_grid_cm1 >= E_MECP_cm1)
    & (v_bohr_s_grid > velocity_floor)
)

```

Therefore, the active LZ bins must satisfy:

$$E \geq V_{\text{MECP}}$$

and

$$v > v_{\text{floor}}.$$

Because the physical velocity is zero at the exact MECP threshold, the first nonzero LZ probability generally occurs at the first grid point strictly above the MECP.

All subthreshold and zero-velocity bins remain assigned

$$P_{\text{LZ}} = 0.$$

4.8.12 Time derivative of the diabatic energy difference

The rate of change of the surface-energy difference is calculated in Hartree per second as:

```

dDeltaE_dt_Eh_s = (
    gradient_gap_magnitude

```

```
) * v_bohr_s_grid
```

or

$$\left| \frac{d\Delta E}{dt} \right| = \Delta F_{\parallel} v.$$

Because

$$\Delta F_{\parallel}$$

is expressed in Hartree per bohr and

$$v$$

is expressed in bohr per second, their product has units of Hartree per second.

This value is converted to joules per second using:

```
dDeltaE_dt_J_s = (
    dDeltaE_dt_Eh_s
    * Eh_to_J
)
```

with

$$1 E_h = 4.3597447222071 \times 10^{-18} \text{ J}.$$

4.8.13 Implemented Landau-Zener parameter

The code evaluates:

```
gamma[mask] = (
    4.0
    * np.pi**2
    * H_J**2
    / (
        h_SI
        * dDeltaE_dt_J_s[mask]
    )
)
```

Therefore, the implemented energy-dependent parameter is

$$\gamma(E) = \frac{4\pi^2 H_{SO}^2}{h \Delta F_{\parallel} v(E)}.$$

Because the velocity appears in the denominator,

$$\gamma(E) \propto \frac{1}{v(E)}.$$

Accordingly, the LZ parameter is largest immediately above the crossing threshold and decreases as the excess energy and velocity increase.

4.8.14 Implemented Landau-Zener probability

The probability is evaluated as:

```
P[mask] = (  
    1.0  
    - np.exp(  
        -2.0 * gamma[mask]  
    )  
)
```

or

$$P_{\text{LZ}}(E) = 1 - \exp[-2\gamma(E)].$$

Substituting the implemented γ gives

$$P_{\text{LZ}}(E) = 1 - \exp\left[-\frac{8\pi^2 H_{\text{SO}}^2}{\hbar \Delta F_{\parallel} v(E)}\right].$$

The numerical result is cleaned and restricted to the physically permitted range:

```
P[~np.isfinite(P)] = 0.0  
P = np.clip(P, 0.0, 1.0)
```

Thus,

$$0 \leq P_{\text{LZ}} \leq 1.$$

The Landau-Zener expression implemented in SPINKS corresponds to a **double passage through the crossing region**. For a single passage, the transition probability is defined as

$$p_{\text{LZ}}(E) = 1 - \exp[-\gamma(E)],$$

where the code evaluates

$$\gamma(E) = \frac{4\pi^2 H_{\text{SO}}^2}{h \Delta F_{\parallel} v(E)}.$$

During a double passage, the system has two independent opportunities to change spin. The total probability is therefore the probability of crossing during the first passage plus the probability of remaining on the original surface during the first passage and crossing during the second passage:

$$P_{\text{LZ}}(E) = p_{\text{LZ}}(E) + [1 - p_{\text{LZ}}(E)]p_{\text{LZ}}(E).$$

This gives

$$P_{\text{LZ}}(E) = 2p_{\text{LZ}}(E) - p_{\text{LZ}}^2(E).$$

Substituting

$$p_{\text{LZ}}(E) = 1 - \exp[-\gamma(E)]$$

into the preceding relation gives

$$\begin{aligned} P_{\text{LZ}}(E) &= 2(1 - e^{-\gamma}) - (1 - e^{-\gamma})^2 \\ &= 1 - e^{-2\gamma(E)}. \end{aligned}$$

Accordingly, the Python implementation

`P[mask] = 1.0 - np.exp(-2.0 * gamma[mask])`

is not a single-passage expression. The factor of 2.0 multiplying gamma implements the double-passage probability

$$\boxed{P_{\text{LZ}}(E) = 1 - \exp[-2\gamma(E)] = 2p_{\text{LZ}}(E) - p_{\text{LZ}}^2(E)}$$

used in the downstream statistical treatment.

This form assumes that the two passages are treated as statistically independent and that coherent phase interference between the passages is not included. In other words, SPINKS applies the incoherent double-passage probability rather than a Stückelberg-interference expression.

4.8.15 Energy dependence of the LZ probability

Since

$$v(E) = \sqrt{\frac{2(E - V_{\text{MECP}})}{\mu}},$$

the implemented LZ exponent varies as

$$\gamma(E) \propto \frac{1}{\sqrt{E - V_{\text{MECP}}}}.$$

Consequently:

- the probability is zero below and at the threshold because the code excludes zero-velocity bins;
- it reaches its largest evaluated value at the first positive-velocity grid point above the MECP; and
- it generally decreases as the total energy increases.

This behavior differs from a probability that would grow monotonically with excess energy.

4.8.16 Conversion of SOC and excess energy to atomic units

For the weak coupling expression, the SOC is converted from cm^{-1} to Hartree:

`H_Eh = float(abs(H_cm)) / autocrm`

or

$$H_{\text{SO}}(E_h) = \frac{H_{\text{SO}}(\text{cm}^{-1})}{219474.6313705}.$$

The energy relative to the MECP is similarly converted:

`E_excess_Eh = (
 E_grid_cm1 - E_MECP_cm1
) / autocrm`

or

$$\epsilon(E_h) = \frac{E - V_{\text{MECP}}}{219474.6313705}.$$

Unlike the LZ calculation, the WC function does not apply an explicit above-threshold mask. The Airy-function expression is evaluated over the complete finite grid, including energies below the MECP.

4.8.17 Weak coupling prefactor

The energy-independent prefactor is calculated as:

```
prefactor = (
    4.0
    * np.pi**2
    * H_Eh**2
    * (
        2.0 * mu_au
        / (gradmean * gradient_gap_magnitude)
    ) ** (2.0 / 3.0)
)
```

Therefore,

$$A_{\text{WC}} = 4\pi^2 H_{\text{SO}}^2 \left(\frac{2\mu}{\bar{g} \Delta F_{\parallel}} \right)^{2/3}.$$

This term depends on:

- the square of the SOC;
- the reaction-coordinate reduced mass;
- the projected gradient mean; and
- the gradient-difference magnitude.

For a fixed molecular system,

$$A_{\text{WC}} \propto H_{\text{SO}}^2.$$

4.8.18 Weak coupling Airy scale

The Airy scaling factor is calculated as:

```
scale = (
    2.0
    * mu_au
    * gradient_gap_magnitude**2
    / gradmean**4
) ** (1.0 / 3.0)
```

or

$$\alpha = \left(\frac{2\mu\Delta F_{\parallel}^2}{\bar{g}^4} \right)^{1/3}.$$

The Airy argument is then:

```
airy_arg[mask] = (
    -E_excess_Eh[mask]
    * scale
)
```

corresponding to

$$x(E) = -\epsilon(E)\alpha.$$

At the exact MECP energy,

$$\epsilon = 0$$

and therefore

$$x = 0.$$

Below the MECP,

$$\epsilon < 0 \Rightarrow x > 0,$$

whereas above the MECP,

$$\epsilon > 0 \Rightarrow x < 0.$$

4.8.19 Evaluation of the Airy function

The Airy function of the first kind is evaluated using:

```
Ai[mask] = airy(
    airy_arg[mask]
)[0]
```

which gives

$$\text{Ai}[x(E)].$$

The implemented weak coupling probability is:

```
P[mask] = (
    prefactor
    * Ai[mask] ** 2
)
```

or

$$P_{\text{WC}}(E) = 4\pi^2 H_{\text{SO}}^2 \left(\frac{2\mu}{\bar{g} \Delta F_{\parallel} \hbar^2} \right)^{2/3} \text{Ai}^2 \left[-\epsilon(E) \left(\frac{2\mu \Delta F_{\parallel}^2}{\bar{g}^4 \hbar^2} \right)^{1/3} \right].$$

The result is cleaned and clipped:

```
P[~np.isfinite(P)] = 0.0
P = np.clip(P, 0.0, 1.0)
```

so that

$$0 \leq P_{\text{WC}} \leq 1.$$

4.8.20 Subthreshold WC probabilities

The weak coupling expression is evaluated at energies below the ZPE-corrected MECP because the Airy-function treatment remains defined for positive Airy arguments.

Thus, Step 8 can produce small nonzero WC probabilities for

$$E < V_{\text{MECP}}.$$

This is intentional in the implemented formula and differs from the LZ calculation, which is explicitly masked to zero below the crossing threshold.

The later rate calculation determines how these probability values enter the effective number and density-of-states treatment.

4.8.21 Effective Landau-Zener probability

The effective LZ curve is calculated using the scalar H_SO_cm:

```
P_LZ_effective, landau_zener_exponent_effective, v_bohr_s, v_m_s = (  
    compute_landau_zener_probability_for_coupling(H_SO_cm)  
)
```

The resulting arrays are:

```
P_LZ_effective  
landau_zener_exponent_effective
```

and the backward-compatible aliases are:

```
P_LZ_effective  
gamma_LZ.
```

Thus,

$$P_{\text{LZ}}^{\text{eff}}(E) = P_{\text{LZ}}[H_{\text{SO}}^{\text{eff}}, E].$$

4.8.22 Effective weak coupling probability

The effective WC curve is calculated as:

```
P_WC_effective, airy_arg_WC_effective, Ai_WC_effective = (  
    compute_WC_for_HSO_cm(H_SO_cm)  
)
```

The resulting quantities are:

```
P_WC_effective  
airy_arg_WC_effective  
Ai_WC_effective
```

with the alias:

```
P_WC_effective = P_WC_effective.
```

Thus,

$$P_{\text{WC}}^{\text{eff}}(E) = P_{\text{WC}}[H_{\text{SO}}^{\text{eff}}, E].$$

The Airy argument and Airy-function values are retained so that the numerical WC calculation can be reproduced independently.

4.8.23 Selection of the SOC matrix

Step 8 first checks whether only a scalar effective SOC is available:

```
SOC_EFFECTIVE_ONLY  
SOC_MATRIX_CHANNELS_AVAILABLE
```

If the calculation is in effective-only mode, no channel-specific or intermediate curves are generated.

Otherwise, the preferred SOC matrix is:

```
SOC_Ms_matrix_scaled_cm1
```

when available. If no scaled matrix is present, Step 8 uses:

```
SOC_Ms_matrix_cm1.
```

The selected matrix is stored as:

```
SOC_channel_matrix_cm1
```

and its source is recorded in:

```
SOC_channel_matrix_label.
```

The scaled matrix therefore takes precedence when the effective SOC was modified or rescaled consistently after Step 2.

4.8.24 Validation of the M_S dimensions

The low- and high-spin magnetic quantum numbers are imported as:

```
Ms_low  
Ms_high
```

and the corresponding total-spin values as:

```
S_low  
S_high.
```

The matrix shape must satisfy:

```
SOC_channel_matrix_cm1.shape == (  
    len(Ms_low),  
    len(Ms_high)  
)
```

or

$$\dim(\mathbf{H}_{\text{SO}}) = (2S_{\text{low}} + 1) \times (2S_{\text{high}} + 1).$$

Step 8 terminates if the matrix dimensions do not match the imported M_S arrays.

4.8.25 Individual SOC channel definition

For every matrix element:

```
H_ij = SOC_channel_matrix_cm1[i, j]  
H_abs = float(abs(H_ij))
```

the corresponding transition is

$$|S_{\text{low}}, M_{S,\text{low}}\rangle \rightarrow |S_{\text{high}}, M_{S,\text{high}}\rangle.$$

Elements satisfying

$$|H_{ij}| \leq 10^{-10} \text{ cm}^{-1}$$

are ignored.

Each nonzero channel is assigned a unique label containing:

- S_{low} ;
- $M_{S,\text{low}}$;
- S_{high} ; and
- $M_{S,\text{high}}$.

For example:

```
S0.5_Ms-0.5__S1.5_Ms+0.5
```

identifies a transition from

$$S = \frac{1}{2}, M_S = -\frac{1}{2}$$

to

$$S = \frac{3}{2}, M_S = +\frac{1}{2}.$$

4.8.26 Individual channel probability curves

For each nonzero matrix element, Step 8 calculates:

```
P_lz_ij, gamma_lz_ij, _ _ = (  
    compute_landau_zener_probability_for_coupling(H_abs)  
)
```

and:

```
P_wc_ij, airy_arg_wc_ij, Ai_wc_ij = (  
    compute_WC_for_HSO_cm(H_abs)  
)
```

Thus,

$$P_{\text{LZ}}^{M_S, M'_S}(E) = P_{\text{LZ}} \left[| H_{M_S, M'_S} | E \right],$$

and

$$P_{\text{WC}}^{M_S, M'_S}(E) = P_{\text{WC}} \left[| H_{M_S, M'_S} | E \right].$$

The complete channel metadata are stored in:

```
P_LZ_channels  
P_WC_channels.
```

For each channel, the dictionaries retain:

- the probability array;
- the LZ parameter or Airy quantities;
- the complex SOC element;
- its absolute magnitude;
- both spin quantum numbers;
- both M_S values;
- the matrix indices; and
- the source SOC matrix.

4.8.27 No sum over M_S -specific channels

Step 8 does not calculate

$$\sum_{M_S, M'_S} P^{M_S, M'_S}(E)$$

and does not multiply an individual probability curve by the number of symmetry-equivalent channels.

Every nonzero matrix element remains an independent probability channel in:

[P_LZ_channels](#)
[P_WC_channels](#).

Any later averaging, weighting, or statistical combination must therefore be performed explicitly in the subsequent rate treatment rather than being hidden in Step 8.

4.8.28 Dependence of individual curves on $|H_{ij}|$

Both probability expressions depend on the square of the SOC magnitude:

$$P_{LZ} \sim f(|H_{ij}|^2),$$

and

$$P_{WC} \sim |H_{ij}|^2 \text{Ai}^2(\dots).$$

Therefore, channels having equal values of

$$|H_{ij}|$$

produce numerically identical probability arrays, even if the underlying complex matrix elements differ in sign or phase.

This observation is used only to reduce visual duplication in the plots.

4.8.29 Grouping equal- $|H_{SO}|$ channels for plotting

The helper function:

[find_matching_hso_group\(...\)](#)

compares the absolute SOC magnitudes using the tolerances:

[MS_CHANNEL_HSO_GROUP_ATOL_CM1](#)

with defaults

$$10^{-8} \text{ cm}^{-1}$$

and

$$10^{-10},$$

respectively.

Channels satisfying

$$|H_i| \approx |H_j|$$

are assigned to one plotting group.

The group stores one representative probability array:

P_representative

but also retains all individual channel labels and metadata.

The recorded grouping rule is

equal |H_SO|; representative only; no sum

4.8.30 Interpretation of grouped channel labels

When multiple channels share one plotted curve, their labels are separated with commas:

$$P^{M_{S,1}, M'_{S,1}}, P^{M_{S,2}, M'_{S,2}}, \dots$$

They are not separated by plus signs.

This notation indicates that the displayed line represents several identical individual curves, not a sum such as

$$P_1 + P_2.$$

The group multiplicity stored in the output is therefore a count of equivalent curves, not a probability weighting factor.

4.8.31 Grouping by $|M_{S,\text{low}}|$

The intermediate treatment groups rows of the SOC matrix according to the absolute lower-spin magnetic quantum number:

```
abs_ms_groups = sorted(
    set([
        round(abs(float(x)), 8)
        for x in Ms_low
    ])
)
```

Thus, the rows corresponding to

$$M_S = +\frac{1}{2}$$

and

$$M_S = -\frac{1}{2}$$

belong to the same intermediate group

$$|M_S| = \frac{1}{2}.$$

Similarly, rows with $M_S = \pm 3/2$ are grouped under

$$|M_S| = \frac{3}{2}.$$

4.8.32 Row-resolved intermediate SOC magnitude

For each lower-spin row i , the SOC magnitude summed over all high-spin M'_S components is calculated as:

```
H_row = float(
    np.sqrt(
        np.sum(np.abs(row) ** 2)
    )
)
```

or

$$H_i^{\text{row}} = \sqrt{\sum_{M'_S} |H_{M_S, i, M'_S}|^2}.$$

This is a root-sum-square SOC over all final M'_S components for one fixed initial M_S .

Only finite, nonzero row magnitudes are retained.

4.8.33 Intermediate SOC for a $|M_S|$ group

If a $|M_S|$ group contains multiple symmetry-related rows, their row magnitudes are combined using:

```
H_int = float(
    np.sqrt(
        np.mean(H_rows ** 2)
    )
)
```

Therefore,

$$H_{\text{int}}(|M_S|) = \sqrt{\frac{1}{n} \sum_{i=1}^n (H_i^{\text{row}})^2}.$$

Since

$$(H_i^{\text{row}})^2 = \sum_{M'_S} |H_{M_S, i, M'_S}|^2,$$

the implemented intermediate SOC is

$$H_{\text{int}}(|M_S|) = \sqrt{\frac{1}{n} \sum_{i \in |M_S|} \sum_{M'_S} |H_{M_S, i, M'_S}|^2}.$$

The value n is stored as the group degeneracy:

```
degeneracy = len(row_indices)
```

4.8.34 Intermediate probability curves

For every nonzero intermediate SOC value, Step 8 calculates:

```
P_lz_int = compute_landau_zener_probability_for_coupling(H_int)
P_wc_int = compute_WC_for_HSO_cm(H_int)
```

giving

$$P_{\text{LZ}}^{|M_S|}(E) = P_{\text{LZ}}[H_{\text{int}}(|M_S|), E],$$

and

$$P_{\text{WC}}^{|M_S|}(E) = P_{\text{WC}}[H_{\text{int}}(|M_S|), E].$$

The results are stored in:

```
P_LZ_intermediate
P_WC_intermediate.
```

Each record retains:

- the probability array;
- the LZ parameter or Airy quantities;
- the intermediate SOC;
- the contributing row SOC magnitudes;
- the absolute lower-spin M_S ;
- the degeneracy;
- the row indices; and
- the source matrix.

4.8.35 No weighted sum of intermediate curves

Step 8 calculates one intermediate curve for each unique value of

$$|M_{S,\text{low}}|.$$

It does not calculate a degeneracy-weighted sum such as

$$\sum_{|M_S|} g_{|M_S|} P^{|M_S|}(E).$$

The degeneracy is stored as metadata, but it is not applied to the probability arrays in Step 8.

This preserves a clear distinction among:

- individual channel probabilities;
- intermediate $|M_S|$ -resolved probabilities; and
- the effective scalar-SOC probability.

4.8.36 Plotting ranges

The LZ plot begins no more than 1000 cm^{-1} below the MECP:

```
lz_plot_min_cm1 = max(  
    total_energy_grid_cm1[0],  
    E_MECP_cm1 - 1000.0  
)
```

The WC plot begins no more than 1500 cm^{-1} below the MECP:

```
wc_plot_min_cm1 = max(  
    total_energy_grid_cm1[0],  
    E_MECP_cm1 - 1500.0  
)
```

The wider subthreshold range for WC allows the Airy-function probability below the MECP to be displayed.

The plots use the actual Step 7 arrays. No interpolated plotting grid is introduced.

4.8.37 Landau–Zener probability plot

The LZ plot contains:

- the effective probability as a black dotted curve;
- intermediate curves as dashed lines;
- one representative line for each unique individual-channel $|H_{S0}|$; and
- a vertical line at the ZPE-corrected MECP.

The plot is saved as:

`<jobname>_Step8_LZ_probabilities.png`

with the title:

`Effective, Intermediate, and MS-specific LZ Probabilities`

The effective, intermediate, and channel-specific curves are visually distinguished but are all evaluated on the same numerical grid.

4.8.38 Weak coupling probability plot

The WC plot is constructed analogously and contains:

- the effective WC curve;
- intermediate $|M_S|$ -resolved curves;
- representative equal- $|H_{S0}|$ channel curves; and
- the active MECP threshold.

The plot is saved as:

`<jobname>_Step8_WC_probabilities.png`

with the title:

`Effective, Intermediate, and MS-specific WC Probabilities`

Because the WC expression is evaluated below the barrier, the plotted WC curve can remain nonzero to the left of the MECP marker.

4.8.39 Step 8 summary file

The file:

`<jobname>_Step8_probability_summary.txt`

records:

- effective SOC;
- active MECP threshold;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- PROJECTED_GRADIENT_MEAN_EH_PER_BOHR;
- reduced mass in amu, kg, and atomic units;
- energy spacing;
- maximum grid index;
- MECP bin;
- energy- and velocity-grid sources;
- minimum and maximum effective LZ probabilities;
- minimum and maximum effective WC probabilities;
- number of intermediate curves;
- number of nonzero individual channels;
- number of unique plotted channel curves; and
- paths to all Step 8 output files.

4.8.40 Effective probability grid file

The complete effective probability grid is written to:

`<jobname>_Step8_effective_probability_grid.txt`

The columns are:

```
total_energy_grid_cm1
E_minus_MECP_cm1
v_m_s
v_bohr_s
P_LZ_effective
landau_zener_exponent_effective
P_WC_effective
airy_arg_WC_effective
Ai_WC_effective
```

For each total-energy bin, the file records:

E ,

$E - V_{\text{MECP}}$,

the velocity in both units, the effective LZ probability, the implemented LZ parameter, the effective WC probability, the Airy argument, and the Airy-function value.

This file provides all quantities needed to reproduce both effective probability curves.

4.8.41 Intermediate probability summary

The file:

`<jobname>_Step8_intermediate_probability_summary.txt`

records, for each intermediate group:

- its label;
- $|M_{S,\text{low}}|$;
- degeneracy;
- intermediate SOC value; and
- the contributing row SOC magnitudes.

A typical entry represents

$$|M_S| = \frac{1}{2},$$

the two rows $M_S = \pm 1/2$, their individual row norms, and the root-mean-square intermediate SOC generated from them.

4.8.42 M_S -resolved channel summary

The file:

`<jobname>_Step8_channel_probability_summary.txt`

contains three sections.

1-LZ plotting groups

This section records:

- group key;
- common $|H_{SO}|$;
- number of equivalent channels;
- channel symbols; and
- individual channel labels.

2-WC plotting groups

The same information is recorded independently for the WC curves.

3-Individual channels

Every nonzero matrix element is listed with:

- symbolic transition label;
- machine-readable label;
- real part of H_{ij} ;
- imaginary part of H_{ij} ;
- $|H_{ij}|$;
- S_{low} ;
- $M_{S,\text{low}}$;
- S_{high} ;
- $M_{S,\text{high}}$; and
- matrix indices.

Thus, grouping for plotting does not remove the underlying individual channel information.

4.8.43 Cluster output transfer

The local output directory is:

```
local_step8 = os.path.join(
    local_base,
    "LZ_WC_probabilities"
)
```

The corresponding cluster path is:

```
remote_step8 = posixpath.join(  
    remote_base,  
    "LZ_WC_probabilities"  
)
```

When an active SFTP connection is available, Step 8 transfers:

- the summary file;
- the effective grid;
- the intermediate summary;
- the channel summary;
- the LZ plot; and
- the WC plot.

The local calculations and files remain valid even if remote transfer fails.

4.8.44 Effective probability variables

The principal effective probability arrays exported are:

```
P_LZ_effective  
landau_zener_exponent_effective  
P_WC_effective  
airy_arg_WC_effective  
Ai_WC_effective
```

Backward-compatible aliases include:

```
P_LZ_effective  
gamma_LZ  
P_WC_effective.
```

The Step 9-compatible names are:

```
effective_LZ_probability_for_rates  
landau_zener_exponent_eff_rate  
effective_WC_probability_for_rates  
effective_WC_airy_argument_for_rates  
effective_WC_airy_value_for_rates  
velocity_bohr_per_s_for_rates.
```

These aliases ensure that Step 9 uses exactly the probabilities and velocity generated in Step 8 rather than recalculating them independently.

4.8.45 Intermediate and channel-specific variables

The intermediate probability dictionaries are:

P_LZ_intermediate
P_WC_intermediate.

The individual channel dictionaries are:

P_LZ_channels
P_WC_channels.

The representative plotting groups are:

P_LZ_channel_groups
P_WC_channel_groups.

The active SOC matrix and its source are retained as:

SOC_channel_matrix_cm1
SOC_channel_matrix_label.

These variables preserve the complete hierarchy

effective $\rightarrow$ intermediate $\rightarrow M_S$ -specific.

4.8.46 Exported probability functions

The calculation functions themselves are retained globally:

compute_landau_zener_probability_for_coupling
compute_WC_for_HSO_cm

This allows later workflow sections to generate probabilities for another SOC value without redefining the equations.

A custom LZ energy grid can be supplied only if matching velocity arrays are supplied explicitly. The function intentionally refuses to generate an independent velocity grid.

4.8.47 Summary of the step 8 algorithm

The complete effective-probability sequence begins with the Step 7 arrays:

$$\{E_j, v(E_j)\},$$

together with

$$\{H_{\text{SO}}^{\text{eff}}, \Delta F_{\parallel}, \bar{g}, \mu, V_{\text{MECP}}\}.$$

For the Landau-Zener treatment,

$$\gamma(E) = \frac{4\pi^2 H_{\text{SO}}^2}{\hbar \Delta F_{\parallel} v(E)},$$

and

$$P_{\text{LZ}}(E) = 1 - \exp[-2\gamma(E)]$$

for positive-velocity bins above the MECP.

For the weak coupling treatment,

$$P_{\text{WC}}(E) = 4\pi^2 H_{\text{SO}}^2 \left(\frac{2\mu}{\bar{g} \Delta F_{\parallel}} \right)^{2/3} \text{Ai}^2 \left[-\epsilon(E) \left(\frac{2\mu \Delta F_{\parallel}^2}{\bar{g}^4} \right)^{1/3} \right],$$

where

$$\epsilon(E) = E - V_{\text{MECP}}.$$

The effective probabilities use

$$H_{\text{SO}} = H_{\text{SO}}^{\text{eff}}.$$

For an individual channel,

$$H_{\text{SO}} = |H_{M_S, M'_S}|,$$

giving

$$P_{\text{LZ}}^{M_S, M'_S}(E)$$

and

$$P_{\text{WC}}^{M_S, M'_S}(E).$$

For an intermediate $|M_S|$ group, the row SOC is

$$H_i^{\text{row}} = \sqrt{\sum_{M'_S} |H_{M_S, i, M'_S}|^2},$$

and the group SOC is

$$H_{\text{int}}(|M_S|) = \sqrt{\frac{1}{n} \sum_{i \in |M_S|} (H_i^{\text{row}})^2}.$$

The corresponding variable flow is:

```

H_SO_cm
GRADIENT_GAP_NORM_EH_PER_BOHR
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
reduced_mass_amu
VaG_MECP_cm1
total_energy_grid_cm1
v_m_s
v_bohr_s
↓
grid and velocity validation
↓
P_LZ_effective
P_WC_effective

SOC_Ms_matrix_scaled_cm1
or SOC_Ms_matrix_cm1
↓
individual nonzero H_ij
↓
P_LZ_channels
P_WC_channels
↓
equal-|H_ij| plotting groups
(no summation)

SOC matrix rows grouped by |Ms_low|
↓
H_row
↓
H_int(|Ms_low|)
↓
P_LZ_intermediate
P_WC_intermediate

```

Therefore, Step 8 transforms the effective and M_S -resolved SOC information into a complete hierarchy of energy-dependent spin-crossing probabilities:

$$\boxed{\left\{H_{\text{SO}}^{\text{eff}}, \mathbf{H}_{\text{SO}}^{M_S, M'_S}, E_j, v_j, \Delta F_{\parallel}, \bar{F}, \mu\right\} \rightarrow \left\{P^{\text{eff}}, P^{|M_S|}, P^{M_S, M'_S}\right\}_{\text{LZ,WC}}}$$

The effective probability arrays are subsequently passed directly to Step 9, where they are combined with the reactant density of states and crossing-point sum of states to calculate the Landau-Zener and weak coupling microcanonical rate constants.

4.9 Step 9: Microcanonical Rate Constants

Step 9 converts the spin-crossing probabilities obtained in Step 8 into microcanonical reaction rate constants. No additional electronic-structure calculations are performed. Instead, the step combines the previously calculated Landau-Zener and weak coupling probabilities with the reactant and MECP rovibrational densities of states to evaluate effective, intermediate, and individual M_S -specific microcanonical rate constants over the selected energy range.

The principal operations performed in Step 9 are

1. importing all quantities required from Steps 1–8,
2. selecting the active ZPE-corrected MECP threshold,
3. constructing the common energy grid,
4. evaluating reactant and MECP rovibrational densities of states,
5. calculating effective Landau-Zener and weak coupling microcanonical rates,
6. calculating intermediate-spin microcanonical rates,
7. calculating individual M_S -specific microcanonical rates,
8. grouping symmetry-equivalent channels for visualization, and
9. generating numerical output files and publication-quality figures.

4.9.1 Import of quantities from the preceding steps

Step 9 begins by verifying that all quantities generated during the previous workflow are available before any microcanonical rate calculations are performed:

```
required_vars_step9 = [  
    "jobname",  
    "freq_reactant_real_cm1",  
    "freq_MECP_real_cm1",  
    "VaG_MECP_cm1",  
    "probability_energy_grid_cm1",  
    "H_SO_cm",  
    "GRADIENT_GAP_NORM_EH_PER_BOHR",  
    "reduced_mass_amu",  
    "Reference_rot_constants_cm1",  
    "MECP_rot_constants_cm1",  
    "local_base",  
    "remote_base"  
]:
```

```

for var in required_vars_step9:
    if var not in globals():
        raise RuntimeError(
            f"Required variable '{var}' is missing. "
            "Run cluster Steps 1–8 first."
        )

```

These variables provide all physical quantities required for the microcanonical rate calculations.

The vibrational frequencies:

```

freq_reactant_real_cm1
freq_MECP_real_cm1

```

define the vibrational density of states for the reactant minimum and the minimum-energy crossing point (MECP), respectively.

The energetic threshold:

```

VaG_MECP_cm1

```

contains the energy of the MECP relative to the reference minimum, while:

```

probability_energy_grid_cm1

```

contains the common energy grid established in Step 7.

The effective spin–orbit coupling:

```

H_SO_cm

```

is the scalar coupling selected in Step 2 and subsequently used in Step 8 to compute the effective Landau-Zener and weak coupling probabilities.

The quantity:

```

GRADIENT_GAP_NORM_EH_PER_BOHR

```

contains the projected gradient difference evaluated in Step 6, whereas:

```

reduced_mass_amu

```

contains the final reaction-coordinate reduced mass selected after the effective-Hessian analysis.

Finally:

```

Reference_rot_constants_cm1

```

and:

```

MECP_rot_constants_cm1

```

contain the rotational constants required for the rotational density-of-states calculations.

Before proceeding, SPINKS verifies that every required variable exists. If any quantity is missing, execution is terminated with an informative error message indicating that the previous workflow

steps must first be completed. This prevents incomplete datasets from propagating into the rate calculations.

4.9.2 Selection of the active ZPE-corrected MECP threshold

Before constructing the common energy grid, Step 9 checks which MECP barrier should be used as the active spin-crossing threshold:

```
# =====
# Energy check and active threshold
# =====

section("STEP 9 ENERGY CHECK")

if "VaG_MECP_electronic_cm1" in globals():
    print(
        f"Electronic MECP barrier      : "
        f"{VaG_MECP_electronic_cm1:.6f} cm^-1"
    )

if "VaG_MECP_ZPE_corrected_cm1" in globals():
    VaG_MECP_cm1 = float(VaG_MECP_ZPE_corrected_cm1)
    E_MECP = VaG_MECP_cm1
    E_MECP_cm1 = VaG_MECP_cm1

    print(
        f"ZPE-corrected MECP barrier  : "
        f"{VaG_MECP_ZPE_corrected_cm1:.6f} cm^-1"
    )
    print(
        f"Forced Step 9 threshold    : "
        f"{VaG_MECP_cm1:.6f} cm^-1"
    )

else:
    E_MECP = float(VaG_MECP_cm1)
    E_MECP_cm1 = E_MECP

    print(
        f"Active MECP barrier          : "
        f"{VaG_MECP_cm1:.6f} cm^-1"
    )
```

The first conditional statement checks whether the electronic MECP barrier is still available:

```
if "VaG_MECP_electronic_cm1" in globals():
```

When available, its value is printed for diagnostic purposes. This value represents the electronic energy difference between the MECP and the selected reference minimum before the zero-point energy correction is applied. It is retained so that the electronic and ZPE-corrected barriers can be compared directly.

The active threshold itself is selected by the second conditional statement:

```
if "VaG_MECP_ZPE_corrected_cm1" in globals():
```

The variable `VaG_MECP_ZPE_corrected_cm1` is generated in Step 7 according to

$$V_{\text{MECP}}^{\text{ZPE}} = V_{\text{MECP}}^{\text{electronic}} + [ZPE_{\text{MECP}}^{\text{effhess}} - ZPE_{\text{reference}}].$$

In the code, this quantity was previously constructed as

$$\begin{aligned} \text{VaG_MECP_ZPE_corrected_cm1} \\ = \text{VaG_MECP_electronic_cm1} + \text{Delta_ZPE_MECP_minus_REF_cm1}. \end{aligned}$$

When this ZPE-corrected value exists, Step 9 explicitly replaces the general MECP-barrier variable with it:

```
VaG_MECP_cm1 = float(VaG_MECP_ZPE_corrected_cm1)
```

Therefore, within Step 9,

$$\text{VaG_MECP_cm1} = V_{\text{MECP}}^{\text{ZPE}}.$$

The same active threshold is then assigned to two additional aliases:

```
E_MECP = VaG_MECP_cm1  
E_MECP_cm1 = VaG_MECP_cm1
```

Thus,

$$E_{\text{MECP}} = E_{\text{MECP_cm1}} = \text{VaG_MECP_cm1} = \text{VaG_MECP_ZPE_corrected_cm1}.$$

These aliases are used throughout the subsequent probability, density-of-states, convolution, and microcanonical-rate calculations. The assignments ensure that all parts of Step 9 employ exactly the same crossing threshold.

The printed statement:

```
print(  
    f"Forced Step 9 threshold      : "  
    f"{VaG_MECP_cm1:.6f} cm^-1"  
)
```

makes this replacement explicit in the console output. The word `Forced` indicates that Step 9 does not continue using a provisional electronic barrier when the ZPE-corrected barrier from Step 7 is available.

This is important because the onset of the spin-crossing contribution and the placement of the MECP density of states must correspond to the physically active, ZPE-corrected threshold rather than only the electronic energy difference. In the subsequent energy-grid construction, the corresponding threshold bin is determined from this value as

$$\text{binX} = \left\lceil \frac{E_{\text{MECP}}}{E_{\text{step}}} \right\rceil.$$

Consequently, changing the active MECP threshold also changes the position at which the shifted MECP density of states and the crossing probabilities begin contributing to the microcanonical rate.

The else block provides a fallback:

```
else:
    E_MECP = float(VaG_MECP_cm1)
    E_MECP_cm1 = E_MECP
```

This branch is used only when VaG_MECP_ZPE_corrected_cm1 is not available. In that case, Step 9 retains the existing value of VaG_MECP_cm1 and assigns it to the two MECP-energy aliases. This preserves compatibility with older workflow states or externally supplied datasets, although the normal SPINKS workflow provides the ZPE-corrected barrier through Step 7.

The resulting variable flow is:

```
VaG_MECP_electronic_cm1
ZPE_MECP_effhess_cm1
ZPE_REF_from_freq_cm1
    ↓
VaG_MECP_ZPE_corrected_cm1
    ↓
VaG_MECP_cm1
    ↓
E_MECP
E_MECP_cm1
    ↓
Step 9 energy grid and rate calculations
```

Therefore, the active threshold used in Step 9 is, whenever available, the ZPE-corrected MECP barrier generated in Step 7. The original electronic barrier remains available only as a separately reported diagnostic quantity.

4.9.3 Construction of the common total-energy grid

After selecting the active ZPE-corrected MECP threshold, Step 9 initializes the physical constants and reconstructs the common energy grid that is used throughout all subsequent density-of-states, probability, and microcanonical rate calculations:

```
section("CONSTANTS AND GRID SETUP")

autocm = 219474.6313705
h_SI = 6.62607015e-34
c_SI = 2.99792458e8
Eh_to_J = 4.3597447222071e-18
planck_hsec = h_SI / Eh_to_J

amu_kg = 1.66053906660e-27
bohr_m = 5.29177210903e-11
c_cm_s = 2.99792458e10
kJmol_per_cm1 = 0.01196266

probability_energy_grid_cm1 = np.asarray(probability_energy_grid_cm1, dtype=float).reshape(-1)

if probability_energy_grid_cm1.size == 0:
    raise RuntimeError("probability_energy_grid_cm1 is empty. Run Step 7 first.")

energy_step_cm1 = 1.0
maximum_energy_bin_index = int(np.ceil(np.nanmax(probability_energy_grid_cm1) /
energy_step_cm1))
total_energy_grid_cm1 = np.arange(maximum_energy_bin_index + 1, dtype=float) *
energy_step_cm1

crossing_energy_bin_index = int(np.ceil(E_MECP / energy_step_cm1))
zero_point_bin_offset = 0

reactant_degeneracy = float(globals().get("reactant_degeneracy", 1.0))
```

The first part of this block defines the physical constants required throughout the downstream calculations. These include the Hartree-to-wavenumber conversion factor (autocm), Planck's constant (h_SI), the speed of light (c_SI), the Hartree-to-joule conversion (Eh_to_J), atomic mass and Bohr-radius conversion factors (amu_kg and bohr_m), and the conversion from wavenumbers to kJ mol⁻¹ (kJmol_per_cm1). These constants are subsequently used for unit conversions in the probability, density-of-states, and rate calculations.

Next, the energy grid generated previously in Step 7 is recovered as a NumPy array:

```
probability_energy_grid_cm1 = np.asarray(probability_energy_grid_cm1, dtype=float).reshape(-1)
```

Rather than constructing a new user-defined energy axis, Step 9 explicitly reuses the common energy grid produced during Step 7. This guarantees that the microcanonical rate calculations are performed on exactly the same energy bins used previously for the Landau-Zener and weak coupling probability calculations. Consequently,

$$E_{\text{rate}} = E_{\text{probability}},$$

ensuring complete consistency between Steps 8 and 9.

Before continuing, the code verifies that the imported energy grid is not empty:

```
if probability_energy_grid_cm1.size == 0:
    raise RuntimeError("probability_energy_grid_cm1 is empty. Run Step 7 first.")
```

If no energy grid is available, execution is terminated immediately because the subsequent density-of-states and convolution calculations require a valid common energy axis.

The energy spacing is then fixed as:

```
energy_step_cm1 = 1.0
```

corresponding to a uniform increment of

$$\Delta E = 1 \text{ cm}^{-1}.$$

This 1 cm^{-1} resolution is identical to that adopted in Step 7 and matches the standard SPINKS implementation.

The highest energy represented on the grid is determined from the imported Step 7 array:

```
maximum_energy_bin_index = int(np.ceil(np.nanmax(probability_energy_grid_cm1) /
energy_step_cm1))
```

and the complete uniformly spaced energy grid is reconstructed as:

```
total_energy_grid_cm1 = np.arange(maximum_energy_bin_index + 1, dtype=float) *
energy_step_cm1
```

which corresponds mathematically to

$$E_i = i\Delta E, i = 0, 1, \dots, \text{maxn}_{\text{py}}.$$

Thus, `total_energy_grid_cm1` contains every integer energy bin from 0 cm^{-1} to the maximum energy selected by the user in Step 7.

Once the common energy grid has been established, the location of the active MECP threshold on that grid is identified by:

```
crossing_energy_bin_index = int(np.ceil(E_MECP / energy_step_cm1))
```

giving

$$\text{binX} = \left\lceil \frac{E_{\text{MECP}}}{\Delta E} \right\rceil$$

where E_{MECP} is the ZPE-corrected crossing energy selected in Section 4.9.2. The variable `crossing_energy_bin_index` therefore marks the first energy bin at which the crossing seam becomes energetically accessible. It is subsequently used to shift the MECP density of states relative to the reactant density of states during the RRKM convolution.

The reactant zero-point reference is assigned as:

```
zero_point_bin_offset = 0
```

indicating that all energies in Step 9 are measured relative to the reference minimum defined in Step 1. Consequently, the reactant density of states begins at the origin of the energy grid, while the MECP density of states begins at the shifted position defined by `crossing_energy_bin_index`.

Finally, the reactant electronic degeneracy is recovered as:

```
reactant_degeneracy = float(globals().get("reactant_degeneracy", 1.0))
```

If a degeneracy factor has already been determined during the earlier stages of the workflow, that value is reused directly. Otherwise, a default value of unity is adopted. This degeneracy factor is incorporated later into the RRKM expression for the microcanonical rate constant,

$$k(E) = \frac{g N(E)}{h \rho_R(E)},$$

where $g = \text{degenR}$, $N(E)$ is the effective number of crossing states, and $\rho_R(E)$ is the reactant rovibrational density of states.

This section therefore reconstructs the common total-energy grid, identifies the location of the ZPE-corrected MECP threshold, and establishes the energy framework that is used consistently throughout all subsequent density-of-states and microcanonical rate calculations.

4.9.4 Evaluation of the reactant and MECP rovibrational densities of states

Before calculating the microcanonical rate constants, Step 9 prepares the vibrational frequencies and rotational constants obtained from the reactant minimum and the MECP calculations. These quantities are subsequently used to construct the harmonic vibrational, rotational, and rovibrational densities of states for both the reactant and the crossing point:

```
section("FREQUENCY AND ROTATIONAL DATA")
```

```

freq_R = np.asarray(freq_reactant_real_cm1, dtype=float).reshape(-1)
freq_X = np.asarray(freq_MECP_real_cm1, dtype=float).reshape(-1)

freq_R = freq_R[np.isfinite(freq_R) & (freq_R > 0.0)]
freq_X = freq_X[np.isfinite(freq_X) & (freq_X > 0.0)]

rot_R_cm1 = np.asarray(Reference_rot_constants_cm1, dtype=float).reshape(-1)
rot_X_cm1 = np.asarray(MECP_rot_constants_cm1, dtype=float).reshape(-1)

rot_R_cm1 = rot_R_cm1[np.isfinite(rot_R_cm1) & (rot_R_cm1 > 0.0)]
rot_X_cm1 = rot_X_cm1[np.isfinite(rot_X_cm1) & (rot_X_cm1 > 0.0)]

if len(rot_R_cm1) != 3:
    raise RuntimeError("Reference_rot_constants_cm1 must contain 3 positive rotational constants.")

if len(rot_X_cm1) != 3:
    raise RuntimeError("MECP_rot_constants_cm1 must contain 3 positive rotational constants.")

if len(freq_R) == 0:
    raise RuntimeError("No positive reactant frequencies were found.")

if len(freq_X) == 0:
    raise RuntimeError("No positive MECP effective-Hessian frequencies were found.")

```

The reactant and MECP vibrational frequencies are first converted into one-dimensional NumPy arrays,

$$\{v_i^R\}, \{v_i^X\},$$

using:

```

freq_R = np.asarray(...)
freq_X = np.asarray(...)

```

The subsequent filtering operation:

```

freq_R = freq_R[np.isfinite(freq_R) & (freq_R > 0.0)]
freq_X = freq_X[np.isfinite(freq_X) & (freq_X > 0.0)]

```

removes any nonphysical entries, including undefined values (NaN), infinite values, and frequencies less than or equal to zero. Consequently, only positive real vibrational frequencies contribute to the density-of-states calculations. For the MECP, these frequencies correspond to the effective-Hessian frequencies obtained in Step 5 after removal of the reaction-coordinate mode.

The rotational constants of the reactant minimum and the MECP are processed similarly:

```

rot_R_cm1 = np.asarray(...)
rot_X_cm1 = np.asarray(...)

```

followed by removal of any invalid or nonpositive values. Since a nonlinear molecule possesses three principal moments of inertia, Step 9 verifies that exactly three positive rotational constants remain for each structure. If this condition is not satisfied, execution is terminated to prevent construction of an incorrect rotational density of states.

The rotational constants are subsequently converted into principal moments of inertia using:

```
def rot_constants_to_inertia_amu_bohr2(B_cm1):
    conv = h_SI / (8.0 * np.pi**2 * c_cm_s) / (amu_kg * bohr_m**2)
    return conv / np.asarray(B_cm1, dtype=float)

inertR = rot_constants_to_inertia_amu_bohr2(rot_R_cm1)
inertX = rot_constants_to_inertia_amu_bohr2(rot_X_cm1)
```

This routine applies the rigid-rotor relationship

$$B = \frac{h}{8\pi^2 cI},$$

which is rearranged to obtain the principal moment of inertia,

$$I = \frac{h}{8\pi^2 cB}$$

where B is the rotational constant expressed in cm^{-1} . The resulting moments of inertia are stored in units of $\text{amu}\cdot\text{Bohr}^2$, matching the units required by the classical rotational density-of-states formalism.

After the vibrational frequencies and moments of inertia have been prepared, Step 9 evaluates the vibrational and rotational densities of states:

```
vibrational_density_reference = build_harmonic_ladder_density(freq_R,
maximum_energy_bin_index, energy_step_cm1)
vibrational_density_crossing = build_harmonic_ladder_density(freq_X,
maximum_energy_bin_index, energy_step_cm1)

rotational_density_reference = build_classical_rotational_density(inertR,
maximum_energy_bin_index, energy_step_cm1, autocm)
rotational_density_crossing = build_classical_rotational_density(inertX,
maximum_energy_bin_index, energy_step_cm1, autocm)
```

The function `build_harmonic_ladder_density()` generates the vibrational density of states by distributing the harmonic vibrational levels over the common 1 cm^{-1} energy grid constructed in Section 4.9.3. Separate vibrational densities are generated for the reactant minimum,

$$\rho_{\text{vib}}^R(E),$$

and for the MECP,

$$\rho_{\text{vib}}^X(E).$$

Similarly, `build_classical_rotational_density()` evaluates the rotational density of states using the product of the three principal moments of inertia,

$$I_A I_B I_C.$$

For each energy bin, the rotational density is computed according to

$$\rho_{\text{rot}}(E) = \frac{4}{a_{\text{u} \rightarrow \text{cm}^{-1}}} \sqrt{2 E I_A I_B I_C},$$

where $a_{\text{u} \rightarrow \text{cm}^{-1}}$ is the Hartree-to- cm^{-1} conversion factor (`autocm`). This expression corresponds directly to the classical rigid-rotor density employed in the original SPINKS implementation.

Finally, the vibrational and rotational densities are combined to generate the rovibrational densities of states used throughout the remainder of Step 9:

```
reactant_state_density = np.zeros(maximum_energy_bin_index + 1, dtype=float)
crossing_internal_state_density = np.zeros(maximum_energy_bin_index + 1, dtype=float)
crossing_shifted_state_density = np.zeros(maximum_energy_bin_index + 1, dtype=float)

...

reactant_state_density[i] = total
crossing_internal_state_density[i] = total

if crossing_energy_bin_index + 1 <= maximum_energy_bin_index:
    crossing_shifted_state_density[crossing_energy_bin_index + 1:maximum_energy_bin_index + 1]
    = crossing_internal_state_density[1:maximum_energy_bin_index - crossing_energy_bin_index + 1]
```

The reactant rovibrational density of states,

$$\rho_R(E),$$

is obtained by convolving the reactant rotational and vibrational densities. An analogous convolution is performed for the MECP to produce its internal rovibrational density,

$$\rho_{\text{TP}}(E),$$

which is subsequently shifted by the ZPE-corrected MECP threshold (`crossing_energy_bin_index`) to generate

$$\rho_X(E),$$

the shifted crossing-point density required by the Landau-Zener formulation. Consequently, Step 9 constructs three distinct rovibrational densities of states:

- $\rho_R(E)$ (`reactant_state_density`): reactant rovibrational density of states.
- $\rho_{\text{TP}}(E)$ (`crossing_internal_state_density`): internal MECP rovibrational density of states used in the weak coupling formulation.
- $\rho_X(E)$ (`crossing_shifted_state_density`): threshold-shifted MECP rovibrational density of states used in the Landau-Zener formulation.

These quantities provide the statistical foundation for the convolution integrals used in the following sections to compute the effective number of crossing states and the corresponding microcanonical reaction rate constants.

4.9.5 Independent evaluation of the rovibrational densities of states

Following preparation of the vibrational frequencies and rotational constants, Step 9 evaluates the reactant and MECP rovibrational densities of states required for the subsequent microcanonical rate calculations. This is accomplished using the same independently implemented density-of-states algorithm employed in the original SPINKS implementation.

The vibrational and rotational densities of states are first evaluated independently using dedicated helper functions:

```
def build_harmonic_ladder_density(freqs_cm1, maxn, energy_step_cm1):
    ...
    return vib

def build_classical_rotational_density(inert, maxn, energy_step_cm1, autocm):
    ...
```

```
return rot
```

```
vibrational_density_reference = build_harmonic_ladder_density(freq_R,
maximum_energy_bin_index, energy_step_cm1)
vibrational_density_crossing = build_harmonic_ladder_density(freq_X,
maximum_energy_bin_index, energy_step_cm1)

rotational_density_reference = build_classical_rotational_density(inertR,
maximum_energy_bin_index, energy_step_cm1, autocm)
rotational_density_crossing = build_classical_rotational_density(inertX,
maximum_energy_bin_index, energy_step_cm1, autocm)
```

The function `build_harmonic_ladder_density()` constructs the vibrational density of states by distributing the harmonic vibrational energy levels over the common energy grid established in Section 4.9.3. Separate vibrational densities are computed for the reactant minimum,

$$\rho_{\text{vib}}^R(E),$$

and the MECP,

$$\rho_{\text{vib}}^X(E).$$

Similarly, `build_classical_rotational_density()` evaluates the rotational density of states using the three principal moments of inertia obtained in Section 4.9.4. The rotational density at each energy bin is calculated according to

$$\rho_{\text{rot}}(E) = \frac{4}{\text{autocm}} \sqrt{2E I_A I_B I_C},$$

where I_A , I_B , and I_C are the principal moments of inertia expressed in $\text{amu} \cdot \text{Bohr}^2$, and `autocm` is the Hartree-to- cm^{-1} conversion factor.

The MECP vibrational and rotational densities are then shifted to the position of the active crossing threshold:

```
vibrational_density_crossing = np.zeros(maximum_energy_bin_index + 1, dtype=float)
if crossing_energy_bin_index + 1 <= maximum_energy_bin_index:
    vibrational_density_crossing[crossing_energy_bin_index + 1:maximum_energy_bin_index + 1]
= vibrational_density_crossing[1:maximum_energy_bin_index - crossing_energy_bin_index + 1]

rotational_density_crossing = np.zeros(maximum_energy_bin_index + 1, dtype=float)
for i in range(1, maximum_energy_bin_index - crossing_energy_bin_index + 1):
    rotational_density_crossing[i + crossing_energy_bin_index] = (
```

```

4.0
* np.sqrt(
2.0 * float(i) * energy_step_cm1 / autocm * np.prod(inertX)
)
/ autocm
)

```

The variable `crossing_energy_bin_index`, determined in Section 4.9.3, represents the energy bin corresponding to the ZPE-corrected MECP barrier. Consequently, the MECP densities of states are translated along the energy axis so that they become accessible only above the spin-crossing threshold. This shift ensures that the crossing-point density contributes only at physically allowed energies.

The vibrational and rotational densities are subsequently combined through discrete convolution to obtain the rovibrational densities of states:

```

reactant_state_density = np.zeros(maximum_energy_bin_index + 1, dtype=float)
crossing_internal_state_density = np.zeros(maximum_energy_bin_index + 1, dtype=float)
crossing_shifted_state_density = np.zeros(maximum_energy_bin_index + 1, dtype=float)

for i in range(1, maximum_energy_bin_index - zero_point_bin_offset + 1):
    total = 0.0

    for j in range(1, i + 1):
        total += rotational_density_reference[j] * vibrational_density_reference[i - j + 1] *
        energy_step_cm1

    reactant_state_density[i] = total

for i in range(1, maximum_energy_bin_index + 1):
    total = 0.0

    for j in range(1, i + 1):
        total += rotational_density_crossing[j] * vibrational_density_crossing[i - j + 1] *
        energy_step_cm1

    crossing_internal_state_density[i] = total

if crossing_energy_bin_index + 1 <= maximum_energy_bin_index:
    crossing_shifted_state_density[crossing_energy_bin_index + 1:maximum_energy_bin_index +
    1] = crossing_internal_state_density[1:maximum_energy_bin_index -
    crossing_energy_bin_index + 1]

```

For each energy bin, the rovibrational density of states is obtained by numerically convolving the rotational and vibrational densities,

$$\rho(E) = \sum_j \rho_{\text{rot}}(E_j) \rho_{\text{vib}}(E - E_j) \Delta E$$

where $\Delta E = 1 \text{ cm}^{-1}$ is the grid spacing.

This procedure produces three distinct rovibrational densities of states:

- **reactant_state_density** ($\rho_R(E)$): the reactant rovibrational density of states.
- **crossing_internal_state_density** ($\rho_{TP}(E)$): the internal MECP rovibrational density of states used in the weak coupling formulation.
- **crossing_shifted_state_density** ($\rho_X(E)$): the threshold-shifted MECP rovibrational density of states used in the Landau-Zener formulation.

The two arrays **crossing_internal_state_density** and **crossing_shifted_state_density** describe the same MECP rovibrational states using different energy references. **crossing_internal_state_density** is indexed by the internal energy of the crossing region, whereas **crossing_shifted_state_density** places the same density on the total-energy grid by shifting it upward by the MECP barrier. The weak coupling convolution uses the internal representation because its probability expression already includes the crossing-energy dependence, whereas the Landau-Zener convolution uses the barrier-shifted representation.

Finally, the maximum values of each density are printed as a diagnostic:

```
print(f"Reactant DOS max      : {np.nanmax(reactant_state_density):.6e}")
print(f"MECP internal DOS max  : {np.nanmax(crossing_internal_state_density):.6e}")
print(f"MECP shifted LZ DOS max : {np.nanmax(crossing_shifted_state_density):.6e}")
```

These diagnostics provide a convenient verification that the reactant and MECP densities of states have been constructed successfully before they are employed in the convolution expressions used to calculate the effective number of crossing states and the corresponding microcanonical rate constants in the subsequent sections.

4.9.6 Evaluation of the Landau-Zener transition probability

Following construction of the rovibrational densities of states, Step 9 defines a general routine for evaluating the energy-dependent Landau-Zener transition probability corresponding to an arbitrary spin-orbit coupling constant. This function is subsequently used to calculate the effective, intermediate, and individual M_S -specific transition probabilities required for the microcanonical rate calculations:

```
def compute_landau_zener_probability_for_coupling(H_cm, E_grid_cm1):
    E_grid_cm1 = np.asarray(E_grid_cm1, dtype=float).reshape(-1)

    H_cm = float(abs(H_cm))
    gradient_gap_local = abs(float(GRADIENT_GAP_NORM_EH_PER_BOHR))
    mu_kg_local = float(reduced_mass_amu) * amu_kg

    H_J = h_SI * c_SI * H_cm * 100.0
```

```
E_excess_J = (
    (E_grid_cm1 - E_MECP)
    * 100.0
    * h_SI
    * c_SI
)
```

```
v_m_s_local = np.zeros_like(E_grid_cm1)
```

```
mask_energy = E_excess_J > 0.0
```

➤

```
v_m_s_local[mask_energy] = np.sqrt(
    2.0 * E_excess_J[mask_energy] / mu_kg_local
```

```
)
```

$$P_{LZ} \propto |H_{SO}|^2,$$

```
v_bohr_s_local = v_m_s_local / bohr_m
```

```
gamma = np.full_like(E_grid_cm1, np.nan, dtype=float)
```

```
P = np.zeros_like(E_grid_cm1, dtype=float)
```

```
mask = (
```

```
    np.isfinite(E_grid_cm1)
```

```
    & np.isfinite(v_bohr_s_local)
```

$$H_{SO}(J) = hc H_{SO}(\text{cm}^{-1}),$$

```
    & (E_grid_cm1 >= E_MECP)
```

```
    & (v_bohr_s_local > 1.0e-12)
```

```
)
```

$$\text{cm}^{-1}\text{cm}^{-1}$$

```
energy_gap_sweep_rate_Eh_s = gradient_gap_local * v_bohr_s_local
```

```
energy_gap_sweep_rate_J_s = energy_gap_sweep_rate_Eh_s * Eh_to_J
```

```
gamma[mask] = (
    4.0 * np.pi**2 * H_J**2
    / (h_SI * energy_gap_sweep_rate_J_s[mask])
```

)

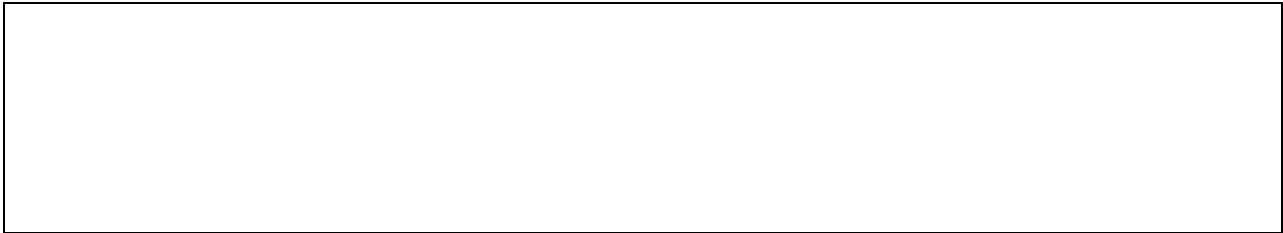
```
P[mask] = 1.0 - np.exp(-2.0 * gamma[mask])
P = np.clip(P, 0.0, 1.0)
```

```
return P, gamma, v_bohr_s_local
```

$$E > E_{\text{MECP}}$$

$$v = \sqrt{\frac{2(E - E_{\text{MECP}})}{\mu}}$$

$$E - E_{\text{MECP}}\mu$$



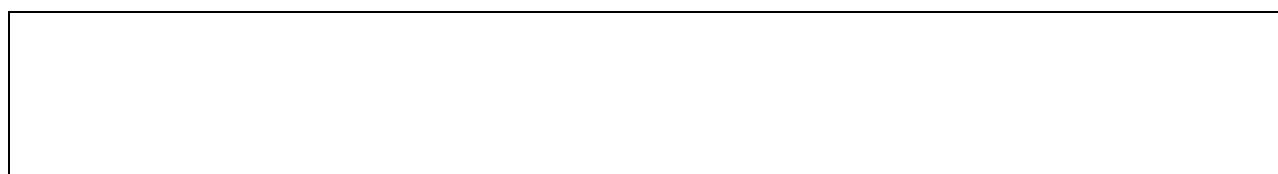
$$\frac{d\Delta E}{dt},$$

$$\Delta F,$$

$$v.$$

$$\frac{d\Delta E}{dt} = \Delta F v$$

$$\gamma = \frac{4\pi^2 H_{\text{SO}}^2}{h (d\Delta E/dt)}$$



The probability is evaluated using

$$P_{\text{LZ}} = 1 - e^{-2\gamma}$$

and is subsequently constrained to the physically meaningful interval

$$0 \leq P_{\text{LZ}} \leq 1.$$

The function returns three arrays:

- P , the Landau-Zener transition probability,
- γ , the corresponding adiabaticity parameter, and
- $v_{\text{bohr_s_local}}$, the reaction-coordinate velocity.

Because the routine accepts an arbitrary spin-orbit coupling value as input, the same implementation is reused throughout Step 9 to generate the effective, intermediate, and individual M_S -specific Landau-Zener probabilities without duplicating the computational procedure.

4.9.7 Calculation of the effective number of crossing states and microcanonical rate constants

Once the energy-dependent Landau-Zener and weak coupling transition probabilities have been determined, Step 9 defines two helper functions that convert these probabilities into the corresponding effective number of crossing states and microcanonical reaction rate constants. Separate routines are employed for the Landau-Zener and weak coupling formalisms because the two methods utilize different MECP rovibrational densities of states. In the implemented calculations, the threshold-shifted MECP density of states is used for the Landau-Zener convolution, whereas the internal MECP density of states is used for the weak coupling convolution because the weak coupling probability already incorporates the crossing-energy dependence. Landau-Zener Microcanonical Rate Helper:

```
def compute_landau_zener_microcanonical_rate(probability):
    cumulative_states = convolve_probability_with_crossing_density(
        probability=probability,
        crossing_density=crossing_internal_state_density,
        probability_start_index=crossing_energy_bin_index + 1,
        output_start_index=crossing_energy_bin_index + 1
    )

    rate = microcanonical_rate_from_cumulative_states(
        cumulative_states,
        reactant_state_density,
        reactant_degeneracy
    )

    return cumulative_states, rate
```

The function accepts the Landau-Zener probability array calculated in Section 4.9.6 and first initializes arrays for the effective number of crossing states (nos) and the corresponding microcanonical rate constants (rate).

The effective number of crossing states is obtained through a discrete convolution of the Landau-Zener probability with the threshold-shifted MECP rovibrational density of states (crossing_shifted_state_density),

$$N_{\text{LZ}}(E_i) = \sum_{j=\text{binX}}^i P_{\text{LZ}}(E_j) \rho_X(E_i - E_j) \Delta E$$

where

- $P_{\text{LZ}}(E_j)$ is the Landau-Zener transition probability,
- $\rho_X(E)$ is the shifted MECP rovibrational density of states,
- $\Delta E = 1 \text{ cm}^{-1}$ is the energy-grid spacing.

In the implementation, this convolution is performed through the nested loops:

```
for i in range(crossing_energy_bin_index + 1, maximum_energy_bin_index + 1):
    ...
    for j in range(crossing_energy_bin_index + 1, i + 1):
```

where each probability value is multiplied by the corresponding shifted density of states before numerical integration over the energy grid.

After the convolution is complete, the accumulated number of crossing states is converted from Hartree to wavenumber units by dividing by the conversion factor autocm.

The Landau-Zener microcanonical rate constant is then evaluated using the RRKM expression:

```
rate = microcanonical_rate_from_cumulative_states(
    reactant_state_density[i] * planck_hsec
)
```

which corresponds to

$$k_{\text{LZ}}(E) = \frac{g_R N_{\text{LZ}}(E)}{h \rho_R(E)}$$

where

- g_R is the reactant electronic degeneracy (reactant_degeneracy),
- $N_{\text{LZ}}(E)$ is the effective number of crossing states,

- $\rho_R(E)$ is the reactant rovibrational density of states,
- h is Planck's constant (planck_hsec).

The effective number of crossing states, $N(E)$, is a dimensionless energy-dependent statistical quantity rather than a literal integer count. It represents the probability-weighted number of crossing-region states accessible at total energy E .

The weak coupling formalism follows an analogous procedure but utilizes the **internal** MECP rovibrational density of states rather than the threshold-shifted density:

```
def compute_weak_coupling_microcanonical_rate(probability):
    cumulative_states = convolve_probability_with_crossing_density(
        probability=probability,
        crossing_density=crossing_internal_state_density,
        probability_start_index=1,
        output_start_index=1
    )

    rate = microcanonical_rate_from_cumulative_states(
        cumulative_states,
        reactant_state_density,
        reactant_degeneracy
    )

    return cumulative_states, rate
```

The corresponding number of crossing states is evaluated as

$$N_{WC}(E_i) = \sum_{j=1}^i P_{WC}(E_j) \rho_{TP}(E_i - E_j) \Delta E$$

where $\rho_{TP}(E)$ is the internal MECP rovibrational density of states (crossing_internal_state_density). Unlike the Landau-Zener formulation, no explicit threshold shift is introduced because the weak coupling probability already incorporates the energetic dependence of the crossing process.

The resulting weak coupling microcanonical rate constant is calculated using the same RRKM relationship,

$$k_{\text{WC}}(E) = \frac{g_R N_{\text{WC}}(E)}{h \rho_R(E)}$$

thereby ensuring that both theories employ an identical statistical treatment while differing only in the transition probability and the associated MECP density of states.

Finally, the statement:

```
compute_landau_zener_microcanonical_rate = compute_landau_zener_microcanonical_rate
```

assigns the Landau-Zener routine as the default microcanonical rate helper for any subsequent generic calls within the workflow, while the dedicated Landau-Zener and weak coupling functions remain available for explicit use throughout the remainder of Step 9.

4.9.8 Calculation of the effective Landau-Zener and weak coupling microcanonical rate constants

After defining the helper routines for evaluating the Landau-Zener and weak coupling microcanonical rate constants, Step 9 calculates the effective microcanonical rates using the effective spin-orbit coupling determined in Step 2. These effective quantities represent the principal microcanonical rate constants that are subsequently used throughout the remainder of the SPINKS workflow:

```
section("EFFECTIVE LZ/WC RATE CALCULATION")

effective_LZ_probability_for_rates,          landau_zener_exponent_eff_rate,
velocity_bohr_per_s_for_rates = (
    compute_landau_zener_probability(H_SO_cm, total_energy_grid_cm1)
)

effective_LZ_cumulative_states,              effective_LZ_rate                =
compute_landau_zener_microcanonical_rate(effective_LZ_probability_for_rates)

probLZ = effective_LZ_probability_for_rates
gamma_LZ_rate = landau_zener_exponent_eff_rate
LZ_cumulative_states = effective_LZ_cumulative_states
LZ_rate = effective_LZ_rate
```

The calculation begins by evaluating the Landau-Zener probability over the entire energy grid using the effective spin-orbit coupling ($H_{\text{SO_cm}}$) obtained in Step 2. The helper function introduced in Section 4.9.6 returns three quantities simultaneously:

- `effective_LZ_probability_for_rates`: the effective Landau-Zener transition probability,

- `landau_zener_exponent_eff_rate`: the corresponding Landau-Zener adiabaticity parameter,
- `velocity_bohr_per_s_for_rates`: the reaction-coordinate velocity.

Thus,

$$H_{SO}^{\text{eff}} \rightarrow P_{LZ}^{\text{eff}}(E), \gamma(E), v(E).$$

The effective Landau-Zener probability is then passed directly to the microcanonical rate helper:

<code>effective_LZ_cumulative_states,</code> <code>compute_landau_zener_microcanonical_rate(effective_LZ_probability_for_rates)</code>	=
---	---

which evaluates the effective number of crossing states,

$$N_{LZ}^{\text{eff}}(E),$$

followed by the effective Landau-Zener microcanonical rate constant,

$$k_{LZ}^{\text{eff}}(E).$$

The workflow therefore follows the sequence

$H_{SO}^{\text{eff}} \rightarrow P_{LZ}^{\text{eff}}(E) \rightarrow N_{LZ}^{\text{eff}}(E) \rightarrow k_{LZ}^{\text{eff}}(E)$
--

Finally, the effective quantities are assigned to the backward-compatible variables:

<code>probLZ = effective_LZ_probability_for_rates</code> <code>gamma_LZ_rate = landau_zener_exponent_eff_rate</code> <code>LZ_cumulative_states = effective_LZ_cumulative_states</code> <code>LZ_rate = effective_LZ_rate</code>

These aliases preserve compatibility with earlier versions of SPINKS and allow subsequent workflow steps to reference the effective Landau-Zener results without modifying existing variable names.

The effective weak coupling microcanonical rate is evaluated in an analogous manner:

<pre>if "compute_WC_for_HSO_cm" in globals(): effective_WC_probability_for_rates, effective_WC_airy_argument_for_rates, effective_WC_airy_value_for_rates = compute_WC_for_HSO_cm(H_SO_cm, total_energy_grid_cm1</pre>

```

)
elif "P_WC_effective" in globals():
    effective_WC_probability_for_rates = np.asarray(P_WC_effective, dtype=float).reshape(-1)
    ...
else:
    raise RuntimeError("WC probability is missing. Run Step 8 first.")

effective_WC_cumulative_states, effective_WC_rate =
compute_weak_coupling_microcanonical_rate(effective_WC_probability_for_rates)

probWC = effective_WC_probability_for_rates
WC_cumulative_states = effective_WC_cumulative_states
WC_rate = effective_WC_rate

```

Whenever the helper routine `compute_WC_for_HSO_cm()` is available, the weak coupling probability is recalculated directly using the effective spin-orbit coupling. Otherwise, the previously computed probability array (`P_WC_effective`) generated in Step 8 is reused after verifying that its dimensions match the common energy grid. If neither source is available, execution is terminated because the weak coupling microcanonical rates cannot be evaluated.

The effective weak coupling probability is subsequently converted into the corresponding effective number of crossing states and microcanonical rate constant,

$$H_{\text{SO}}^{\text{eff}} \rightarrow P_{\text{WC}}^{\text{eff}}(E) \rightarrow N_{\text{WC}}^{\text{eff}}(E) \rightarrow k_{\text{WC}}^{\text{eff}}(E)$$

through the statement:

```

effective_WC_cumulative_states, effective_WC_rate =
compute_weak_coupling_microcanonical_rate(effective_WC_probability_for_rates)

```

As for the Landau-Zener formulation, the effective weak coupling quantities are copied to the generic variables:

```

probWC = effective_WC_probability_for_rates
WC_cumulative_states = effective_WC_cumulative_states
WC_rate = effective_WC_rate

```

to maintain backward compatibility throughout the remainder of the workflow.

Finally, Step 9 reports the ranges of the effective probabilities and microcanonical rate constants:

```

print(f"P_LZ effective range : ...")
print(f"k_LZ effective range : ...")
print(f"P_WC effective range : ...")
print(f"k_WC effective range : ...")

```

These diagnostic values provide a convenient verification that both theories have been evaluated successfully before the calculation proceeds to the intermediate-spin and individual M_S -specific microcanonical rate constants described in the following sections.

4.9.9 Calculation of the intermediate microcanonical rate constants

In addition to the effective microcanonical rate constants, Step 9 evaluates the microcanonical rates corresponding to the intermediate spin-orbit couplings generated in Step 8. These intermediate quantities represent the coupling strengths associated with the grouped $|M_S|$ manifolds and provide insight into the contribution of each intermediate spin projection to the overall spin-crossing process.

Landau-Zener Intermediate Rate Constants:

```
section("INTERMEDIATE RATE CALCULATION")

k_LZ_intermediate = {}
N_LZ_intermediate = {}
P_LZ_intermediate_rate_grid = {}

if "P_LZ_intermediate" in globals() and len(P_LZ_intermediate) > 0:
    for int_label, int_data in P_LZ_intermediate.items():
        H_int = float(int_data["H_int_cm1"])

        P_int, gamma_int, _ = compute_landau_zener_probability(
            H_int,
            total_energy_grid_cm1
        )

        N_int, k_int = compute_landau_zener_microcanonical_rate(P_int)

        P_LZ_intermediate_rate_grid[int_label] = P_int
        N_LZ_intermediate[int_label] = N_int
        k_LZ_intermediate[int_label] = {
            "H_int_cm1": H_int,
            "abs_Ms_low": float(int_data["abs_Ms_low"]),
            "degeneracy": int(int_data["degeneracy"]),
            "k": k_int,
            "N": N_int,
            "P": P_int
        }
    }
```

The intermediate Landau-Zener calculations begin by creating dictionaries that will store the transition probabilities, effective numbers of crossing states, and microcanonical rate constants for every intermediate spin manifold generated in Step 8.

The calculation then loops over every intermediate coupling:

```
for int_label, int_data in P_LZ_intermediate.items():
```

where each intermediate is identified by its corresponding spin–orbit coupling:

```
H_int = float(int_data["H_int_cm1"])
```

Rather than reusing the probabilities stored in Step 8, SPINKS recalculates the Landau-Zener probability over the common Step 9 energy grid:

```
P_int, gamma_int, _ = compute_landau_zener_probability(
    H_int,
    total_energy_grid_cm1
)
```

This ensures that every intermediate probability is evaluated using exactly the same energy grid, MECP threshold, reduced mass, and projected gradient difference employed throughout the remainder of Step 9.

The resulting probability is subsequently converted into the corresponding effective number of crossing states and microcanonical rate constant:

```
N_int, k_int = compute_landau_zener_microcanonical_rate(P_int)
```

which corresponds to the sequence

$$H_{\text{int}} \rightarrow P_{\text{LZ}}^{\text{int}}(E) \rightarrow N_{\text{LZ}}^{\text{int}}(E) \rightarrow k_{\text{LZ}}^{\text{int}}(E)$$

Finally, the complete set of intermediate quantities is stored in the dictionary `k_LZ_intermediate`, together with the corresponding intermediate spin–orbit coupling (`H_int_cm1`), the associated absolute spin projection (`abs_Ms_low`), and the degeneracy of the grouped manifold. These quantities are subsequently used for the generation of intermediate-rate plots and numerical summaries.

The intermediate weak coupling rate constants are calculated using an analogous procedure:

```
k_WC_intermediate = {}
N_WC_intermediate = {}
P_WC_intermediate_rate_grid = {}

if "P_WC_intermediate" in globals() and len(P_WC_intermediate) > 0:
    for int_label, int_data in P_WC_intermediate.items():

        H_int = float(int_data["H_int_cm1"])

        if "compute_WC_for_HSO_cm" in globals():
            P_int, _, _ = compute_WC_for_HSO_cm(
                H_int,
                total_energy_grid_cm1
```

```

    )
else:
    P_int = np.asarray(
        int_data["P"],
        dtype=float
    ).reshape(-1)

    N_int, k_int = compute_weak_coupling_microcanonical_rate(P_int)

    ...

```

For each intermediate spin–orbit coupling, Step 9 first attempts to recompute the weak coupling probability using the dedicated weak coupling probability routine. If this routine is unavailable, the probability array calculated previously in Step 8 is reused after confirming that its dimensions are consistent with the common energy grid.

Each probability array is then transformed into the corresponding effective number of crossing states and microcanonical rate constant,

$$H_{\text{int}} \rightarrow P_{\text{WC}}^{\text{int}}(E) \rightarrow N_{\text{WC}}^{\text{int}}(E) \rightarrow k_{\text{WC}}^{\text{int}}(E)$$

through the statement:

```
N_int, k_int = compute_weak_coupling_microcanonical_rate(P_int)
```

The resulting intermediate quantities are stored in `k_WC_intermediate` together with the corresponding spin projection and degeneracy information inherited from Step 8.

After all intermediate spin manifolds have been processed, Step 9 reports the total number of intermediate Landau-Zener and weak coupling microcanonical rate curves generated:

```

print(f"Computed LZ intermediate rates for {len(k_LZ_intermediate)} curves.")

...

print(f"Computed WC intermediate rates for {len(k_WC_intermediate)} curves.")

```

These intermediate microcanonical rate constants provide a decomposition of the effective rate into contributions associated with the grouped $|M_S|$ manifolds. Unlike the effective rates, which are based on a single effective spin–orbit coupling, the intermediate rates preserve the dependence of the microcanonical kinetics on the intermediate spin projections identified during the spin–orbit coupling analysis in Step 8.

4.9.10 Calculation of M_S -specific microcanonical rate constants

The final stage of the rate evaluation extends the RRKM treatment from the effective and intermediate spin-orbit couplings to every individual M_S -resolved spin-crossing channel identified in Step 8. Rather than using an averaged or intermediate coupling, each allowed spin-transition channel is treated independently by employing its corresponding absolute spin-orbit coupling magnitude, $|H_{SO}|$. This produces a complete set of M_S -specific microcanonical Landau-Zener and weak coupling rate constants that describe the contribution of every symmetry-allowed spin-crossing pathway. No averaging or summation of channels is performed at this stage; every transition is propagated independently through the complete SPINKS rate formalism.

The corresponding implementation begins by allocating separate dictionaries for storing the M_S -specific Landau-Zener probabilities, effective numbers of crossing states, and microcanonical rate constants:

```
# =====  
# MS-specific channel rates  
# =====  
  
section("MS-SPECIFIC CHANNEL RATE CALCULATION")  
  
k_LZ_channels = {}  
N_LZ_channels = {}  
P_LZ_channels_rate_grid = {}  
  
if "P_LZ_channels" in globals() and len(P_LZ_channels) > 0:  
    for ch_label, ch_data in P_LZ_channels.items():  
        H_abs = float(ch_data["H_abs_cm1"])  
  
        P_ch, gamma_ch, _ = compute_landau_zener_probability(  
            H_abs,  
            total_energy_grid_cm1  
        )  
  
        N_ch, k_ch = compute_landau_zener_microcanonical_rate(P_ch)  
  
        P_LZ_channels_rate_grid[ch_label] = P_ch  
        N_LZ_channels[ch_label] = N_ch  
        k_LZ_channels[ch_label] = k_ch
```

For each M_S -specific transition channel generated previously, the code retrieves the corresponding absolute spin-orbit coupling,

$$|H_{SO}|,$$

stored as `H_abs_cm1`. The use of the absolute magnitude reflects the fact that both the Landau-Zener and weak coupling transition probabilities depend on the square of the electronic coupling,

$$P = f(H_{SO}^2),$$

so channels differing only in the sign of the spin-orbit matrix element yield identical transition probabilities and therefore identical microcanonical rate constants.

Using this channel-specific coupling, the routine `compute_landau_zener_probability()` is called to evaluate the complete Landau-Zener probability curve,

$$P_{LZ}^{(M_S)}(E),$$

over the common total-energy grid established earlier in Step 9. Since this routine was introduced in Section 4.9.6, the same physical procedure is reused here without modification; only the value of the spin-orbit coupling changes from one spin channel to another. Consequently, every M_S -resolved transition possesses its own unique probability profile whenever its spin-orbit coupling differs from the others.

The calculated probability array is then converted into the corresponding effective number of crossing states,

$$N_{LZ}^{(M_S)}(E),$$

through the RRKM convolution with the shifted transition-state density of states,

$$N_{LZ}^{(M_S)}(E) = \sum_j P_{LZ}^{(M_S)}(E_j) \rho_X(E - E_j) \Delta E,$$

after which the M_S -specific microcanonical Landau-Zener rate constant is obtained using

$$k_{LZ}^{(M_S)}(E) = \frac{g_R N_{LZ}^{(M_S)}(E)}{h \rho_R(E)}.$$

Thus, every spin channel is propagated independently through the complete nonadiabatic statistical framework, producing a distinct energy-dependent microcanonical rate constant whenever the associated spin-orbit coupling differs from the others.

The resulting probability, number-of-states, and rate arrays are stored separately for every transition channel:

<code>P_LZ_channels_rate_grid[ch_label] = P_ch</code>

```
N_LZ_channels[ch_label] = N_ch
k_LZ_channels[ch_label] = k_ch
```

The use of independent dictionaries preserves the complete identity of each spin-transition pathway throughout the calculation. Consequently, every M_S -resolved transition remains individually accessible for subsequent visualization, comparison, and analysis, while avoiding any premature averaging or combination of channels.

An analogous procedure is subsequently performed for the weak coupling formalism:

```
k_WC_channels = {}
N_WC_channels = {}
P_WC_channels_rate_grid = {}

if "P_WC_channels" in globals() and len(P_WC_channels) > 0:
    for ch_label, ch_data in P_WC_channels.items():
        H_abs = float(ch_data["H_abs_cm1"])

        if "compute_WC_for_HSO_cm" in globals():
            P_ch, _, _ = compute_WC_for_HSO_cm(H_abs, total_energy_grid_cm1)
        else:
            P_ch = np.asarray(ch_data["P"], dtype=float).reshape(-1)

        N_ch, k_ch = compute_weak_coupling_microcanonical_rate(P_ch)

        P_WC_channels_rate_grid[ch_label] = P_ch
        N_WC_channels[ch_label] = N_ch
        k_WC_channels[ch_label] = k_ch
```

Here, the channel-specific weak coupling probability is either recalculated directly from the spin-orbit coupling using `compute_WC_for_HSO_cm()` or, if this routine is unavailable, the probability array generated previously in Step 8 is reused after verifying consistency with the common energy grid. The probability is then transformed into the corresponding effective number of transition states,

$$N_{\text{WC}}^{(M_S)}(E),$$

followed by the weak coupling microcanonical rate constant,

$$k_{\text{WC}}^{(M_S)}(E) = \frac{g_R N_{\text{WC}}^{(M_S)}(E)}{h \rho_R(E)}.$$

The resulting arrays are stored independently for every spin-transition channel, thereby producing a complete set of M_S -resolved weak coupling microcanonical rate constants. As in the Landau-Zener treatment, each allowed transition is retained as an independent physical process, and no summation or symmetry reduction is applied during this stage of the calculation.

4.9.11 Construction of symmetry-collapsed M_S -specific rate curves

Although every M_S -specific spin-transition channel is evaluated independently in the previous subsection, many transitions possess identical absolute spin-orbit coupling magnitudes,

$$|H_{\text{SO},i}| = |H_{\text{SO},j}|,$$

which results in identical Landau-Zener or weak coupling probabilities and consequently identical microcanonical rate constants. Plotting every equivalent transition separately would therefore produce multiple completely overlapping curves that unnecessarily increase the complexity of the graphical output without providing additional physical information. To avoid this redundancy while preserving the identity of all symmetry-related transitions, SPINKS automatically groups equivalent channels and retains only a single representative rate curve for visualization. Importantly, this grouping is performed solely for presentation purposes; no probabilities, numbers of crossing states, or rate constants are summed or averaged. The term *symmetry-collapsed* therefore refers only to the removal of redundant, overlapping curves from plots and summary tables; the independently calculated M_S -specific probabilities and rate constants remain unchanged.

The corresponding implementation begins by constructing new dictionaries that store representative Landau-Zener rate curves for each symmetry-equivalent group:

```
# =====  
# Symmetry-collapsed MS-specific channel rates  
# =====  
  
section("SYMMETRY-COLLAPSED MS-SPECIFIC RATE CURVES")  
  
k_LZ_channel_groups = {}  
N_LZ_channel_groups = {}  
P_LZ_channel_groups_rate_grid = {}  
  
if "P_LZ_channel_groups" in globals() and len(P_LZ_channel_groups) > 0:  
    for group_key, group_data in P_LZ_channel_groups.items():
```

Each symmetry group generated previously in Step 8 contains the labels of all equivalent spin-transition channels together with a representative channel. If the predefined representative is unavailable, the code automatically selects the first valid channel belonging to the group. The corresponding probability, number-of-states, and microcanonical rate arrays are then copied from the previously calculated M_S -specific results:

```
k_rep = np.asarray(  
    k_LZ_channels[representative_label],  
    dtype=float  
) .copy()
```

```

N_rep = np.asarray(
    N_LZ_channels[representative_label],
    dtype=float
).copy()

P_rep = np.asarray(
    P_LZ_channels_rate_grid[representative_label],
    dtype=float
).copy()

```

Because all members of a symmetry group possess identical absolute spin–orbit couplings,

$$|H_{\text{SO},1}| = |H_{\text{SO},2}| = \cdots = |H_{\text{SO},n}|,$$

their transition probabilities satisfy

$$P_1(E) = P_2(E) = \cdots = P_n(E),$$

which immediately implies

$$N_1(E) = N_2(E) = \cdots = N_n(E),$$

and therefore

$$k_1(E) = k_2(E) = \cdots = k_n(E).$$

Consequently, retaining only one representative curve preserves the complete physical information contained within the group while eliminating redundant graphical duplication.

The representative curve together with the complete list of equivalent transition labels is stored as:

```

k_LZ_channel_groups[group_key] = {
    "H_abs_cm1": float(group_data["H_abs_cm1"]),
    "labels": labels,
    "channel_symbols": list(
        group_data.get("channel_symbols", [])
    ),
    "degeneracy": len(labels),
    "representative_label": representative_label,
    "k_representative": k_rep,
    "N_representative": N_rep,
    "P_representative": P_rep,
    "combination_rule": "representative_only_no_sum"
}

```

The field:

```
combination_rule = "representative_only_no_sum"
```

explicitly documents that the representative curve is retained only for visualization and reporting. No summation, averaging, weighting, or degeneracy scaling is performed at any stage of the calculation. Every original M_S -specific channel remains available through its individual arrays generated in Section 4.9.10.

An identical procedure is subsequently applied to the weak coupling treatment, where representative weak coupling probability, number-of-states, and microcanonical rate arrays are constructed for every symmetry-equivalent group using the corresponding M_S -specific weak coupling channels. Thus, both the Landau-Zener and weak coupling formalisms employ the same symmetry-collapsing strategy while preserving complete channel-specific information for subsequent analysis.

Finally, the total numbers of representative Landau-Zener and weak coupling curves are reported together with the grouping rule:

```
print(f"Representative LZ MS curves      : {len(k_LZ_channel_groups)}")
print(f"Representative WC MS curves      : {len(k_WC_channel_groups)}")
print("MS-specific combination rule      : representative only; no summation")
```

This diagnostic confirms that SPINKS performs only a graphical reduction of symmetry-equivalent M_S -specific channels. The underlying microcanonical rate constants remain unchanged, and every individual spin-transition pathway calculated in Section 4.9.10 is preserved for quantitative analysis and export.

4.9.12 Generation of final rate arrays for canonical calculations

Following the evaluation of the effective, intermediate, and M_S -specific microcanonical rate constants, Step 9 constructs the standardized output arrays that serve as the direct interface between the microcanonical and canonical stages of the SPINKS workflow. This stage reorganizes the previously calculated quantities into a consistent set of one-dimensional arrays, removes the zero-energy bin required internally by the nonadiabatic statistical formalism, generates logarithmic representations of the microcanonical rate constants, and assigns backward-compatible variable names for subsequent calculations. No additional statistical or quantum-mechanical approximations are introduced during this stage; the procedure simply prepares the complete microcanonical data set for the canonical Boltzmann averaging performed in Step 10.

The implementation begins by extracting the physical energy grid together with the corresponding rovibrational densities of states:

```
# =====
# Final arrays and backward-compatible names
# =====

section("FINAL RATE ARRAYS")
```

```

E_rate_LZ_cm1 = total_energy_grid_cm1[1:]
E_rate_WC_cm1 = total_energy_grid_cm1[1:]

rho_reactant_cm = reactant_state_density[1:]
rho_MECP_internal_cm = crossing_internal_state_density[1:]
rho_MECP_shifted_plot = crossing_shifted_state_density[1:]

```

Since the first common energy bin corresponds to zero energy and is used only for the internal recursive construction of the densities of states, it is omitted from all exported arrays. Consequently, the microcanonical quantities distributed to the remaining workflow correspond exclusively to physically accessible energies above the reference minimum. The arrays $\rho_R(E)$, $\rho_{TP}(E)$, and $\rho_X(E)$ therefore constitute the final reactant, internal transition-state, and shifted transition-state rovibrational densities of states employed in all subsequent canonical calculations.

The effective Landau-Zener probability, adiabaticity parameter, reaction velocity, number of crossing states, and microcanonical rate constants are then assembled into standardized arrays:

```

P_LZ_rate_grid = effective_LZ_probability_for_rates[1:]
gamma_LZ_rate_grid = landau_zener_exponent_eff_rate[1:]
effective_velocity_bohr_per_s_grid = velocity_bohr_per_s_for_rates[1:]

N_MECP_LZ = effective_LZ_cumulative_states[1:]
k_LZ_micro_s_inv = effective_LZ_rate[1:]

k_LZ_effective_micro_s_inv = k_LZ_micro_s_inv
N_MECP_LZ_effective = N_MECP_LZ

```

The corresponding weak coupling quantities are prepared in an analogous manner:

```

P_WC_rate_grid = effective_WC_probability_for_rates[1:]
N_MECP_WC = effective_WC_cumulative_states[1:]
k_WC_micro_s_inv = effective_WC_rate[1:]

k_WC_effective_micro_s_inv = k_WC_micro_s_inv
N_MECP_WC_effective = N_MECP_WC

```

These standardized variable names provide a consistent interface for the canonical kinetic calculations implemented in Step 10 while maintaining compatibility with previous versions of the SPINKS workflow.

For visualization purposes, logarithmic representations of the effective microcanonical rate constants are also generated. Because the logarithm is defined only for positive values, the code first identifies all positive rate constants:

```

positive_k = k_LZ_micro_s_inv > 0.0
log_k_LZ_raw = np.full_like(k_LZ_micro_s_inv, np.nan)
log_k_LZ_raw[positive_k] = np.log10(k_LZ_micro_s_inv[positive_k])

```

with an identical procedure applied to the weak coupling rates. Assigning NaN to zero-valued rate constants prevents undefined logarithmic values from appearing in subsequent plots while preserving the correct physical behavior below the spin-crossing threshold.

Finally, aliases are assigned to the rovibrational densities of states and effective microcanonical quantities:

```

reactant_state_density_independent = reactant_state_density
crossing_shifted_state_density_independent = crossing_shifted_state_density
crossing_internal_state_density_independent = crossing_internal_state_density

LZ_cumulative_states_independent = effective_LZ_cumulative_states
LZ_rate_independent = effective_LZ_rate

WC_cumulative_states_independent = effective_WC_cumulative_states
WC_rate_independent = effective_WC_rate

```

thereby preserving compatibility with routines that expect the retired compatibility variable names while simultaneously providing the standardized SPINKS nomenclature adopted throughout the remainder of the workflow. Diagnostic information describing the energy range and maximum effective Landau-Zener and weak coupling microcanonical rate constants is then reported to verify successful completion of the array construction stage before the generation of graphical and text-based outputs.

4.9.13 Summary of the step 9 algorithm

The complete microcanonical-rate evaluation begins with the ZPE-corrected MECP threshold, the reactant and MECP rovibrational properties, and the effective, intermediate, and M_S -specific spin-crossing probabilities obtained in Step 8. The reactant and MECP harmonic vibrational frequencies and rigid-rotor moments of inertia are first converted into rovibrational densities of states. The MECP density of states is subsequently shifted onto the common total-energy grid according to the active ZPE-corrected crossing energy,

$$\rho_X(E) = \begin{cases} 0, & E < E_{\text{MECP}} \\ \rho_{TP}(E - E_{\text{MECP}}), & E \geq E_{\text{MECP}}, \end{cases}$$

where $\rho_R(E)$, $\rho_{TP}(E)$, and $\rho_X(E)$ denote the reactant, internal MECP, and shifted MECP rovibrational densities of states, respectively.

The effective Landau-Zener and weak coupling probabilities are then transformed into the corresponding effective numbers of crossing states,

$$N_{\text{LZ}}(E) = \sum_j P_{\text{LZ}}(E_j) \rho_X(E - E_j) \Delta E,$$

$$N_{WC}(E) = \sum_j P_{WC}(E_j) \rho_{TP}(E - E_j) \Delta E,$$

followed by evaluation of the effective microcanonical rate constants,

$$k_{LZ}(E) = \frac{g_R N_{LZ}(E)}{h \rho_R(E)},$$

$$k_{WC}(E) = \frac{g_R N_{WC}(E)}{h \rho_R(E)}.$$

The same statistical procedure is subsequently repeated for every intermediate spin-orbit coupling generated in Step 8,

$$P^{(i)}(E) \rightarrow N^{(i)}(E) \rightarrow k^{(i)}(E),$$

thereby producing a complete hierarchy of intermediate Landau-Zener and weak coupling microcanonical rate constants.

The algorithm is then applied independently to every M_S -resolved spin-crossing channel,

$$P^{(M_S)}(E) \rightarrow N^{(M_S)}(E) \rightarrow k^{(M_S)}(E),$$

using the corresponding channel-specific absolute spin-orbit coupling magnitude. Each spin-transition pathway therefore produces its own energy-dependent microcanonical rate constant without averaging or summation.

Channels possessing identical absolute spin-orbit couplings,

$$|H_{SO,1}| = |H_{SO,2}| = \dots,$$

are subsequently grouped into representative symmetry-equivalent families. Because these channels generate identical probabilities and therefore identical microcanonical rate constants,

$$k_1(E) = k_2(E) = \dots,$$

only one representative curve is retained for graphical visualization while preserving the complete list of equivalent transition labels. Importantly, no probabilities, numbers of crossing states, or microcanonical rate constants are summed during this procedure.

The corresponding computational sequence is:

Reactant frequencies
MECP frequencies
Rotational constants
ZPE-corrected MECP energy



SPINKS rovibrational
densities of states
(ρ_R , ρ_{TP} , ρ_X)



Effective probabilities
(PLZ, PWC)



Effective microcanonical
rate constants
(k_{LZeff} , k_{WCeff})



Intermediate
microcanonical rates
(k_{int})



MS-specific
microcanonical rates
(k_{MS})



Symmetry-collapsed
representative MS curves



Final energy grids,
rate arrays,
diagnostic plots,
text summaries,
and exported variables



Step 10
Canonical rate calculations

Therefore, Step 9 transforms the effective, intermediate, and M_S -resolved spin-crossing probabilities generated in Step 8 into the corresponding hierarchy of Landau-Zener and weak coupling microcanonical reaction rate constants using the nonadiabatic statistical/RRKM formalism. The resulting energy grids, rovibrational densities of states, effective numbers of crossing states, microcanonical rate constants, representative M_S -specific rate curves, graphical outputs, and formatted numerical arrays constitute the complete microcanonical kinetic description and provide the direct input for the canonical Boltzmann averaging performed in Step 10.

4.10 Step 10: Canonical Rate Constants

Following the construction of the effective and M_S -resolved microcanonical rate constants in Step 9, Step 10 converts these energy-dependent quantities into canonical reaction rate constants through Boltzmann averaging over the reactant density of states. This transformation enables direct comparison with experimentally measured temperature-dependent rate constants while maintaining the same Landau-Zener and weak coupling formalisms developed in the preceding step.

The canonical averaging procedure is performed separately for the effective Landau-Zener and weak coupling rate constants, the intermediate weak coupling channels associated with each unique $|M_S|$, and the representative M_S -specific channels retained after symmetry reduction. In every case, the microcanonical rate constants are weighted by the reactant density of states and the Boltzmann factor before normalization by the canonical reactant partition function.

Mathematically, the canonical rate constant is obtained from the corresponding microcanonical rate constant according to

$$k(T) = \frac{\int k(E) \rho_R(E) e^{-E/(k_B T)} dE}{\int \rho_R(E) e^{-E/(k_B T)} dE},$$

where $k(E)$ is the microcanonical rate constant, $\rho_R(E)$ is the reactant density of states, k_B is the Boltzmann constant, and T is the absolute temperature. Within SPINKS, these integrals are evaluated numerically on the common energy grid established in Step 9 using the discrete energy spacing ΔE .

In addition to computing the effective canonical Landau-Zener and weak coupling rate constants, Step 10 also evaluates canonical rate constants for every intermediate and representative M_S -specific spin-crossing channel. The resulting temperature-dependent rate constants are exported as publication-quality numerical tables and graphical plots, providing a complete canonical description of the spin-forbidden reaction kinetics over the user-defined temperature range.

4.10.1 Import of step 9 microcanonical quantities

Step 10 begins by importing the microcanonical quantities generated during Step 9. These quantities constitute the complete microcanonical description of the spin-forbidden reaction and provide the direct input for canonical Boltzmann averaging. Before any calculations are performed, the program verifies that all required variables are present to ensure that the

preceding microcanonical workflow has been completed successfully and that all energy grids, densities of states, and microcanonical rate constants are mutually consistent.

The corresponding implementation first defines the complete list of variables required for the canonical calculations:

```
required_vars_step10 = [
    "jobname",
    "E_rate_LZ_cm1",
    "E_rate_WC_cm1",
    "rho_reactant_cm",
    "k_LZ_effective_micro_s_inv",
    "k_WC_effective_micro_s_inv",
    "local_base",
    "remote_base"
]

for var in required_vars_step10:
    if var not in globals():
        raise RuntimeError(
            f"Required variable '{var}' is missing. Run cluster Step 9 first."
        )
```

The imported variables consist of the common energy grids,

$$E_{\text{LZ}} \text{ and } E_{\text{WC}},$$

the reactant rovibrational density of states,

$$\rho_R(E),$$

and the effective Landau-Zener and weak coupling microcanonical rate constants,

$$k_{\text{LZ}}(E) \text{ and } k_{\text{WC}}(E),$$

which were obtained during Step 9 through the RRKM statistical treatment of the effective spin-crossing probabilities. In addition, the local and remote project directories are imported so that all canonical plots and numerical output files generated during this stage can be stored in the appropriate workflow directories.

The program then determines the execution mode of the workflow and evaluates whether only the effective spin-orbit coupling should be considered:

```
RUN_MODE = globals().get("RUN_MODE", "CLUSTER")
workflow_mode = "MECP_ONLY"

SOC_EFFECTIVE_ONLY = bool(
```

```

globals().get("SOC_EFFECTIVE_ONLY", False)
)

SOC_MATRIX_CHANNELS_AVAILABLE = bool(
    globals().get("SOC_MATRIX_CHANNELS_AVAILABLE", True)
)

if SOC_EFFECTIVE_ONLY or not SOC_MATRIX_CHANNELS_AVAILABLE:
    print("\nEffective-only SOC mode active.")
    print("Only effective LZ/WC rates will be computed/plotted.")

```

When an effective spin–orbit coupling is available but individual M_S -resolved channels cannot be constructed, the subsequent canonical calculations are restricted to the effective Landau-Zener and weak coupling rate constants. Otherwise, the workflow proceeds with the complete hierarchy of effective, intermediate, and M_S -specific canonical rate calculations established in the preceding microcanonical stage.

To facilitate the remaining calculations, Step 10 also defines a collection of helper routines responsible for section formatting, validated numerical input, figure generation, and construction of mathematically formatted labels for M_S -specific transition channels:

```

def section(title):
    ...

def ask_float(...):
    ...

def ask_int(...):
    ...

def save_current_figure(...):
    ...

def ms_tex_value(...):
    ...

def make_canonical_group_legend(...):
    ...

```

These utility functions provide standardized handling of user input, automate the generation of publication-quality figures, and produce consistent mathematical notation for the representative M_S -specific canonical rate curves plotted later in the workflow. In particular, the function `make_canonical_group_legend()` automatically identifies symmetry-equivalent spin-transition channels possessing identical absolute spin–orbit couplings and generates concise figure legends while explicitly indicating that the displayed curves represent coincident channels rather than summed contributions.

4.10.2 Construction of the canonical temperature grid

Unlike the microcanonical calculations of Step 9, which evaluate reaction rates as functions of total internal energy, the canonical treatment requires averaging over a thermal ensemble of reactant states at a specified temperature. Step 10 therefore begins by constructing a user-defined temperature grid over which the canonical Landau-Zener and weak coupling rate constants will be evaluated. The canonical averaging follows the standard SPINKS expression,

$$k(T) = \frac{\sum k(E) \rho_R(E) e^{-E/(k_B T)} \Delta E}{\sum \rho_R(E) e^{-E/(k_B T)} \Delta E},$$

where $k(E)$ denotes the microcanonical reaction rate constant obtained in Step 9, $\rho_R(E)$ is the reactant rovibrational density of states, k_B is the Boltzmann constant, and T is the absolute temperature.

The implementation begins by defining the Boltzmann constant expressed in wavenumber units:

```
kB_cm1_per_K = 0.695034800
```

which allows the Boltzmann factor,

$$e^{-E/(k_B T)},$$

to be evaluated directly using the common energy grid expressed in cm^{-1} . The user is then prompted to specify the temperature interval and the number of temperature points:

```
T_min = ask_float(  
    "Minimum temperature for canonical rates..."  
)  
  
T_max = ask_float(  
    "Maximum temperature for canonical rates..."  
)  
  
nT = ask_int(  
    "Number of temperature points..."  
)
```

The lower and upper temperatures define the range over which the canonical rate constants will be computed, while nT specifies the number of uniformly spaced temperature values. A consistency check is subsequently performed to ensure that the upper temperature exceeds the lower temperature:

```
if T_max <= T_min:  
    raise RuntimeError(  
        "Invalid temperature range..."  
    )
```

The complete temperature grid is then generated as:

```
T_grid_K = np.linspace(
    T_min,
    T_max,
    nT
)
```

thereby producing

$$T_i = T_{\min} + (i - 1) \frac{T_{\max} - T_{\min}}{n_T - 1}, i = 1, \dots, n_T,$$

which forms the common temperature grid used throughout all subsequent canonical calculations.

The microcanonical quantities imported from Step 9 are then converted into one-dimensional NumPy arrays to ensure numerical consistency during the canonical averaging procedure:

```
E_rate_LZ_cm1 = np.asarray(...)
E_rate_WC_cm1 = np.asarray(...)
rho_reactant_cm = np.asarray(...)

k_LZ_effective_micro_s_inv = np.asarray(...)
k_WC_effective_micro_s_inv = np.asarray(...)
```

Additional consistency checks verify that the energy grids, reactant density of states, and effective microcanonical rate constants possess identical dimensions before canonical averaging proceeds. Finally, the average energy-grid spacing:

```
dE_LZ = float(
    np.mean(np.diff(E_rate_LZ_cm1))
)

dE_WC = float(
    np.mean(np.diff(E_rate_WC_cm1))
)
```

is computed for both the Landau-Zener and weak coupling energy grids. These quantities correspond to the integration step,

$$\Delta E,$$

used in the discrete Boltzmann summations performed during canonical averaging. Diagnostic information summarizing the temperature interval, number of temperature points, energy-grid limits, and energy spacing is then printed to verify that the canonical calculations will be performed over the intended thermodynamic range.

4.10.3 Canonical Boltzmann averaging

The central objective of Step 10 is to transform the energy-dependent microcanonical reaction rate constants obtained in Step 9 into temperature-dependent canonical rate constants. Unlike the microcanonical treatment, where the reaction rate is evaluated at a fixed total energy, the canonical formalism assumes that the reactant population is distributed among all accessible rovibrational states according to the Boltzmann distribution. Consequently, the observable reaction rate at a given temperature is obtained as the Boltzmann-weighted average of the microcanonical rate constants over the entire reactant energy manifold.

The corresponding implementation is performed through the helper routine:

```
def boltzmann_average_microcanonical_rate(
    energy_cm1,
    state_density,
    microcanonical_rate,
    temperature_grid_K,
    energy_spacing_cm1
):
    energy_cm1 = np.asarray(energy_cm1, dtype=float).reshape(-1)
    state_density = np.asarray(state_density, dtype=float).reshape(-1)
    microcanonical_rate = np.asarray(
        microcanonical_rate,
        dtype=float
    ).reshape(-1)
    temperature_grid_K = np.asarray(
        temperature_grid_K,
        dtype=float
    ).reshape(-1)

    if (
        energy_cm1.shape != state_density.shape
        or energy_cm1.shape != microcanonical_rate.shape
    ):
        raise RuntimeError(
            f"Energy, state-density, and microcanonical-rate arrays "
            f" $\rho_R(E)$  "must have the same shape."
            f" $k(E)$  "
            f"
            f"thermal_energy_cm1 = kB_cm1_per_K * temperature_grid_K  $\Delta E$ "
        )
```

```
canonical_rate = np.zeros_like(temperature_grid_K, dtype=float)
```

```
normalization_integral = np.zeros_like(
    temperature_grid_K,
    dtype=float
```

```
)
```

$$w(E, T) = \exp \left(- \frac{E_i}{k_B T} \right),$$

```
for temperature_index, thermal_energy in enumerate(k_B
```

```
    thermal_energy_cm1
```

```
):
```

```
    boltzmann_weight = np.exp(-energy_cm1 / thermal_energy)
```

```
    denominator = np.sum(
        state_density * boltzmann_weight
```

```
    ) * energy_spacing_cm1
```

$$Q_R(T) = \sum_i \rho_R(E_i) \exp \left(- \frac{E_i}{k_B T} \right) \Delta E.$$

```
    numerator = np.sum(
```

```
        microcanonical_rate Q_R(T)
```

```
        * state_density
```

```
        * boltzmann_weight
    ) * energy_spacing_cm1
```

```
    normalization_integral[temperature_index] = denominator
```

```
if denominator > 0.0 and np.isfinite(denominator):
```

$$N(T) = \sum_i k(E_i) \rho_R(E_i) \exp \left(- \frac{E_i}{k_B T} \right) \Delta E.$$

```
    canonical_rate[temperature_index] = numerator / denominator
```

```
else:
```

```
1.     canonical_rate[temperature_index] = 0.0 k(E)
```

```
2.     ρ_R(E)
```

```
3.     canonical_rate[~np.isfinite(canonical_rate)] = 0.0
```

```
canonical_rate = np.clip(canonical_rate, 0.0, None)
```

```
return canonical_rate, normalization_integral
```

$$k(T) = \frac{\sum_i k(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}.$$

Any undefined numerical values arising from divisions by extremely small partition functions are replaced with zero, while negative values resulting from floating-point roundoff are clipped to zero. The routine therefore returns two quantities for every temperature considered:

$$k(T),$$

the canonical reaction rate constant, and

$$Q_R(T),$$

the corresponding reactant rovibrational partition function. These quantities constitute the fundamental thermodynamic outputs required for the effective, intermediate, and M_5 -specific canonical Landau-Zener and weak coupling calculations described in the following subsections.

4.10.4 Evaluation of the effective canonical Landau-Zener and weak coupling rate constants

The first application of the canonical averaging procedure is the calculation of the effective canonical Landau-Zener and weak coupling reaction rate constants. These quantities represent the thermal averages of the effective microcanonical rate constants obtained in Step 9 and therefore provide the primary temperature-dependent kinetic predictions of the SPINKS workflow. Since the effective spin-orbit coupling already incorporates the complete electronic

coupling information for the spin-crossing event, these canonical rate constants constitute the principal theoretical predictions for comparison with experiment or other kinetic models.

The corresponding implementation applies the canonical averaging routine independently to the effective Landau-Zener and weak coupling microcanonical rate constants:

```
# =====
# Effective canonical rates
# =====

section("EFFECTIVE CANONICAL LZ/WC RATES")

k_LZ_canonical_s_inv, LZ_canonical_normalization = boltzmann_average_microcanonical_rate(
    E_rate_LZ_cm1,
    rho_reactant_cm,
    k_LZ_effective_micro_s_inv,
    T_grid_K,
    dE_LZ
)

k_WC_canonical_s_inv, WC_canonical_normalization =
boltzmann_average_microcanonical_rate(
    E_rate_WC_cm1,
    rho_reactant_cm,
    k_WC_effective_micro_s_inv,
    T_grid_K,
    dE_WC
)
```

For the Landau-Zener treatment, the microcanonical rate constant,

$$k_{\text{LZ}}(E),$$

calculated in Step 9 is transformed into the canonical rate constant,

$$k_{\text{LZ}}(T),$$

according to

$$k_{\text{LZ}}(T) = \frac{\sum_i k_{\text{LZ}}(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}.$$

Similarly, the weak coupling microcanonical rate,

$$k_{\text{WC}}(E),$$

is converted into

$$k_{\text{WC}}(T) = \frac{\sum_i k_{\text{WC}}(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}.$$

Thus, both canonical rate constants are obtained using the same Boltzmann averaging procedure described in Section 4.10.3, differing only in the underlying microcanonical reaction rates.

The calculated canonical rate arrays are subsequently assigned to standardized variable names:

```
kcanon_LZ = k_LZ_canonical_s_inv  
kcanon_WC = k_WC_canonical_s_inv
```

thereby preserving compatibility with subsequent workflow stages while clearly identifying these quantities as the effective canonical Landau-Zener and weak coupling reaction rate constants. The corresponding reactant partition functions,

$$Q_R^{\text{LZ}}(T) \text{ and } Q_R^{\text{WC}}(T),$$

are retained as additional outputs of the canonical averaging routine for diagnostic purposes. Since the reactant rovibrational density of states is identical for both theories, these partition functions are expected to be numerically equivalent apart from insignificant floating-point differences.

Finally, the minimum and maximum values of both canonical rate curves are reported:

```
print(  
    f"k_LZ canonical range : "  
    f"{np.nanmin(k_LZ_canonical_s_inv):.6e} "  
    f"to "  
    f"{np.nanmax(k_LZ_canonical_s_inv):.6e} s^-1"  
)  
  
print(  
    f"k_WC canonical range : "  
    f"{np.nanmin(k_WC_canonical_s_inv):.6e} "  
    f"to "  
    f"{np.nanmax(k_WC_canonical_s_inv):.6e} s^-1"  
)
```

These diagnostic quantities verify successful completion of the canonical averaging procedure and provide a concise summary of the temperature dependence of the effective Landau-Zener and weak coupling rate constants before the more detailed intermediate and M_S -specific calculations are performed.

4.10.5 Evaluation of the intermediate canonical Landau-Zener and weak coupling rate constants

Following the evaluation of the effective canonical rate constants, Step 10 applies the same canonical averaging procedure to every intermediate spin–orbit coupling introduced in Step 8 and propagated through the microcanonical calculations of Step 9. These intermediate canonical rate constants provide a continuous kinetic description connecting the lowest spin-projection coupling to the effective spin–orbit coupling and enable direct analysis of the dependence of canonical reaction kinetics on the magnitude of the electronic coupling. Each intermediate spin–orbit coupling is treated independently, and no averaging or interpolation between different intermediate channels is performed.

The implementation begins by allocating dictionaries that will store the intermediate canonical rate constants:

```
# =====  
# Intermediate canonical rates  
# =====  
  
section("CANONICAL INTERMEDIATE RATES")  
  
k_LZ_intermediate_canonical = {}  
k_WC_intermediate_canonical = {}
```

For every intermediate Landau-Zener spin–orbit coupling, the corresponding microcanonical rate constant generated in Step 9 is extracted and canonically averaged:

```
for key, data in k_LZ_intermediate.items():  
  
    k_E = np.asarray(  
        data["k"][1:],  
        dtype=float  
    ).reshape(-1)  
  
    k_T, _ = boltzmann_average_microcanonical_rate(  
        E_rate_LZ_cm1,  
        rho_reactant_cm,  
        k_E,  
        T_grid_K,  
        dE_LZ  
    )
```

The resulting canonical rate constant is therefore

$$k_{\text{LZ}}^{(i)}(T) = \frac{\sum_j k_{\text{LZ}}^{(i)}(E_j) \rho_R(E_j) e^{-E_j/(k_B T) \Delta E}}{\sum_j \rho_R(E_j) e^{-E_j/(k_B T) \Delta E}},$$

where the superscript i denotes a particular intermediate spin-orbit coupling generated in Step 8. Since each intermediate coupling possesses its own microcanonical rate curve, every intermediate channel produces an independent canonical reaction-rate curve spanning the complete user-defined temperature interval.

Each intermediate canonical Landau-Zener rate is stored together with the corresponding physical quantities:

```
k_LZ_intermediate_canonical[key] = {  
    "H_int_cm1": data["H_int_cm1"],  
    "abs_Ms_low": data["abs_Ms_low"],  
    "degeneracy": data["degeneracy"],  
    "k_T": k_T  
}
```

Thus, every canonical rate curve remains explicitly associated with:

- the intermediate spin-orbit coupling magnitude H_{int} ,
- the corresponding lower-spin projection,
- the associated degeneracy,
- and the complete temperature-dependent canonical rate constant.

Retaining these quantities together preserves the physical identity of each intermediate spin-crossing pathway throughout the remainder of the workflow.

An entirely analogous procedure is subsequently performed for the weak coupling formalism:

```
for key, data in k_WC_intermediate.items():  
  
    k_E = np.asarray(  
        data["k"][1:],  
        dtype=float  
    ).reshape(-1)  
  
    k_T, _ = boltzmann_average_microcanonical_rate(  
        E_rate_WC_cm1,  
        rho_reactant_cm,  
        k_E,  
        T_grid_K,  
        dE_WC  
    )  
  
    k_WC_intermediate_canonical[key] = {  
        "H_int_cm1": data["H_int_cm1"],  
        "abs_Ms_low": data["abs_Ms_low"],  
        "degeneracy": data["degeneracy"],  
        "k_T": k_T  
    }
```

The weak coupling intermediate canonical rate constants,

$$k_{\text{WC}}^{(i)}(T),$$

are therefore obtained using exactly the same canonical Boltzmann averaging expression, differing only in the underlying weak coupling microcanonical rate constants calculated in Step 9. Consequently, both the Landau-Zener and weak coupling theories produce complete families of intermediate canonical rate curves that can be compared directly over the entire temperature range.

Upon completion of the averaging procedure, the total numbers of intermediate canonical Landau-Zener and weak coupling rate curves are reported:

```
print(
    f"LZ intermediate canonical curves : "
    f"{len(k_LZ_intermediate_canonical)}"
)

print(
    f"WC intermediate canonical curves : "
    f"{len(k_WC_intermediate_canonical)}"
)
```

These diagnostic quantities confirm that every intermediate spin-orbit coupling generated during the preceding stages of the workflow has been successfully propagated through the canonical Boltzmann averaging procedure before the subsequent evaluation of the symmetry-collapsed M_S -specific canonical rate constants.

4.10.6 Evaluation of the symmetry-collapsed M_S -specific canonical rate constants

The final canonical kinetic calculations extend the Boltzmann averaging procedure to the symmetry-collapsed M_S -specific spin-crossing channels constructed in Step 9. As described previously, Step 9 identifies sets of spin-transition channels possessing identical absolute spin-orbit coupling magnitudes and retains a single representative microcanonical rate curve for each symmetry-equivalent group. Step 10 canonically averages these representative microcanonical rate curves directly, thereby producing a corresponding set of representative M_S -specific canonical rate constants. Because all channels within a symmetry group possess identical spin-orbit couplings, they also possess identical microcanonical and canonical reaction rate constants. Consequently, no summation or averaging of channel rates is required or performed during this stage.

The implementation begins by allocating dictionaries that will contain the representative canonical Landau-Zener and weak coupling rate curves:

```
# =====
# Symmetry-collapsed MS-specific canonical rates
# =====

section("CANONICAL SYMMETRY-COLLAPSED MS-SPECIFIC RATES")

k_LZ_channel_groups_canonical = {}
k_WC_channel_groups_canonical = {}
```

For every representative Landau-Zener channel group generated in Step 9, the representative microcanonical rate curve is extracted:

```
k_group_E = np.asarray(
    data["k_representative"][1:],
    dtype=float
).reshape(-1)
```

and converted into its canonical counterpart through the canonical averaging routine:

```
k_group_T, _ = boltzmann_average_microcanonical_rate(
    E_rate_LZ_cm1,
    rho_reactant_cm,
    k_group_E,
    T_grid_K,
    dE_LZ
)
```

This calculation evaluates

$$k_{\text{LZ}}^{(\text{group})}(T) = \frac{\sum_i k_{\text{LZ}}^{(\text{group})}(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E},$$

where

$$k_{\text{LZ}}^{(\text{group})}(E)$$

denotes the representative microcanonical rate constant for one symmetry-equivalent family of M_S -specific spin-transition channels. Since every member of the group possesses the same absolute spin-orbit coupling magnitude,

$$|H_{\text{SO},1}| = |H_{\text{SO},2}| = \cdots = |H_{\text{SO},n}|,$$

all channels within the group generate identical canonical rate constants,

$$k_1(T) = k_2(T) = \cdots = k_n(T).$$

Therefore, canonical averaging of a single representative microcanonical curve completely describes the thermal kinetics of the entire symmetry-equivalent group.

The resulting representative canonical rate curve is then stored together with the associated physical information describing the symmetry group:

```
k_LZ_channel_groups_canonical[key] = {
    "H_abs_cm1": float(data["H_abs_cm1"]),
    "degeneracy": int(data.get("degeneracy", 1)),
    "channel_symbols": list(
        data.get("channel_symbols", [])
    ),
    "labels": list(data.get("labels", [])),
    "representative_label": data.get(
        "representative_label"
    ),
    "combination_rule": "representative_only_no_sum",
    "k_T": k_group_T
}
```

In addition to the canonical rate constant, each representative group retains the absolute spin-orbit coupling magnitude, the number of equivalent channels, the corresponding transition symbols, the labels of all equivalent spin-transition pathways, and the identity of the representative channel selected during Step 9. The explicit assignment:

```
combination_rule =
"representative_only_no_sum"
```

documents that the representative canonical curve is retained solely for visualization and reporting purposes. At no point are canonical rate constants summed, averaged, weighted, or multiplied by the degeneracy of the symmetry group. Every original M_S -specific channel remains individually available through the microcanonical results generated in Step 9.

An identical procedure is subsequently applied to the weak coupling treatment. For every representative weak coupling symmetry group, the representative microcanonical rate curve is canonically averaged:

```
k_group_T, _ = boltzmann_average_microcanonical_rate(
    E_rate_WC_cm1,
    rho_reactant_cm,
    k_group_E,
    T_grid_K,
    dE_WC
)
```

and stored in the corresponding representative dictionary:

```
k_WC_channel_groups_canonical[key] = {
    ...
    "combination_rule":
    "representative_only_no_sum",
    ...
}
```

```
"k_T": k_group_T
}
```

Consequently, both the Landau-Zener and weak coupling formalisms employ the same symmetry-collapsing strategy established in Step 9, thereby preserving complete consistency between the microcanonical and canonical treatments of symmetry-equivalent spin-transition channels.

Finally, the program reports the total numbers of representative canonical rate curves generated for each theoretical treatment:

```
print(
    f"Representative LZ MS-specific canonical curves : "
    f"{len(k_LZ_channel_groups_canonical)}"
)

print(
    f"Representative WC MS-specific canonical curves : "
    f"{len(k_WC_channel_groups_canonical)}"
)

print(
    "MS-specific canonical combination rule : "
    "no summation"
)
```

These diagnostic messages confirm that the canonical workflow preserves the same symmetry-grouping philosophy introduced in Step 9. The reported representative curves are intended only to eliminate redundant overlapping plots arising from symmetry-equivalent spin-transition channels. The underlying canonical reaction kinetics remain completely unchanged, and no physical contributions are combined during this stage of the calculation.

4.10.7 Generation of canonical rate plots

Following the calculation of the effective, intermediate, and symmetry-collapsed M_S -specific canonical reaction rate constants, Step 10 automatically generates publication-quality graphical representations of the complete canonical kinetic results. These figures provide direct visualization of the temperature dependence of the canonical Landau-Zener and weak coupling rate constants and facilitate comparison among the effective, intermediate, and M_S -specific spin-crossing pathways. Two complementary graphical representations are produced: a temperature-dependent canonical rate plot and an Arrhenius-style representation plotted as a function of reciprocal temperature.

The program first creates a dedicated output directory for the canonical calculations and registers the filenames that will be used to store the generated figures:

```
local_step10 = os.path.join(
    local_base,
    "Canonical_rates"
```

```
)
os.makedirs(local_step10, exist_ok=True)

def register_plot(name):
    ...
```

This directory contains all graphical and numerical outputs associated with the canonical stage of the workflow, thereby maintaining the organized directory structure established throughout SPINKS. If the cluster version is being used, the corresponding remote directory is also created automatically before any files are transferred.

The first figure presents the canonical reaction rate constants directly as functions of temperature:

```
plt.figure(figsize=(9.2,5.2))

plt.plot(
    T_grid_K,
    k_LZ_canonical_s_inv,
    ...
)

plt.plot(
    T_grid_K,
    k_WC_canonical_s_inv,
    ...
)
```

The effective Landau-Zener and weak coupling canonical rate constants are displayed together with all intermediate canonical rate curves and the representative M_S -specific canonical rate constants. Intermediate curves are distinguished by their associated intermediate spin-orbit couplings, whereas the representative M_S -specific curves are labeled according to the corresponding spin-transition symbols generated automatically by the routine `make_canonical_group_legend()`. Because symmetry-equivalent spin-transition channels possess identical canonical rate constants, only one representative curve is displayed for each symmetry group, thereby eliminating redundant overlapping curves without altering the underlying kinetics.

The resulting figure therefore provides a complete comparison of

$$k_{\text{LZ}}^{\text{eff}}(T), k_{\text{WC}}^{\text{eff}}(T),$$

the intermediate canonical rate constants,

$$k_{\text{LZ}}^{(i)}(T), k_{\text{WC}}^{(i)}(T),$$

and the representative M_S -specific canonical rate constants,

$$k_{\text{LZ}}^{(M_S)}(T), k_{\text{WC}}^{(M_S)}(T),$$

over the entire user-defined temperature range.

A second figure is subsequently generated in Arrhenius form:

```
plt.figure(figsize=(9.2,5.2))

plt.plot(
    1000.0/T_grid_K,
    np.log10(
        k_LZ_canonical_s_inv
    ),
    ...
)

plt.plot(
    1000.0/T_grid_K,
    np.log10(
        k_WC_canonical_s_inv
    ),
    ...
)
```

The horizontal axis is expressed as

$$\frac{1000}{T},$$

while the vertical axis represents

$$\log_{10}[k(T)].$$

Only positive canonical rate constants are included in the logarithmic representation in order to avoid undefined logarithmic values. This Arrhenius-style plot provides a convenient visualization of the temperature dependence of the canonical reaction kinetics and facilitates comparison among the effective, intermediate, and M_S -specific reaction pathways over a broad temperature range.

Together, these two figures provide complementary views of the canonical kinetic behavior predicted by SPINKS. The temperature-dependent plot illustrates the absolute variation of the canonical reaction rates with temperature, whereas the Arrhenius-style representation

emphasizes their logarithmic temperature dependence and permits straightforward comparison with conventional kinetic analyses.

4.10.8 Export of canonical results and output files

After completion of the canonical calculations and graphical analysis, Step 10 exports the complete set of numerical results to a series of formatted output files and updates the global workspace with the quantities required by the subsequent stages of the workflow. This final stage preserves all canonical kinetic data in both human-readable and machine-readable formats while ensuring compatibility with the remaining SPINKS modules.

The program first defines the filenames for the principal numerical outputs:

```
canonical_rate_file = ...  
summary_file_step10 = ...  
intermediate_canonical_file_step10 = ...  
representative_canonical_file_step10 = ...
```

Each file stores a distinct component of the canonical kinetic calculations. The principal canonical rate file contains the complete temperature-dependent effective Landau-Zener and weak coupling rate constants together with the corresponding reactant partition functions. Separate files summarize the intermediate canonical rate constants and the representative M_S -specific canonical rate constants, thereby preserving the hierarchical organization established throughout Steps 8–10.

The effective canonical results are assembled into a numerical table:

```
canonical_rate_table =  
np.column_stack(  
    table_columns  
)
```

which contains

$$T, \frac{1000}{T}, k_{\text{LZ}}(T), k_{\text{WC}}(T), Q_R(T).$$

This table is subsequently written to disk using:

```
np.savetxt(  
    canonical_rate_file,  
    canonical_rate_table,  
    ...
```

```
)
```

providing a complete numerical description of the effective canonical reaction kinetics over the selected temperature interval.

Additional formatted text files summarize the intermediate canonical rate constants and the representative M_S -specific canonical rate constants. For every intermediate spin-orbit coupling, the program records the corresponding spin projection, degeneracy, intermediate spin-orbit coupling magnitude, and the minimum and maximum canonical rate constants observed over the temperature interval. Similarly, each representative M_S -specific symmetry group is documented together with its absolute spin-orbit coupling magnitude, equivalent-channel multiplicity, representative transition label, and canonical rate range. These summaries provide concise numerical overviews of the complete canonical kinetic hierarchy while avoiding duplication of the full temperature-dependent rate arrays.

For the cluster implementation, all generated output files and graphical figures are automatically transferred to the corresponding remote project directory using the active SFTP connection:

```
sftp.put(  
    local_file,  
    remote_file  
)
```

thereby maintaining identical local and remote workflow directories throughout the calculation.

Finally, the complete collection of canonical quantities is exported to the global workspace:

```
globals().update({  
    ...  
})
```

including the temperature grid, effective canonical Landau-Zener and weak coupling rate constants, reactant partition functions, intermediate canonical rate dictionaries, representative M_S -specific canonical rate dictionaries, plotting utilities, output directories, and filenames generated during this stage. These exported variables constitute the complete canonical kinetic data set and remain available for all subsequent workflow steps without requiring any recalculation.

4.10.9 Summary of the step 10 algorithm

Step 10 transforms the microcanonical reaction rate constants generated in Step 9 into temperature-dependent canonical rate constants by performing Boltzmann averaging over the complete reactant rovibrational manifold. For each temperature, the reactant rovibrational partition function is first evaluated,

$$Q_R(T) = \sum_i \rho_R(E_i) \exp\left(-\frac{E_i}{k_B T}\right) \Delta E,$$

after which the canonical reaction rate constant is obtained from

$$k(T) = \frac{\sum_i k(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{Q_R(T)}.$$

This canonical averaging procedure is applied independently to the effective, intermediate, and representative M_S -specific microcanonical Landau-Zener and weak coupling rate constants generated during Step 9, yielding

$$k_{\text{LZ}}^{\text{eff}}(T), k_{\text{WC}}^{\text{eff}}(T), \\ k_{\text{LZ}}^{(i)}(T), k_{\text{WC}}^{(i)}(T),$$

and

$$k_{\text{LZ}}^{(M_S)}(T), k_{\text{WC}}^{(M_S)}(T),$$

respectively.

Channels possessing identical absolute spin-orbit coupling magnitudes retain the symmetry-grouping strategy introduced in Step 9. Because these channels produce identical microcanonical and canonical reaction rate constants,

$$k_1(T) = k_2(T) = \dots,$$

only a single representative canonical curve is retained for graphical visualization. No summation, averaging, weighting, or degeneracy scaling of M_S -specific canonical rate constants is performed.

The computational sequence implemented in Step 10 is summarized schematically below:

Step 9 microcanonical rates

↓

Temperature grid
(Tmin, Tmax, nT)

↓

Boltzmann weighting
 $\exp(-E/k_B T)$

↓

Reactant partition

function QR(T)
↓
Effective canonical
LZ/WC rates
↓
Intermediate
canonical rates
↓
Representative
MS-specific
canonical rates
↓
Canonical plots
(k(T), Arrhenius)
↓
Output tables,
summary files,
exported variables,
and remote transfer

Consequently, Step 10 completes the statistical kinetic treatment initiated in Step 9 by converting the complete hierarchy of microcanonical Landau-Zener and weak coupling reaction rate constants into their corresponding temperature-dependent canonical rate constants. The resulting canonical rate curves, numerical tables, graphical outputs, and exported variables constitute the final thermal kinetic predictions of SPINKS and provide the quantities most directly comparable with experimentally measured temperature-dependent reaction rate constants.

4.11 Step 11: Conditional WC Threshold-Truncation Correction

Step 11 provides a conditional correction procedure for weak coupling probabilities that reach or exceed unity. The weak coupling expression used in Step 8 is an analytical approximation and may produce values greater than or equal to one under sufficiently strong coupling or within particular energy regions. Because a transition probability cannot physically exceed unity, SPINKS evaluates the underlying, unclipped weak coupling probability and determines whether the threshold-truncation rule must be applied.

When the raw weak coupling probability remains below unity over the complete energy grid, Step 11 performs no correction and preserves all Step 8–10 results unchanged. When the raw probability reaches or exceeds unity, the probability is set to zero beginning at the first unphysical energy point and continuing through the remainder of the energy grid. The corrected probability arrays are then propagated through the complete weak coupling kinetic treatment, including the effective, intermediate, symmetry-collapsed M_S -specific, microcanonical, and canonical rate calculations. All corrected quantities are stored with the prefix SPINKS_ so that the original Python results remain available for direct comparison.

4.11.1 Import of required quantities and conditional workflow setup

Step 11 begins by verifying that all quantities required to reconstruct the raw weak coupling probability and recompute the corresponding rate constants are available from Steps 8–10:

```
required_vars_step105 = [  
    "jobname",  
    "total_energy_grid_cm1",  
    "E_MECP",  
    "H_SO_cm",  
    "autocm",  
    "mu_au",  
    "gradmean",  
    "gradient_gap_magnitude",  
    "compute_weak_coupling_microcanonical_rate",  
    "effective_WC_probability_for_rates",  
    "effective_WC_rate",  
    "effective_WC_cumulative_states",  
    "E_rate_WC_cm1",  
    "rho_reactant_cm",  
    "local_base",  
    "remote_base",  
]  
  
for var in required_vars_step105:  
    if var not in globals():  
        raise RuntimeError(
```

```
)  
    f"{var} is missing. Run Steps 8–10 first."  
)
```

The imported quantities include the common total-energy grid, the MECP threshold, the effective spin–orbit coupling, the reduced mass, the mean gradient, the gradient-difference quantity, the previously calculated weak coupling probabilities and rates, and the reactant density of states required for canonical averaging.

The workflow mode and execution mode are also recovered:

```
RUN_MODE = globals().get("RUN_MODE", "CLUSTER")  
workflow_mode = globals().get(  
    "workflow_mode",  
    "MECP_ONLY"  
)
```

This allows the correction stage to remain compatible with the preceding cluster workflow and to place all newly generated files in the appropriate local and remote directories.

Step 11 additionally defines helper routines for section formatting, file writing, figure saving, M_S notation, and construction of representative-channel legends:

```
def section(title):  
    ...  
  
def write_local_text(path, text):  
    ...  
  
def save_current_figure(path):  
    ...  
  
def ms_tex_value(x):  
    ...  
  
def make_corrected_group_legend(  
    prefix,  
    group_data  
):  
    ...
```

The legend helper preserves the same symmetry-grouping convention used in Steps 9 and 10. Multiple symbols indicate coincident curves arising from channels with identical $|H_{SO}|$, and no summation is implied.

4.11.2 Reconstruction of the raw weak coupling probability

The correction decision is based on the raw, unclipped weak coupling probability rather than the probability array previously constrained to the physical interval. Step 11 therefore reconstructs

the underlying weak coupling expression directly from the effective spin–orbit coupling and the common energy grid:

```
def compute_raw_weak_coupling_probability(
    coupling_cm1,
    energy_grid_cm1
):
    energy_grid_cm1 = np.asarray(
        energy_grid_cm1,
        dtype=float
    ).reshape(-1)

    coupling_Eh = (
        float(abs(coupling_cm1))
        / float(autocm)
    )

    excess_energy_Eh = (
        energy_grid_cm1 - float(E_MECP)
    ) / float(autocm)
```

The absolute spin–orbit coupling is converted from cm^{-1} to Hartree according to

$$H_{\text{SO}}^{\text{Eh}} = \frac{|H_{\text{SO}}^{\text{cm}^{-1}}|}{C_{\text{Eh} \rightarrow \text{cm}^{-1}}},$$

where autocm is the Hartree-to-wavenumber conversion factor.

The energy relative to the MECP is similarly converted to Hartree:

$$E_{\text{excess}} = \frac{E - E_{\text{MECP}}}{C_{\text{Eh} \rightarrow \text{cm}^{-1}}}.$$

The raw weak coupling probability is evaluated using the Airy-function expression implemented in the code:

```
prefactor = (
    4.0
    * np.pi**2
    * coupling_Eh**2
    * (
        2.0 * float(mu_au)
        / (
            float(projected_gradient_mean)
            * float(gradient_gap_magnitude)
        )
    ) ** (2.0 / 3.0)
)
```

```

scale = (
    2.0
    * float(mu_au)
    * float(gradient_gap_magnitude)**2
    / float(projected_gradient_mean)**4
) ** (1.0 / 3.0)

```

The corresponding prefactor is

$$A_{\text{WC}} = 4\pi^2 H_{\text{SO}}^2 \left[\frac{2\mu}{\bar{g} \Delta F_{\parallel}} \right]^{2/3},$$

where μ is the reduced mass, $\bar{g}$ corresponds to projected_gradient_mean, and $\Delta F_{\parallel}$ corresponds to gradient_gap_magnitude.

The Airy-function argument is defined through

$$x(E) = -(E - E_{\text{MECP}}) \left[\frac{2\mu \Delta F_{\parallel}^2}{\bar{g}^4} \right]^{1/3}.$$

The code evaluates the Airy function as:

```

airy_argument[valid] = (
    -excess_energy_Eh[valid] * scale
)

airy_value[valid] = airy(
    airy_argument[valid]
)[0]

raw_probability[valid] = (
    prefactor * airy_value[valid] ** 2
)

```

Thus, the raw probability is

$$P_{\text{WC}}^{\text{raw}}(E) = A_{\text{WC}} \text{Ai}^2[x(E)].$$

No clipping to unity is performed at this stage. This is essential because the purpose of Step 11 is to determine whether the original analytical expression itself becomes unphysical.

The routine returns:

```

return raw_probability, airy_argument, airy_value

```

including the raw probability, the Airy-function argument, and the Airy-function values. Retaining these intermediate quantities allows the complete mathematical behavior of the weak coupling expression to be inspected and exported.

4.11.3 Application of the weak coupling threshold-truncation rule

The threshold-truncation rule is implemented by a dedicated helper routine:

```
def apply_probability_threshold_truncation(
    raw_probability,
    energy_grid_cm1,
    threshold=1.0
):
```

The routine first verifies that the probability and energy arrays have identical lengths:

```
if raw_probability.size != energy_grid_cm1.size:
    raise RuntimeError(
        "raw_probability and energy_grid_cm1 "
        "must have the same length."
    )
```

The indices at which the raw probability reaches or exceeds the selected threshold are then identified:

```
threshold_indices = np.flatnonzero(
    raw_probability >= float(threshold)
)[0]
```

For the default threshold,

$$P_{\text{threshold}} = 1,$$

the first unphysical index satisfies

$$P_{\text{WC}}^{\text{raw}}(E_i) \geq 1.$$

If no such point exists, the original probability remains unchanged. If one or more such points exist, the first offending energy is recorded and the threshold-truncation rule is applied:

```
if threshold_indices.size > 0:
    correction_needed = True
    first_bad_index = int(threshold_indices[0])
    first_bad_energy = float(
        energy_grid_cm1[first_bad_index]
    )
    first_bad_value = float(
        raw_probability[first_bad_index]
```

```
)
```

```
corrected_probability[first_bad_index:] = 0.0
```

The corrected probability is therefore

$$P_{\text{WC}}^{\text{diff}}(E) = \begin{cases} P_{\text{WC}}^{\text{raw}}(E), & E < E_{\text{first}}, \\ 0, & E \geq E_{\text{first}}, \end{cases}$$

where E_{first} is the first energy for which

$$P_{\text{WC}}^{\text{raw}}(E) \geq 1.$$

The result is subsequently restricted to the physical interval:

```
corrected_probability[~np.isfinite(corrected_probability)] = 0.0
```

```
corrected_probability = np.clip(  
    corrected_probability,  
    0.0,  
    1.0  
)
```

The routine returns a structured dictionary containing the corrected probability, the correction flag, the first unphysical index, the first unphysical energy, the corresponding raw probability, and the threshold:

```
return {  
    "corrected_probability": corrected_probability,  
    "correction_needed":  
        correction_needed,  
    "first_bad_index":  
        first_bad_index,  
    "first_bad_energy":  
        first_bad_energy,  
    "first_bad_value":  
        first_bad_value,  
    "threshold":  
        float(threshold),  
}
```

This design allows the same convention to be applied consistently to the effective, intermediate, and M_S -specific weak coupling channels.

4.11.4 Conditional evaluation of the threshold-corrected effective weak coupling rate

The raw effective weak coupling probability is first reconstructed using the effective spin–orbit coupling:

```
P_WC_raw_effective, \  
airy_arg_WC_raw_effective, \  
Ai_WC_raw_effective = (  
    compute_raw_weak_coupling_probability(  
        H_SO_cm,  
        total_energy_grid_cm1  
    )  
)
```

The threshold-truncation rule is then tested:

```
effective_correction_result = (  
    apply_probability_threshold_truncation(  
        P_WC_raw_effective,  
        total_energy_grid_cm1,  
        threshold=1.0  
    )  
)
```

The principal diagnostic values are extracted:

```
WC_threshold_correction_needed = (  
    effective_correction_result[  
        "correction_needed"  
    ]  
)  
  
WC_threshold_first_bad_index = (  
    effective_correction_result[  
        "first_bad_index"  
    ]  
)  
  
WC_threshold_first_bad_energy = (  
    effective_correction_result[  
        "first_bad_energy"  
    ]  
)  
  
WC_threshold_first_bad_value = (  
    effective_correction_result[  
        "first_bad_value"  
    ]  
)
```

These values are exported immediately so that later workflow stages can determine whether the correction was activated.

No-correction branch

When the raw effective probability never reaches unity, the code terminates the correction workflow without recomputing rates:

```
if not WC_threshold_correction_needed:
```

```
    print(
        "Raw WC probability "
        "never reached 1."
    )

    print(
        "SPINKS WC correction "
        "is NOT needed."
    )

    print(
        "No CORRECTED-prefixed "
        "variables were created."
    )
```

In this branch:

- no CORRECTED-prefixed probability arrays are generated,
- no weak coupling microcanonical rates are recomputed,
- no canonical rates are recomputed,
- no figures are produced,
- no correction files are saved,
- and no remote uploads are attempted.

This preserves the original Step 8–10 results exactly when the analytical weak coupling expression remains physically acceptable.

Correction branch

When the raw effective probability reaches or exceeds unity, the corrected effective probability is obtained as:

```
CORRECTED_P_WC_effective = (
    effective_correction_result["corrected_probability"]
)
```

The corresponding effective number of crossing states and microcanonical rate constant are recomputed:

```
CORRECTED_N_WC_effective, \
```

```
CORRECTED_k_WC_effective = (
    compute_weak_coupling_microcanonical_rate(
        CORRECTED_P_WC_effective
    )
)
```

Therefore,

$$N_{\text{WC}}^{\text{diff}}(E) = \sum_j P_{\text{WC}}^{\text{diff}}(E_j) \rho_{TP}(E - E_j) \Delta E,$$

and

$$k_{\text{WC}}^{\text{diff}}(E) = \frac{g_R N_{\text{WC}}^{\text{diff}}(E)}{h \rho_R(E)}.$$

The corrected arrays are assigned several compatible names:

```
CORRECTED_k_WC_effective_micro_s_inv = (
    CORRECTED_k_WC_effective
)

CORRECTED_k_WC_effective = (
    CORRECTED_k_WC_effective
)

CORRECTED_N_WC_effective = (
    CORRECTED_N_WC_effective
)
```

The code then reports the maximum original clipped probability, maximum raw probability, maximum corrected probability, original microcanonical rate, and corrected microcanonical rate. This provides an immediate numerical comparison between the ordinary Python treatment and the threshold-truncation rule.

4.11.5 Correction of intermediate weak coupling probabilities and rates

When effective correction is required, Step 11 applies the same procedure to all intermediate weak coupling channels generated in Steps 8 and 9:

```
CORRECTED_P_WC_intermediate = {}
CORRECTED_N_WC_intermediate = {}
CORRECTED_k_WC_intermediate = {}
```

For each intermediate coupling:

```
for key, data in \
    k_WC_intermediate.items():
```

```
H_int = float(
    data["H_int_cm1"]
)
```

the raw probability is reconstructed:

```
P_raw, airy_arg_raw, Ai_raw = (
    compute_raw_weak_coupling_probability(
        H_int,
        total_energy_grid_cm1
    )
)
```

The threshold-truncation rule is then applied independently:

```
result = apply_probability_threshold_truncation(
    P_raw,
    total_energy_grid_cm1,
    threshold=1.0
)
corrected_probability = result["corrected_probability"]
```

The corrected number of crossing states and microcanonical rate constant are evaluated through the same weak coupling RRKM routine:

```
corrected_states, corrected_rate = (
    compute_weak_coupling_microcanonical_rate(
        corrected_probability
    )
)
```

For each intermediate channel i ,

$$P_{WC,i}^{\text{raw}}(E) \rightarrow P_{WC,i}^{\text{diff}}(E) \rightarrow N_{WC,i}^{\text{diff}}(E) \rightarrow k_{WC,i}^{\text{diff}}(E).$$

The probability dictionary stores both corrected and uncorrected quantities:

```
CORRECTED_P_WC_intermediate[key] = {
    "H_int_cm1": H_int,
    "abs_Ms_low":
        float(data["abs_Ms_low"]),
    "degeneracy":
        int(data["degeneracy"]),
    "P_raw": P_raw,
    "corrected_probability": corrected_probability,
    "airy_arg_raw": airy_arg_raw,
    "Ai_raw": Ai_raw,
    "correction_needed":
        result["correction_needed"],
    "first_bad_index":
        result["first_bad_index"],
    "first_bad_energy":
```

```

    result["first_bad_energy"],
    "first_bad_value":
    result["first_bad_value"],
}

```

The corrected kinetic quantities are stored separately:

```

CORRECTED_k_WC_intermediate[key] = {
    "H_int_cm1": H_int,
    "abs_Ms_low":
    float(data["abs_Ms_low"]),
    "degeneracy":
    int(data["degeneracy"]),
    "corrected_states": corrected_states,
    "corrected_rate": corrected_rate,
    "corrected_probability": corrected_probability,
    "P_raw": P_raw,
    "correction_needed":
    result["correction_needed"],
    "first_bad_index":
    result["first_bad_index"],
    "first_bad_energy":
    result["first_bad_energy"],
    "first_bad_value":
    result["first_bad_value"],
}

```

Importantly, the effective probability determines whether Step 11 is activated globally, but each intermediate channel retains its own `correction_needed` flag. Because intermediate couplings may differ in magnitude, some intermediate channels may reach unity while others may remain below it. The code therefore applies the correction independently to every intermediate coupling rather than assuming identical behavior.

4.11.6 Correction of symmetry-collapsed M_S -specific weak coupling channels

Step 11 next extends the downstream statistical treatment to the symmetry-collapsed M_S -specific weak coupling groups constructed in Step 9:

```

CORRECTED_P_WC_channel_groups = {}
CORRECTED_N_WC_channel_groups = {}
CORRECTED_k_WC_channel_groups = {}

```

For each representative group, the absolute spin-orbit coupling and the corresponding channel metadata are retrieved:

```

for group_key, group_data in \
    k_WC_channel_groups.items():

    H_abs = float(
        group_data["H_abs_cm1"]
    )

```

```

labels = list(
    group_data.get("labels", [])
)

symbols = list(
    group_data.get(
        "channel_symbols",
        []
    )
)

representative_label = (
    group_data.get(
        "representative_label"
    )
)

```

The raw weak coupling probability is then reconstructed using the group-specific coupling:

```

P_raw, airy_arg_raw, Ai_raw = (
    compute_raw_weak_coupling_probability(
        H_abs,
        total_energy_grid_cm1
    )
)

```

The threshold-truncation rule is applied independently to the representative group:

```

result = apply_probability_threshold_truncation(
    P_raw,
    total_energy_grid_cm1,
    threshold=1.0
)

corrected_probability = result["corrected_probability"]

```

The representative corrected number of crossing states and rate constant are recomputed:

```

corrected_representative_states, \
corrected_representative_rate = (
    compute_weak_coupling_microcanonical_rate(
        corrected_probability
    )
)

```

For each group g ,

$$P_{WC,g}^{\text{raw}}(E) \rightarrow P_{WC,g}^{\text{diff}}(E) \rightarrow N_{WC,g}^{\text{diff}}(E) \rightarrow k_{WC,g}^{\text{diff}}(E).$$

The corrected representative data are stored together with all channel labels and symmetry information:

```

CORRECTED_k_WC_channel_groups[

```

```

    group_key
] = {
    "H_abs_cm1": H_abs,
    "degeneracy": int(
        group_data.get(
            "degeneracy",
            len(labels)
        )
    ),
    "channel_symbols": symbols,
    "labels": labels,
    "representative_label":
        representative_label,
    "corrected_representative_states":
        corrected_representative_states,
    "corrected_representative_rate":
        corrected_representative_rate,
    "corrected_probability": corrected_probability,
    "P_raw": P_raw,
    "correction_needed":
        result["correction_needed"],
    "first_bad_index":
        result["first_bad_index"],
    "first_bad_energy":
        result["first_bad_energy"],
    "first_bad_value":
        result["first_bad_value"],
    "combination_rule":
        "representative_only_no_sum",
}

```

As in Steps 9 and 10, channels possessing identical $|H_{SO}|$ values generate identical probability and rate curves. Therefore, one representative corrected curve is retained for each group. The explicit rule

representative_only_no_sum

confirms that no probabilities, numbers of states, microcanonical rates, or canonical rates are summed or multiplied by the number of equivalent channels.

4.11.7 Recalculation of canonical weak coupling rate constants

When the Step 10 canonical averaging routine and temperature variables are available, Step 11 recomputes the corrected canonical weak coupling rates:

```

CORRECTED_k_WC_canonical_s_inv = None
CORRECTED_WC_canonical_normalization = None

CORRECTED_k_WC_intermediate_canonical = {}

```

```
CORRECTED_k_WC_channel_groups_canonical = {}
```

The corrected effective microcanonical rate is canonically averaged using the same expression introduced in Step 10:

```
CORRECTED_k_WC_canonical_s_inv, \
CORRECTED_WC_canonical_normalization = (
    boltzmann_average_microcanonical_rate(
        E_rate_WC_cm1,
        rho_reactant_cm,
        CORRECTED_k_WC_effective[1:],
        T_grid_K,
        dE_WC
    )
)
```

The corrected canonical rate is therefore

$$k_{\text{WC}}^{\text{diff}}(T) = \frac{\sum_i k_{\text{WC}}^{\text{diff}}(E_i) \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}}.$$

The first element is removed from the corrected microcanonical array using [1:] so that its dimensions match the Step 10 physical energy grid.

Every corrected intermediate microcanonical curve is treated analogously:

```
for key, data in \
    CORRECTED_k_WC_intermediate.items():

    k_T, _ = (
        boltzmann_average_microcanonical_rate(
            E_rate_WC_cm1,
            rho_reactant_cm,
            data["corrected_rate"][1:],
            T_grid_K,
            dE_WC
        )
    )
```

The resulting dictionary retains the intermediate coupling, spin projection, degeneracy, corrected canonical rate, correction flag, and first unphysical energy.

Representative M_S -specific groups are also canonically averaged:

```
for key, data in \
    CORRECTED_k_WC_channel_groups.items():

    k_T, _ = (
        boltzmann_average_microcanonical_rate(
            E_rate_WC_cm1,
            rho_reactant_cm,
            data[
```

```

        "corrected_representative_rate"
    ][1:],
    T_grid_K,
    dE_WC
)
)

```

Each representative canonical entry preserves

- $|H_{SO}|$,
- equivalent-channel count,
- symbols,
- labels,
- representative identity,
- correction status,
- first bad energy,
- and the no-summation rule.

If Step 10 variables are unavailable, canonical correction is skipped while the corrected probability and microcanonical results remain available.

4.11.8 Diagnostics, output files, plots, and exported variables

Selected-energy diagnostic

Step 11 prints a direct comparison at selected energies:

```

selected_energies = [
    3000,
    3500,
    3560,
    3566,
    3600,
    4000,
    5000,
    6000,
    8000
]

```

For each selected energy, the code reports

- raw unclipped weak coupling probability,
- original Python probability,

- corrected SPINKS probability,
- original Python microcanonical rate,
- and corrected microcanonical rate.

This diagnostic makes the onset and consequences of the correction directly visible at representative energy points.

Output files

When correction is required, Step 11 creates a dedicated directory:

```
local_step11 = os.path.join(
    local_base,
    "WC_threshold_correction"
)

remote_step11 = posixpath.join(
    remote_base,
    "WC_threshold_correction"
)
```

Four principal text files are generated:

```
summary_file_step11 = ...
effective_grid_file_step11 = ...
intermediate_file_step11 = ...
channel_file_step11 = ...
```

The summary file records the effective SOC, MECP energy, correction flag, first unphysical energy and probability, maximum original and corrected probabilities, maximum microcanonical rates, maximum canonical rates, and counts of corrected intermediate and representative channels.

The effective-grid file contains the full energy-dependent comparison:

$$E, P_{\text{WC}}^{\text{raw}}, P_{\text{WC}}^{\text{Python}}, P_{\text{WC}}^{\text{diff}}, \\ N_{\text{WC}}^{\text{Python}}, N_{\text{WC}}^{\text{diff}}, k_{\text{WC}}^{\text{Python}}, k_{\text{WC}}^{\text{diff}}.$$

The intermediate summary records each coupling, spin projection, degeneracy, correction status, first unphysical energy, first unphysical probability, and maximum corrected rate.

The representative M_S -specific summary records each group coupling, equivalent-channel count, channel symbols, representative label, correction status, first unphysical point, and maximum representative corrected rate.

Probability and rate plots

Step 11 generates a probability comparison plot containing:

```

plt.plot(
    total_energy_grid_cm1,
    effective_WC_probability_for_rates,
    label="Python WC clipped"
)

plt.plot(
    total_energy_grid_cm1,
    P_WC_raw_effective,
    label="Raw WC unclipped"
)

plt.plot(
    total_energy_grid_cm1,
    CORRECTED_P_WC_effective,
    label="Corrected $P_{WC}$"
)

```

The plot additionally marks

- $P = 1$,
- the MECP energy,
- and the first energy at which the raw probability reaches unity.

A microcanonical-rate comparison plot is also generated:

```

plt.plot(
    total_energy_grid_cm1[mask_py],
    np.log10(
        effective_WC_rate[mask_py]
    ),
    label="Python $k_{WC}(E)$"
)

plt.plot(
    total_energy_grid_cm1[mask_corrected],
    np.log10(
        CORRECTED_k_WC_effective[
            mask_corrected
        ]
    ),
    label="Corrected $k_{WC}(E)$"
)

```

Only positive rates are plotted logarithmically.

When canonical correction is available, two additional figures are produced:

1. $k_{WC}(T)$ versus temperature;
2. $\log_{10}[k_{WC}(T)]$ versus $1000/T$.

These plots directly compare the original Python canonical rate with the threshold-corrected canonical rate.

Remote transfer

For cluster calculations, all text files and figures are transferred using SFTP to the corresponding remote correction directory. If the transfer fails, the local files remain available and the remote-path variables are reset to None.

Exported variables

The complete corrected data set is exported through:

```
globals().update({
    ...
})
```

Important exported quantities include:

```
P_WC_raw_effective
CORRECTED_P_WC_effective
CORRECTED_N_WC_effective
CORRECTED_k_WC_effective
CORRECTED_k_WC_effective_micro_s_inv
CORRECTED_k_WC_intermediate
CORRECTED_k_WC_channel_groups
CORRECTED_k_WC_canonical_s_inv
CORRECTED_k_WC_intermediate_canonical
CORRECTED_k_WC_channel_groups_canonical
WC_threshold_correction_needed
WC_threshold_first_bad_energy
```

The original Python variables are not overwritten. This permits subsequent workflow stages to choose explicitly between the original weak coupling results and the threshold-corrected results.

4.11.9 Summary of the step 11 algorithm

Step 11 conditionally applies the threshold-truncation rule when the analytical weak coupling probability becomes unphysical. The raw weak coupling probability is reconstructed from the spin-orbit coupling, reduced mass, mean gradient, gradient difference, MECP energy, and Airy-function expression:

$$P_{\text{WC}}^{\text{raw}}(E) = 4\pi^2 H_{\text{SO}}^2 \left[\frac{2\mu}{\bar{g} \Delta F_{\parallel} \hbar^2} \right]^{2/3} \text{Ai}^2 \left[-(E - E_{\text{MECP}}) \left(\frac{2\mu \Delta F_{\parallel}^2}{\bar{g}^4 \hbar^2} \right)^{1/3} \right].$$

The code then identifies the first energy satisfying

$$P_{WC}^{raw}(E) \geq 1.$$

When no such energy exists, Step 11 terminates without modifying the Step 8–10 probability or rate arrays. No CORRECTED-prefixed rates, figures, or files are generated.

When the threshold is reached, the corrected probability is defined as

$$P_{WC}^{diff}(E) = \begin{cases} P_{WC}^{raw}(E), & E < E_{first}, \\ 0, & E \geq E_{first}. \end{cases}$$

The corrected effective number of crossing states and microcanonical rate constant are then recomputed:

$$N_{WC}^{diff}(E) = \sum_j P_{WC}^{diff}(E_j) \rho_{TP}(E - E_j) \Delta E,$$

$$k_{WC}^{diff}(E) = \frac{g_R N_{WC}^{diff}(E)}{h \rho_R(E)}.$$

The same correction is applied independently to every intermediate weak coupling channel and every symmetry-collapsed M_S -specific representative channel:

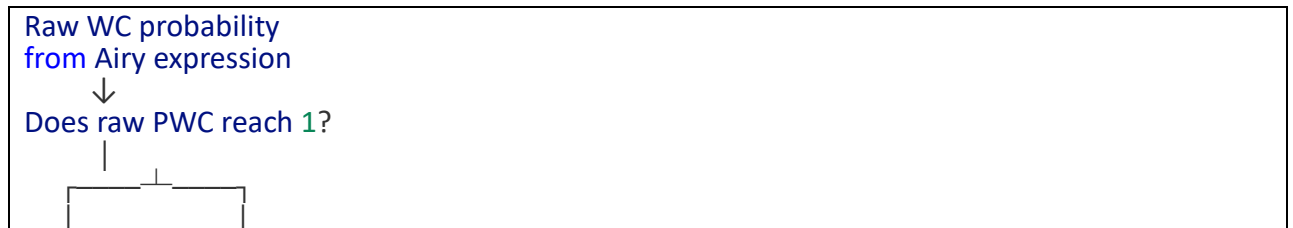
$$P_i^{raw}(E) \rightarrow P_i^{diff}(E) \rightarrow N_i^{diff}(E) \rightarrow k_i^{diff}(E).$$

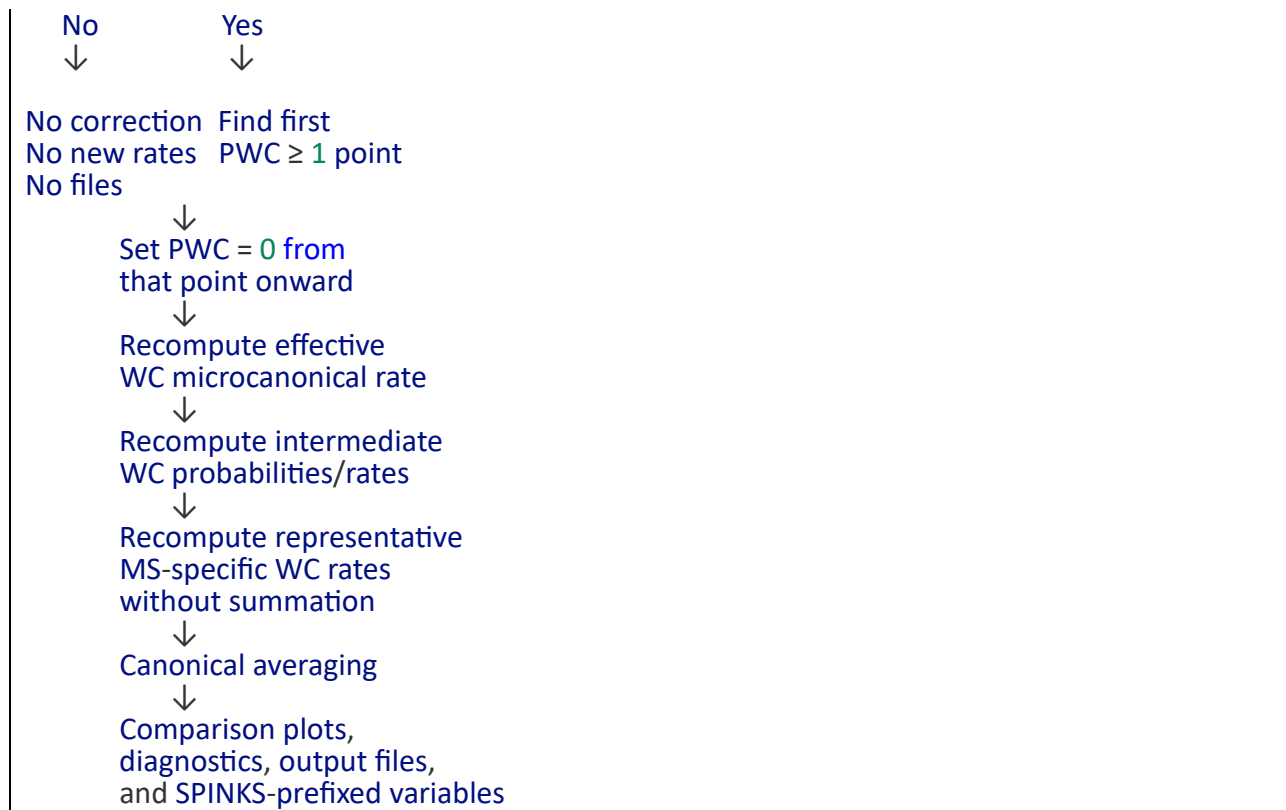
Channels with identical $|H_{SO}|$ retain a single representative corrected curve. No channel probabilities, numbers of states, microcanonical rates, or canonical rates are summed.

When Step 10 variables are available, all corrected microcanonical rates are further transformed into canonical rate constants:

$$k_{WC}^{diff}(T) = \frac{\sum_i k_{WC}^{diff}(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}.$$

The complete Step 11 workflow can therefore be summarized as:





Accordingly, Step 11 serves as a conditional consistency layer between the analytical weak coupling probability and the threshold-truncation rule. It preserves the original Python treatment whenever the raw probability remains physical, but when $P_{WC}^{\text{raw}} \geq 1$, it constructs a complete parallel hierarchy of threshold-corrected effective, intermediate, and M_S -specific weak coupling probabilities, microcanonical rates, and canonical rates without overwriting the original results.

4.12 Step 12: Results Reporting and Interactive Diagnostics

Step 12 completes the SPINKS workflow by consolidating the principal physical quantities and kinetic results generated throughout the preceding stages into a final human-readable report. It also provides an interactive energy-resolved diagnostic for direct inspection of the Landau-Zener and weak coupling probabilities, numbers of states, and microcanonical rate constants at user-selected energies.

A second major function of Step 12 is the automatic generation of two complete NAST input files. One input uses the electronic MECP barrier together with internal NAST zero-point-energy treatment, whereas the second uses the externally calculated ZPE-corrected MECP barrier and disables additional NAST ZPE correction. The two files therefore provide equivalent but independently testable representations of the barrier convention.

Step 12 also determines whether the original weak coupling results or the NAST-corrected weak coupling results from Step 11 must be used in the final report. The original arrays are retained when no correction was required, whereas all downstream weak coupling quantities are automatically replaced by their NAST_ counterparts when Step 11 was activated.

4.12.1 Initialization and helper functions

Step 12 begins by importing the numerical and file-system packages required for reporting, array processing, directory construction, and optional cluster transfer:

```
import os
import posixpath
import numpy as np

print(
    "\n===== STEP 12: "
    "FINAL RESULTS REPORTING AND DIAGNOSTICS "
    "===== \n"
)
```

The energy-conversion constants used throughout the final report are then defined:

```
hartree_to_cm = float(
    globals().get(
        "hartree_to_cm",
        219474.6313705
    )
)

hartree_to_kjmol = 2625.499638
cm_to_kjmol = 0.01196266
```

These quantities correspond to the conversions

$$1 E_h = 219474.6313705 \text{ cm}^{-1},$$

and

$$1 E_h = 2625.499638 \text{ kJ mol}^{-1}.$$

The `cm_to_kjmol` factor is additionally defined for wavenumber-to-energy conversion, although the final energy blocks in this implementation primarily convert directly from Hartree.

Several general-purpose helper functions are constructed. The section-formatting function generates standardized console headings:

```
def section(title):
    line = (
        "\n"
        + "=" * 72
        + f"\n {title}\n"
        + "=" * 72
        + "\n"
    )
    print(line)
    return line
```

The scalar-formatting helper safely handles missing, nonfinite, and nonnumeric quantities:

```
def fmt_scalar(x, digits=12):
    try:
        if x is None:
            return "None"

        x = float(x)

        if not np.isfinite(x):
            return "nan"

        return f"{x:.{digits}g}"

    except Exception:
        return str(x)
```

This function prevents the final report from failing when an optional quantity is unavailable and ensures consistent numerical formatting.

Two additional routines retrieve arrays or scalar values from the global workspace using ordered lists of possible variable names:

```
def as_array_or_none(*names):
    for name in names:
        if name in globals():
            arr = np.asarray(
```

```

        globals()[name],
        dtype=float
    ).reshape(-1)

    if arr.size > 0:
        return arr, name

    return None, None

```

and:

```

def scalar_or_none(*names):
    for name in names:
        if name in globals():
            try:
                return (
                    float(globals()[name]),
                    name
                )
            except Exception:
                pass

    return None, None

```

This design provides compatibility among different code versions that may use alternative variable names for the same physical quantity. The selected source name is returned together with the numerical value and later written explicitly to the final report.

The `energy_block()` helper converts a Hartree-valued quantity into Hartree, wavenumber, and kJ mol⁻¹ representations:

```

def energy_block(label, value_Eh):
    if (
        value_Eh is None
        or not np.isfinite(value_Eh)
    ):
        return [
            f"{label}_hartree = nan",
            f"{label}_cm1 = nan",
            f"{label}_kJmol = nan",
        ]

    return [
        f"{label}_hartree = "
        f"{value_Eh:.15f}",

        f"{label}_cm1 = "
        f"{value_Eh * hartree_to_cm:.8f}",

        f"{label}_kJmol = "
        f"{value_Eh * hartree_to_kjmol:.8f}",
    ]

```

For a quantity E expressed in Hartree,

$$E_{\text{cm}^{-1}} = E_{E_h} C_{E_h \rightarrow \text{cm}^{-1}},$$

and

$$E_{\text{kJ mol}^{-1}} = E_{E_h} C_{E_h \rightarrow \text{kJ mol}^{-1}}.$$

The NAST-array formatting function divides long arrays into multiple namelist-compatible lines:

```
def format_namelist_array(
    name,
    values,
    per_line=10
):
    values = np.asarray(
        values,
        dtype=float
    ).reshape(-1)

    chunks = []

    for i in range(
        0,
        len(values),
        per_line
    ):
        chunk = ", ".join(
            f"{x:.8g}"
            for x in values[
                i:i + per_line
            ]
        )

        if i == 0:
            chunks.append(
                f"{name} = {chunk}"
                + (
                    " "
                    if i + per_line
                    < len(values)
                    else ""
                )
            )
        else:
            chunks.append(
                f"      {chunk}"
                + (
                    " "
                    if i + per_line
                    < len(values)
                    else ""
                )
            )
```

```
)  
)
```

```
return "\n".join(chunks)
```

The remaining helpers write text files and simultaneously print and retain console messages:

```
def write_text(path, text):  
    os.makedirs(  
        os.path.dirname(path),  
        exist_ok=True  
    )  
  
    with open(  
        path,  
        "w",  
        encoding="utf-8",  
        newline="\n"  
    ) as f:  
        f.write(text)  
def print_and_store(  
    lines,  
    text=""  
):  
    print(text)  
    lines.append(str(text))
```

The latter is important because the canonical table and interactive diagnostic output are later embedded directly in the final report.

4.12.2 Automatic selection of the active weak coupling results

Step 12 determines whether the original weak coupling results from Steps 8–10 or the NAST-corrected results from Step 11 must be used:

```
use_threshold_corrected_WC = (  
    "WC_threshold_correction_needed"  
    in globals()  
  
    and  
    WC_threshold_correction_needed  
    is True  
  
    and  
    "CORRECTED_P_WC_effective"  
    in globals()  
  
    and  
    "CORRECTED_k_WC_effective_micro_s_inv"  
    in globals()  
)
```

The correction is activated only when all of the following conditions are satisfied:

1. the Step 11 correction flag exists;
2. that flag is explicitly True;
3. the corrected weak coupling probability is available;
4. the corrected weak coupling microcanonical rate is available.

When these conditions are met, the active weak coupling quantities are assigned from the Step 11 arrays:

```
if use_threshold_corrected_WC:
    active_probWC = (
        CORRECTED_P_WC_effective
    )

    active_rateWC = (
        CORRECTED_k_WC_effective_micro_s_inv
    )

    active_nosWC = (
        CORRECTED_N_WC_effective
    )

    active_WC_label = (
        "threshold-corrected WC"
    )
```

Otherwise, the original weak coupling quantities are retained:

```
else:
    active_probWC = effective_WC_probability_for_rates
    active_rateWC = effective_WC_rate
    active_nosWC = effective_WC_cumulative_states
    active_WC_label = "Raw WC"
```

This conditional assignment ensures that all subsequent reporting and diagnostics use a single internally consistent set of weak coupling quantities:

$$P_{\text{WC}}^{\text{active}}(E), N_{\text{WC}}^{\text{active}}(E), k_{\text{WC}}^{\text{active}}(E).$$

The selected source is immediately recorded:

```
step12_console_lines = []
debug_records = []

print_and_store(
    step12_console_lines,
    f"WC source: {active_WC_label}"
)
```

The original Step 8–10 arrays are not overwritten. The active variables serve only as aliases used for final reporting.

4.12.3 Reporting of canonical rate constants

Step 12 first verifies that the canonical temperature grid and both effective canonical rate arrays are available:

```
required_canonical_vars = [  
    "T_grid_K",  
    "k_LZ_canonical_s_inv",  
    "k_WC_canonical_s_inv",  
]  
  
missing_canonical = [  
    v  
    for v in required_canonical_vars  
    if v not in globals()  
]
```

If one or more quantities are missing, a warning is stored and canonical weak coupling reporting is disabled:

```
if missing_canonical:  
    print_and_store(  
        step12_console_lines,  
        "\nWARNING: Canonical Step 10 "  
        "variables not found."  
    )  
  
    print_and_store(  
        step12_console_lines,  
        f"Missing: {missing_canonical}"  
    )  
  
    active_k_WC_canonical = None
```

When the canonical arrays are available, Step 12 selects the corrected canonical weak coupling rate whenever Step 11 was activated:

```
else:  
    if (  
        use_threshold_corrected_WC  
        and  
        "CORRECTED_k_WC_canonical_s_inv"  
        in globals()  
    ):  
        active_k_WC_canonical = (  
            CORRECTED_k_WC_canonical_s_inv  
        )  
    else:
```

```

active_k_WC_canonical = (
    k_WC_canonical_s_inv
)

```

The Landau-Zener rate always remains the Step 10 result because Step 11 corrects only the weak coupling treatment.

The canonical results are then printed and stored in a formatted table:

```

for T, k_lz, k_wc in zip(
    T_grid_K,
    k_LZ_canonical_s_inv,
    active_k_WC_canonical
):
    log_lz = (
        np.log10(k_lz)
        if k_lz > 0
        else np.nan
    )

    log_wc = (
        np.log10(k_wc)
        if k_wc > 0
        else np.nan
    )

```

For each temperature, the table contains

$$T, \frac{1000}{T}, k_{\text{LZ}}(T), \log_{10} k_{\text{LZ}}(T), \\ k_{\text{WC}}^{\text{active}}(T), \log_{10} k_{\text{WC}}^{\text{active}}(T).$$

The output formatting is implemented as:

```

line = (
    f"{T:8.2f} "
    f"{1000.0/T:10.6f} "
    f"{k_lz:18.10e} "
    f"{log_lz:14.8f} "
    f"{k_wc:18.10e} "
    f"{log_wc:14.8f}"
)

```

Logarithmic values are reported only for positive rate constants. Zero or negative values are represented as nan, avoiding undefined logarithmic operations.

4.12.4 Interactive microcanonical diagnostic

Step 12 next verifies the availability of the quantities required for energy-resolved microcanonical analysis:

```

required_micro_vars = [
    "total_energy_grid_cm1",
    "energy_step_cm1",
    "crossing_energy_bin_index",
    "E_MECP",
    "reactant_state_density",
    "crossing_shifted_state_density",
    "crossing_internal_state_density",
    "effective_LZ_probability_for_rates",
    "effective_LZ_cumulative_states",
    "effective_LZ_rate",
]

```

If any of these variables are missing, execution stops:

```

missing_micro = [
    v
    for v in required_micro_vars
    if v not in globals()
]

if missing_micro:
    raise RuntimeError(
        "Missing microcanonical variables: "
        f"{missing_micro}"
    )

```

The user is then prompted to enter one or more total energies relative to the reference minimum:

```

while True:
    ans = input(
        "Energy in cm^-1, "
        "or 'stop': "
    ).strip()

```

The loop terminates when the user enters stop, s, q, quit, or exit.

For a numerical energy $E_{\text{requested}}$, the corresponding discrete-grid index is obtained as:

```

idx = int(
    round(
        requested_energy_cm1 / energy_step_cm1
    )
)

```

or mathematically,

$$i_{\text{dbg}} = \text{round}\left(\frac{E_{\text{requested}}}{\Delta E}\right).$$

The requested energy is therefore mapped to the nearest available energy bin,

$$E_{\text{bin}} = i_{\text{dbg}} \Delta E.$$

The program checks that the selected index lies within the available energy range. If not, the allowed interval is reported.

The transition-state internal density of states is selected relative to the shifted MECP index:

```
if (
    idx > crossing_energy_bin_index
    and
    (idx - crossing_energy_bin_index)
    < len(crossing_internal_state_density)
):
    crossing_internal_density_value = (
        crossing_internal_state_density[idx - crossing_energy_bin_index]
    )
else:
    crossing_internal_density_value = np.nan
```

Thus, the internal transition-state density is evaluated using the energy measured above the crossing threshold:

$$E_{\text{internal}} = E_{\text{bin}} - E_{\text{MECP}}.$$

When the selected energy does not correspond to a valid internal transition-state bin, `crossing_internal_state_density` is reported as `nan`.

A complete diagnostic record is then constructed:

```
rec = {
    "requested_energy_cm1":
        requested_energy_cm1,

    "grid_energy_cm1":
        float(total_energy_grid_cm1[idx]),

    "energy_index":
        int(idx),

    "crossing_energy_bin_index":
        int(crossing_energy_bin_index),

    "E_MECP_cm1":
        float(E_MECP),

    "WC_source":
        active_WC_label,

    "reactant_state_density":
        float(reactant_state_density[idx]),

    "crossing_shifted_state_density":
        float(crossing_shifted_state_density[idx]),
```

```

"crossing_internal_state_density":
    float(crossing_internal_density_value),

"P_LZ":
    float(effective_LZ_probability_for_rates[idx]),

"P_WC":
    float(active_probWC[idx]),

"N_LZ":
    float(effective_LZ_cumulative_states[idx]),

"N_WC":
    float(active_nosWC[idx]),

"k_LZ_s_inv":
    float(effective_LZ_rate[idx]),

"k_WC_s_inv":
    float(active_rateWC[idx]),
}

```

This record includes the selected energy, energy-bin index, MECP position, reactant and crossing-state densities of states, effective probabilities, effective numbers of states, and microcanonical rates.

The logarithmic rates are also evaluated:

```

"log10_k_LZ": (
    float(
        np.log10(
            effective_LZ_rate[idx]
        )
    )
    if effective_LZ_rate[idx] > 0
    else np.nan
),

"log10_k_WC": (
    float(
        np.log10(
            active_rateWC[idx]
        )
    )
    if active_rateWC[idx] > 0
    else np.nan
),

```

When the threshold-truncation correction is active and the raw probability is available, the uncorrected raw value is additionally included:

```

if (
    use_threshold_corrected_WC

```

```

    and
    "P_WC_raw_effective"
    in globals()
):
    rec["P_WC_raw"] = float(
        P_WC_raw_effective[idx]
    )

```

The diagnostic output is divided into five conceptual blocks:

1. energy-grid information;
2. density-of-states information;
3. effective probabilities;
4. effective numbers of states;
5. microcanonical rates.

The printed result therefore provides a direct energy-specific comparison of

$$P_{\text{LZ}}(E), P_{\text{WC}}(E), \\ N_{\text{LZ}}(E), N_{\text{WC}}(E),$$

and

$$k_{\text{LZ}}(E), k_{\text{WC}}(E).$$

Every selected-energy record is appended to `debug_records`, allowing all user-requested diagnostics to be reproduced later in the final output file.

4.12.5 Construction of the final output directory

After the interactive diagnostic has been completed, Step 12 creates the final output directory:

```

final_output_dir = os.path.join(
    local_base,
    "final_output_and_interoperability_exports"
)

os.makedirs(
    final_output_dir,
    exist_ok=True
)

```

The corresponding cluster directory is also constructed when a remote base path is available:

```

remote_final_output_dir = None

if "remote_base" in globals():
    remote_final_output_dir = (
        posixpath.join(
            remote_base,
            "final_output_and_interoperability_exports"
        )
    )

```

If both the recursive remote-directory function and active SFTP connection are available, the remote directory is created automatically. Failure to create it does not prevent local file generation.

This directory becomes the principal folder for the final SPINKS deliverables and contains

- the complete final output report;
- the electronic-barrier NAST input;
- the ZPE-corrected-barrier NAST input.

4.12.6 Collection of final physical quantities

Step 12 gathers the physical quantities required for the final report and NAST input. For each quantity, several possible variable names are searched in a fixed priority order.

Reactant and MECP frequencies:

```

freR, reference_frequencies_source = (
    as_array_or_none(
        "reference_vibrational_frequencies_cm1",
        "freq_reactant_real_cm1"
    )
)

freX, crossing_frequencies_source = (
    as_array_or_none(
        "freq_MECP_effhess_real_cm1",
        "freq_MECP_eff_real_cm1",
        "freq_MECP_effhess_real_cm1"
    )
)

```

The reactant frequencies are denoted

$$\{\nu_{R,i}\},$$

and the effective-Hessian MECP frequencies are denoted

$$\{v_{X,i}\}.$$

The selected source names are retained for traceability.

Rotational or inertial quantities:

```
inertR, reference_inertias_source = (
  as_array_or_none(
    "inertR",
    "Reference_moments_of_inertia_amu_bohr2",
    "Reference_moments_of_inertia_amu_bohr2",
    "Reference_rot_constants_cm1",
    "reference_rotational_constants_cm1"
  )
)
inertX, crossing_inertias_source = (
  as_array_or_none(
    "inertX",
    "MECP_moments_of_inertia_amu_bohr2",
    "MECP_moments_of_inertia_amu_bohr2",
    "MECP_rot_constants_cm1",
    "crossing_rotational_constants_cm1"
  )
)
```

These arrays may contain moments of inertia or rotational constants, depending on the representation propagated from the earlier workflow. The exact source is explicitly recorded.

Reduced mass, spin–orbit coupling, and gradient quantities:

```
redmass, redmass_source = (
  scalar_or_none(
    "reduced_mass_amu"
  )
)

soc_eff, soc_source = (
  scalar_or_none(
    "H_SO_cm"
  )
)

grad, gradient_gap_source = (
  scalar_or_none(
    "GRADIENT_GAP_NORM_EH_PER_BOHR",
    "gradient_gap_magnitude"
  )
)

gradmean, projected_gradient_mean_source = (
  scalar_or_none(
    "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR",
    "projected_gradient_mean_Eh_per_Bohr"
  )
)
```

```
)  
)
```

These quantities correspond to

$$\mu, H_{\text{SO}}, \Delta F_{\parallel}, \tilde{g}.$$

Importantly, Step 12 preferentially selects the explicitly named finalized gradient quantities:

```
GRADIENT_GAP_NORM_EH_PER_BOHR  
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
```

when available. This ensures that the same validated values used in the probability and rate calculations are reported and passed to NAST.

Absolute electronic energies and electronic barrier:

```
Ele_R_abs_Eh, Ele_R_source = (  
    scalar_or_none(  
        "Ele_REF_hartree",  
        "E_ref_hartree"  
    )  
)  
  
Ele_X_abs_Eh, Ele_X_source = (  
    scalar_or_none(  
        "Ele_MECP_hartree"  
    )  
)
```

The preferred electronic barrier is obtained directly from:

```
Ele_barrier_Eh, \  
Ele_barrier_source = (  
    scalar_or_none(  
        "E_MECP_electronic_hartree"  
    )  
)
```

If this quantity is unavailable, it is reconstructed from the absolute electronic energies:

```
if (  
    Ele_barrier_Eh is None  
    and  
    Ele_R_abs_Eh is not None  
    and  
    Ele_X_abs_Eh is not None  
):  
    Ele_barrier_Eh = (  
        Ele_X_abs_Eh  
        - Ele_R_abs_Eh  
    )
```

Therefore,

$$V_{\text{MECP}}^{\text{elec}} = E_{\text{MECP}}^{\text{elec}} - E_R^{\text{elec}}.$$

Reactant and MECP zero-point energies

The reactant ZPE is searched using several possible source variables:

```
ZPE_R_Eh, ZPE_R_source = (
    scalar_or_none(
        "ZPE_REF_freq_hartree",
        "ZPE_REF_from_freq_hartree",
        "ZPE_REF_hartree",
        "ZPE_REF_thermo_hartree"
    )
)
```

The MECP ZPE is similarly selected:

```
ZPE_X_Eh, ZPE_X_source = (
    scalar_or_none(
        "ZPE_MECP_effhess_hartree",
        "ZPE_X_hartree",
        "ZPE_MECP_hartree",
        "ZPE_MECP_from_freq_hartree"
    )
)
```

If the MECP ZPE is unavailable but the MECP frequencies are present, it is reconstructed as:

```
ZPE_X_Eh = (
    0.5
    * float(np.sum(freX))
    / hartree_to_cm
)
```

corresponding to

$$\text{ZPE}_X = \frac{1}{2} \sum_i hc \nu_{X,i}.$$

In Hartree units,

$$\text{ZPE}_X(E_h) = \frac{1}{2} \sum_i \nu_{X,i}.$$

The ZPE-corrected barrier is then evaluated:

```
ZPE_corrected_barrier_Eh = None
if (
```

```

    Ele_barrier_Eh is not None
    and
    ZPE_R_Eh is not None
    and
    ZPE_X_Eh is not None
):
    ZPE_corrected_barrier_Eh = (
        Ele_barrier_Eh
        + (
            ZPE_X_Eh
            - ZPE_R_Eh
        )
    )
)

```

Thus,

$$V_{\text{MECP}}^0 = V_{\text{MECP}}^{\text{elec}} + \text{ZPE}_X - \text{ZPE}_R.$$

NAST temperature and energy-grid parameters

The first and last temperatures of the Step 10 grid are assigned as the NAST temperature limits:

```

export_temperature_min_K = float(
    np.asarray(
        T_grid_K,
        dtype=float
    )[0]
) if "T_grid_K" in globals() else 300.0
export_temperature_max_K = float(
    np.asarray(
        T_grid_K,
        dtype=float
    )[-1]
) if "T_grid_K" in globals() else 350.0

```

The energy spacing is obtained from:

```

export_energy_step_cm1 = float(
    globals().get(
        "energy_step_cm1",
        1.0
    )
)

```

The maximum total energy is taken from the Step 7 value E_max_cm1. The code explicitly requires this quantity:

```

if "E_max_cm1" not in globals():
    raise RuntimeError(
        "Cannot determine NAST maxn "
        "because E_max_cm1 from "
        "Step 7 is missing."
    )

```

The number of NAST energy bins is then calculated as:

```
export_maximum_bin_index = int(
    np.ceil(
        float(E_max_cm1)
        / export_energy_step_cm1
    )
)
```

or

$$\text{maxn} = \left\lceil \frac{E_{\text{max}}}{\Delta E} \right\rceil.$$

The actual maximum energy represented by the integer NAST grid is:

```
export_maximum_energy_cm1 = (
    export_maximum_bin_index
    * export_energy_step_cm1
)
```

so that

$$E_{\text{max}} = \text{maxn} \times \Delta E.$$

4.12.7 Determination of global probability and rate extrema

Before constructing the final report, Step 12 defines a helper that extracts the minimum and maximum finite values from an array:

```
def finite_min_max(values):
    if values is None:
        return np.nan, np.nan

    arr = np.asarray(
        values,
        dtype=float
    ).reshape(-1)

    finite = arr[
        np.isfinite(arr)
    ]

    if finite.size == 0:
        return np.nan, np.nan

    return (
        float(np.min(finite)),
        float(np.max(finite))
    )
```

```
)
```

This function ignores NaN and infinite values and prevents invalid entries from distorting the reported extrema.

The effective Landau-Zener probability extrema are obtained from:

```
P_LZ_min, P_LZ_max = (  
    finite_min_max(  
        globals().get(  
            "effective_LZ_probability_for_rates",  
            None  
        )  
    )  
)
```

The raw weak coupling probability is selected preferentially from the unclipped Step 11 array:

```
P_WC_raw_array = globals().get(  
    "P_WC_raw_effective",  
    None  
)
```

If it is unavailable, effective_WC_probability_for_rates is used as a fallback:

```
if P_WC_raw_array is None:  
    P_WC_raw_array = (  
        globals().get(  
            "effective_WC_probability_for_rates",  
            None  
        )  
    )  
  
    P_WC_raw_source = "effective_WC_probability_for_rates"
```

This distinction is important because when Step 11 has been executed, the report can separately show

$$P_{WC}^{\text{raw}}(E)$$

and

$$P_{WC}^{\text{final}}(E).$$

The final weak coupling extrema are always calculated from the active downstream probability:

```
P_WC_final_min, \  
P_WC_final_max = (  
    finite_min_max(  
        active_probWC  
    )  
)
```

The report source is explicitly identified:

```
if use_threshold_corrected_WC:
    P_WC_final_source = (
        "threshold-corrected WC "
        "probability from Step 11"
    )
else:
    P_WC_final_source = (
        "Raw WC probability; "
        "no Step 11 correction applied"
    )
```

The microcanonical extrema are calculated from:

```
effective_LZ_rate
```

and:

```
active_rateWC
```

whereas the canonical extrema are calculated from:

```
k_LZ_canonical_s_inv
```

and:

```
active_k_WC_canonical
```

respectively.

Consequently, the final output reports the global minimum and maximum values of

$$P_{\text{LZ}}, P_{\text{WC}}^{\text{raw}}, P_{\text{WC}}^{\text{final}}, \\ k_{\text{LZ}}(E), k_{\text{WC}}(E),$$

and

$$k_{\text{LZ}}(T), k_{\text{WC}}(T).$$

4.12.8 Collection or reconstruction of intermediate spin–orbit couplings

Step 12 compiles the intermediate spin–orbit couplings introduced in Step 8 for inclusion in the final report:

```
intermediate_soc_records = []
```

The preferred source is the previously constructed P_LZ_intermediate dictionary:

```
if (
    "P_LZ_intermediate"
```

```

in globals()

and
isinstance(
    P_LZ_intermediate,
    dict
)

and
P_LZ_intermediate
):

```

For every intermediate entry, Step 12 stores

- its label;
- $|M_{S,\text{low}}|$;
- degeneracy;
- intermediate SOC;
- contributing row SOC values;
- source information.

```

intermediate_soc_records.append({
    "label":
        str(label),

    "abs_Ms_low":
        float(data["abs_Ms_low"]),

    "degeneracy":
        int(
            data.get(
                "degeneracy",
                1
            )
        ),

    "H_int_cm1":
        float(data["H_int_cm1"]),

    "H_rows_cm1":
        np.asarray(
            data.get(
                "H_rows_cm1",
                []
            ),
            dtype=float
        ).reshape(-1),

    "source":

```

```

    "P_LZ_intermediate "
    "from Step 8",
})

```

If the Step 8 intermediate dictionary is unavailable, Step 12 reconstructs the intermediate SOC_s from the M_S -resolved SOC matrix:

```

elif (
    "SOC_Ms_matrix_cm1"
    in globals()

    and
    "Ms_low"
    in globals()
):

```

The code verifies that the number of SOC-matrix rows matches the number of low-spin M_S values.

The unique absolute spin projections are then identified:

```

unique_abs_ms = sorted(
    set(
        round(
            abs(float(ms)),
            8
        )
        for ms in ms_low_for_intermediate
    )
)

```

For each $|M_S|$, all equivalent rows are collected. The norm of each SOC row is calculated as:

```

row_soc = float(
    np.sqrt(
        np.sum(
            np.abs(row) ** 2
        )
    )
)

```

or

$$H_i^{\text{row}} = \left[\sum_j |H_{ij}|^2 \right]^{1/2}.$$

The intermediate SOC is then obtained as the root-mean-square of the equivalent $+M_S$ and $-M_S$ row norms:

```

H_int_cm1 = float(
    np.sqrt(
        np.mean(
            row_soc_values ** 2
        )
    )
)

```

```
)  
)  
)
```

Thus,

$$H_{\text{int}} = \left[\frac{1}{n} \sum_{i=1}^n (H_i^{\text{row}})^2 \right]^{1/2}.$$

For a pair of symmetry-equivalent rows,

$$H_{\text{int}} = \sqrt{\frac{(H_{+M_S}^{\text{row}})^2 + (H_{-M_S}^{\text{row}})^2}{2}}.$$

This fallback reproduces the same intermediate-SOC definition used in Step 8 and ensures that the final report can still contain intermediate couplings even when the previously constructed dictionary is unavailable.

4.12.9 Construction of the final SPINKS report

The main final report begins with the project identity and output directory:

```
report.append(  
    "FINAL SPINKS OUTPUT"  
)  
  
report.append(  
    "=" * 72  
)  
  
report.append(  
    f"jobname = {jobname}"  
)  
  
report.append(  
    "final_output_dir = "  
    f"{final_output_dir}"  
)
```

The report is then divided into several scientific sections.

All M_S -specific SOC channels

When available, every complex SOC matrix element is written:

```
for key, value in \
```

```
H_SO_channels_cm1.items():
report.append(
    f"{key:45s} = "
    f"{value.real: .10f} "
    f"{value.imag:+.10f}i cm^-1 ; "
    f"|H| = {abs(value):.10f}"
)
```

Each channel therefore includes

$$\text{Re}(H_{\text{SO}}), \text{Im}(H_{\text{SO}}), |H_{\text{SO}}|.$$

If no channel-resolved matrix is available, the report explicitly states this.

Classified SOC matrix components

The classified components from earlier SOC analysis are reported separately:

```
if (
    "H_SO_intermediate_components_cm1"
    in globals()
    and
    H_SO_intermediate_components_cm1
):
```

These quantities are individual complex SOC matrix elements classified according to their spin-projection relationships. They are distinct from the intermediate SOC values, which are subsequently constructed from row norms and RMS combinations.

Intermediate SOC values

The report explains that intermediate SOC values are calculated separately for each unique $|M_{S,\text{low}}|$:

```
report.append(
    "Intermediate SOC values are "
    "calculated separately for "
    "each unique |Ms_low| value "
)
```

For each intermediate coupling, the report includes

$$|M_{S,\text{low}}|, g, H_{\text{int}},$$

where g is the number of equivalent low-spin rows. The contributing row SOC values and their data source are also written.

If no intermediate SOC values exist, the report notes that the workflow may be operating in effective-only SOC mode.

Effective SOC

The effective spin–orbit coupling is reported with its selected source:

```
report.append(  
    f"H_SO_cm = "  
    f"{fmt_scalar(soc_eff)} cm^-1 ; "  
    f"source = {soc_source}"  
)
```

The report additionally includes:

```
H_SO_ORCA_effective_cm
```

and:

```
H_SO_source
```

when available. This permits direct comparison between the final effective SOC used by SPINKS and the value obtained directly from the ORCA analysis.

Gradient quantities

The two gradient quantities used by the probability expressions are reported explicitly:

```
report.append(  
    "grad / "  
    "GRADIENT_GAP_NORM_EH_PER_BOHR = "  
    f"{fmt_scalar(grad)} Eh/Bohr ; "  
    f"source = {gradient_gap_source}"  
)  
report.append(  
    "gradmean / "  
    "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR = "  
    f"{fmt_scalar(gradmean)} Eh/Bohr ; "  
    f"source = {projected_gradient_mean_source}"  
)
```

Thus, the final report distinguishes

$$\Delta F_{\parallel}$$

from

$$\bar{g}.$$

The exact source variable is preserved, demonstrating which finalized quantity was passed to the NAST input.

Rotational data, reduced mass, and frequencies

The report records both reactant and MECP rotational or inertial arrays, the reduced mass, and the complete frequency arrays.

For the frequencies, both the source and number of modes are reported:

```
report.append(  
    f"freR count = "  
    f"{0 if freR is None else len(freR)}"  
)
```

and:

```
report.append(  
    f"freX count = "  
    f"{0 if freX is None else len(freX)}"  
)
```

This provides a direct consistency check of the reactant and crossing-state vibrational degrees of freedom.

Energy quantities

The following energy blocks are generated:

```
Electronic_energy_reactant_absolute  
Electronic_energy_MECP_absolute  
Electronic_barrier  
ZPE_reactant  
ZPE_MECP  
ZPE_corrected_barrier
```

Each is reported in

$$E_h, \text{ cm}^{-1}, \text{ kJ mol}^{-1}.$$

The report also records the source of the electronic barrier, reactant ZPE, and MECP ZPE.

Global probability and rate extrema

The final report contains a dedicated section summarizing the global minimum and maximum values of all principal effective probabilities and rates.

The weak coupling probability is divided into two distinct entries:

1. raw weak coupling probability;
2. final weak coupling probability used downstream.

This distinction allows the final output to show, for example,

$$\max P_{\text{WC}}^{\text{raw}} > 1,$$

while simultaneously showing that the final NAST-corrected probability remains within the accepted convention.

The report also identifies whether the weak coupling microcanonical and canonical extrema were obtained from the original or NAST-corrected results.

Canonical and interactive output

All console messages stored in `step12_console_lines` are appended to the report:

```
report.extend(  
    step12_console_lines  
)
```

Thus, the final file contains the complete canonical rate table and all interactive energy diagnostics printed during execution.

Every selected-energy record is additionally written as a key-value block:

```
for rec in debug_records:  
    for key, value in rec.items():  
        report.append(  
            f"{key} = {value}"  
        )
```

This preserves the exact numerical data independently of the formatted console representation.

The complete report is finally assembled and written:

```
final_report_text = (  
    "\n".join(report)  
    + "\n"  
)  
  
final_report_file = os.path.join(  
    final_output_dir,  
    f"{jobname}_FINAL_OUTPUT.txt"  
)  
  
write_text(  
    final_report_file,  
    final_report_text  
)
```

4.12.10 Validation of quantities required for NAST input

Before constructing the NAST files, Step 12 verifies that every required physical quantity is available:

```

if freR is None:
    raise RuntimeError(
        "Cannot write NAST input: "
        "freR was not found."
    )

```

Equivalent checks are performed for

- freX;
- inertR;
- inertX;
- reduced_mass_amu;
- H_SO_cm;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- PROJECTED_GRADIENT_MEAN_EH_PER_BOHR;
- electronic MECP barrier;
- ZPE-corrected MECP barrier.

These checks prevent incomplete or physically inconsistent NAST input files from being written.

4.12.11 Construction of the NAST input template

The complete NAST input is generated by:

```

def build_nast_interoperability_input(
    zpe_value,
    crossing_energy_hartree
):

```

The function produces three namelist sections:

```

&keys
&inputdata
&probability

```

NAST key block:

```

&keys
zpe = {int(zpe_value)}
printmore = .true.
&end

```

The zpe flag determines whether NAST applies its own zero-point-energy treatment. The printmore flag requests detailed NAST output.

Frequency and rotational data

The reactant and MECP frequency arrays are inserted as:

```
{format_namelist_array(  
  "freR",  
  reference_frequencies  
)}  
  
{format_namelist_array(  
  "freX",  
  crossing_frequencies  
)}
```

The rotational or inertial quantities are similarly included:

```
{format_namelist_array(  
  "inertR",  
  reference_inertias  
)}  
  
{format_namelist_array(  
  "inertX",  
  crossing_inertias  
)}
```

These arrays define the rovibrational properties used by NAST to construct reactant densities of states and crossing-state sums of states.

Energy reference and grid

The reactant energy is set to zero:

```
enR = 0.0
```

The crossing energy is supplied through:

```
enX = {crossing_energy_hartree:.15f}
```

The energy grid is defined by:

```
maxn = {export_maximum_bin_index}  
energy_step_cm1 = {export_energy_step_cm1:.8g}
```

and the canonical temperature interval by:

```
T1 = {export_temperature_min_K:.8g}  
T2 = {export_temperature_max_K:.8g}
```

Thus, the common energy grid contains

$$E_n = n\Delta E, n = 0, \dots, \text{maxn}.$$

Probability block

The quantities required by the NAST spin-crossing probability expressions are written as:

```
&probability
redmass = {redmass:.15f}
soc = {soc_eff:.15f}
grad = {gradient_gap:.15e}
projected_gradient_mean_value = {projected_gradient_mean_value:.15e}
&end
```

These correspond to

$$\mu, H_{\text{SO}}, \Delta F_{\parallel}, \bar{g}.$$

The final NAST template is therefore:

```
def build_nast_interoperability_input(
    zpe_value,
    crossing_energy_hartree
):
    return f""""&keys
zpe = {int(zpe_value)}
printmore = .true.
&end

&inputdata
{format_namelist_array("freR", reference_frequencies)}

{format_namelist_array("freX", crossing_frequencies)}

{format_namelist_array("inertR", reference_inertias)}
{format_namelist_array("inertX", crossing_inertias)}

enR = 0.0
enX = {crossing_energy_hartree:.15f}

maxn = {export_maximum_bin_index}
energy_step_cm1 = {export_energy_step_cm1:.8g}
T1 = {export_temperature_min_K:.8g}
T2 = {export_temperature_max_K:.8g}
&end

&probability
redmass = {redmass:.15f}
soc = {soc_eff:.15f}
grad = {gradient_gap:.15e}
projected_gradient_mean_value = {projected_gradient_mean_value:.15e}
&end
""""
```

4.12.12 Generation of the two NAST barrier conventions

Two separate NAST input files are generated.

Electronic-barrier input with **zpe = 1**:

```
nast_export_zpe1_text = build_nast_interoperability_input(  
    zpe_value=1,  
    crossing_energy_hartree=Ele_barrier_Eh  
)
```

This file uses

$$E_X = V_{\text{MECP}}^{\text{elec}},$$

and enables NAST zero-point-energy treatment:

```
zpe = 1
```

The barrier supplied to enX is therefore the electronic barrier,

$$V_{\text{MECP}}^{\text{elec}} = E_{\text{MECP}}^{\text{elec}} - E_R^{\text{elec}}.$$

NAST subsequently accounts for the reactant and crossing-state ZPEs from the supplied frequency arrays.

Externally ZPE-corrected input with **zpe = 0**:

```
nast_export_zpe0_text = build_nast_interoperability_input(  
    zpe_value=0,  
    crossing_energy_hartree=ZPE_corrected_barrier_Eh  
)
```

This file uses

$$E_X = V_{\text{MECP}}^0,$$

where

$$V_{\text{MECP}}^0 = V_{\text{MECP}}^{\text{elec}} + \text{ZPE}_X - \text{ZPE}_R.$$

The internal NAST ZPE correction is disabled:

```
zpe = 0
```

This avoids applying the zero-point-energy correction twice.

The two approaches can be expressed as

$$zpe = 1, enX = V_{MECP}^{elec}$$

and

$$zpe = 0, enX = V_{MECP}^0$$

respectively.

When the SPINKS implementation and the externally calculated ZPE correction use equivalent conventions, the resulting kinetic predictions should be consistent.

The two files are assigned descriptive names:

```
nast_export_zpe1_file = os.path.join(
    final_output_dir,
    f"{jobname}_NAST_"
    "zpe1_electronic_barrier.inp"
)
nast_export_zpe0_file = os.path.join(
    final_output_dir,
    f"{jobname}_NAST_"
    "zpe0_ZPE_corrected_barrier.inp"
)
```

Both files are then written to disk.

4.12.13 Optional cluster upload and global export

When a remote output directory and SFTP connection are available, Step 12 uploads

- the final SPINKS report;
- the $zpe = 1$ NAST input;
- the $zpe = 0$ NAST input.

```
sftp.put(
    final_report_file,
    posixpath.join(
        remote_final_output_dir,
        os.path.basename(
            final_report_file
        )
    )
)
```

Equivalent transfer commands are used for both NAST files.

Failure to upload the files produces a warning but does not invalidate the locally generated outputs.

Finally, the most important Step 12 quantities are exported:

```
globals().update({
    "final_output_dir":
        final_output_dir,

    "final_report_file":
        final_report_file,

    "nast_export_zpe1_file":
        nast_export_zpe1_file,

    "nast_export_zpe0_file":
        nast_export_zpe0_file,

    "debug_records_step12":
        debug_records,

    "ZPE_corrected_barrier_Eh_final":
        ZPE_corrected_barrier_Eh,

    "Electronic_barrier_Eh_final":
        Ele_barrier_Eh,
})
```

These exported variables preserve the final paths, interactive diagnostic records, and finalized barrier energies for any later workflow stage or user analysis.

4.12.14 Summary of the step 12 algorithm

Step 12 is the final consolidation and external-interface stage of SPINKS. It first determines whether the original weak coupling quantities or the NAST-corrected quantities from Step 11 should be used:

$$(P_{WC}, N_{WC}, k_{WC})_{\text{active}} = \begin{cases} (P_{WC}, N_{WC}, k_{WC})_{\text{original}}, & \text{no Step 11 correction,} \\ (P_{WC}, N_{WC}, k_{WC})_{\text{NAST}}, & \text{Step 11 correction active.} \end{cases}$$

The selected weak coupling data are used consistently in both the canonical summary and the interactive microcanonical diagnostic.

For each user-selected energy, the nearest grid point is identified as

$$i = \text{round}\left(\frac{E}{\Delta E}\right),$$

and the corresponding quantities are reported:

$$\begin{aligned} &\rho_R(E), \rho_X(E - E_{\text{MECP}}), \rho_{TP}(E - E_{\text{MECP}}), \\ &P_{\text{LZ}}(E), P_{\text{WC}}(E), \\ &N_{\text{LZ}}(E), N_{\text{WC}}(E), \\ &k_{\text{LZ}}(E), k_{\text{WC}}(E). \end{aligned}$$

The final report consolidates the complete set of physically important workflow quantities, including

- all M_S -specific complex SOC channels;
- classified SOC components;
- intermediate SOC values;
- effective SOC;
- $\Delta F_{\parallel}$;
- mean gradient;
- rotational or inertial data;
- reduced mass;
- reactant and MECP frequencies;
- absolute electronic energies;
- electronic MECP barrier;
- reactant and MECP ZPEs;
- ZPE-corrected MECP barrier;
- global minimum and maximum probabilities;
- global minimum and maximum microcanonical rates;
- global minimum and maximum canonical rates;
- canonical temperature table;
- user-selected microcanonical diagnostics.

The electronic barrier is defined as

$$V_{\text{MECP}}^{\text{elec}} = E_{\text{MECP}}^{\text{elec}} - E_R^{\text{elec}},$$

and the ZPE-corrected barrier is

$$V_{\text{MECP}}^0 = V_{\text{MECP}}^{\text{elec}} + \text{ZPE}_{\text{MECP}} - \text{ZPE}_R.$$

Intermediate SOC values are preferentially imported from Step 8. When necessary, they are reconstructed from the M_S -resolved SOC matrix using

$$H_i^{\text{row}} = \sqrt{\sum_j |H_{ij}|^2},$$

followed by the RMS combination

$$H_{\text{int}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (H_i^{\text{row}})^2}.$$

The NAST grid size is determined from the maximum energy selected in Step 7:

$$\text{maxn} = \left\lceil \frac{E_{\text{max}}}{\Delta E} \right\rceil.$$

Two complete NAST inputs are produced:

$$\text{zpe} = 1, \text{enX} = V_{\text{MECP}}^{\text{elec}},$$

and

$$\text{zpe} = 0, \text{enX} = V_{\text{MECP}}^0.$$

The first allows NAST to calculate the ZPE correction from the supplied frequencies, whereas the second supplies the externally calculated ZPE-corrected barrier directly and disables additional NAST ZPE treatment.

The overall Step 12 workflow can therefore be summarized as:

Results from Steps 1–11

↓
Select active WC source
Original or NAST-corrected
↓
Print canonical
LZ/WC rate table
↓
Interactive energy selection
↓
Report DOS, probabilities,
numbers of states, and
microcanonical rates
↓
Collect final SOC,
gradient, mass, frequency,
rotational, and energy data
↓
Determine global minimum
and maximum probabilities
and rates
↓
Construct FINAL_OUTPUT.txt
↓
Build NAST input 1
zpe = 1
electronic barrier
↓
Build NAST input 2
zpe = 0
ZPE-corrected barrier
↓
Save locally and,
when available,
upload to cluster

Step 12 therefore converts the complete internal SPINKS data structure into a transparent and reproducible final deliverable. It preserves the provenance of every important physical quantity, records whether weak coupling correction was required, enables direct energy-resolved kinetic inspection, and produces ready-to-run NAST input files for independent validation of the SPINKS microcanonical and canonical rate calculations.

References

- (1) Neese, F. Software update: The ORCA program system, version 5.0. *WIREs Comput. Mol. Sci.* **2022**, 12, e1606.
- (2) Dergachev, V. D.; Rooein, M.; Dergachev, I. D.; Lykhin, A. O.; Mauban, R. C.; Varganov, S. A. NAST: Nonadiabatic Statistical Theory Package for Predicting Kinetics of Spin-Dependent Processes. *Top. Curr. Chem.* **2022**, 380, 15.